UNIWERSYTET MORSKI W GDYNI
WYDZIAŁ ELEKTRYCZNY
Zakład Inżynierii Systemów
Wydział Informatyki

Nr ewidencyjny:.....
Data złożenia pracy:.....

PRACA DYPLOMOWA INŻYNIERSKA

Temat: Implementacja webowego systemu zarządzania działalnością gastronomiczną z użyciem Spring i React

	Dyplomant	Dyplomant
Imię i nazwisko:	Igor Kohsin	Oskar Krupa
Numer albumu:	48441	48447
Specjalność:	Aplikacje Internetowe i Mobilne	
Promotor:	dr hab. Ewa Ratajczak-Ropel, prof. UMG	
Recenzent	Prof. dr hab. inż. Ireneusz Czarnowski	
Ocena:		
Wynik studiów:		
Data egzaminu:		

Recenzent: ..... Promotor: ..... Dziekan: .....

Wyrażam zgodę / nie wyrażam zgody na

udostępnianie mojej pracy dyplomowej

....
data podpis podpis

Gdynia, 2026

……..... Gdynia, dnia r.

Imię i Nazwisko

OŚWIADCZENIE

Świadomy/a odpowiedzialności prawnej oświadczam, że złożona praca inżynierska/magisterska* pt.: *Implementacja webowego systemu zarządzania działalnością gastronomiczną z użyciem Spring i React* została napisana przeze mnie samodzielnie.

Równocześnie oświadczam, że w pracy wykorzystano tylko cytowaną literaturę, a więc praca nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz. U. 1994, nr 24, poz. 83) oraz dóbr osobistych chronionych prawem cywilnym.

Ponadto praca nie zawiera informacji i danych uzyskanych w sposób nielegalny i nie była wcześniej przedmiotem innych procedur urzędowych związanych z uzyskaniem dyplomów lub tytułów zawodowych uczelni wyższej.

Oświadczam ponadto, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną na płycie CD.

Na podstawie art. 75 §2 kodeksu postępowania administracyjnego wnoszę o odebranie tego oświadczenia jako dowodu prawdziwości okoliczności w nim podatnych, przy czym jestem świadomy odpowiedzialności karnej z art. 233 §1 i §6 k.k. za złożenie fałszywego oświadczenia.

……

podpis

* niepotrzebne skreślić

……..... Gdynia, dnia r.

Imię i Nazwisko

OŚWIADCZENIE

Świadomy/a odpowiedzialności prawnej oświadczam, że złożona praca inżynierska/magisterska* pt.: *Implementacja webowego systemu zarządzania działalnością gastronomiczną z użyciem Spring i React* została napisana przeze mnie samodzielnie.

Równocześnie oświadczam, że w pracy wykorzystano tylko cytowaną literaturę, a więc praca nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz. U. 1994, nr 24, poz. 83) oraz dóbr osobistych chronionych prawem cywilnym.

Ponadto praca nie zawiera informacji i danych uzyskanych w sposób nielegalny i nie była wcześniej przedmiotem innych procedur urzędowych związanych z uzyskaniem dyplomów lub tytułów zawodowych uczelni wyższej.

Oświadczam ponadto, że niniejsza wersja pracy jest identyczna z załączoną wersją elektroniczną na płycie CD.

Na podstawie art. 75 §2 kodeksu postępowania administracyjnego wnoszę o odebranie tego oświadczenia jako dowodu prawdziwości okoliczności w nim podatnych, przy czym jestem świadomy odpowiedzialności karnej z art. 233 §1 i §6 k.k. za złożenie fałszywego oświadczenia.

……

podpis

* niepotrzebne skreślić

SPIS TREŚCI

Wykaz ważniejszych oznaczeń i skrótów 7
Wstęp 8
1. Tworzenie aplikacji internetowych 9
1.1 Aplikacje internetowe i ich projektowanie (Igor Kohsin, Oskar Krupa) 9
1.2 Frontend (Oskar Krupa) 10
1.2.1 Technologie strony frontend 10
1.2.2 Frontend na przykładzie React 11
1.2.3 Ekosystem React 13
1.2.4 Architektura części frontend aplikacji React 14
1.2.5 Bezpieczeństwo warstwy frontend 16
1.3 Backend (Igor Kohsin) 18
1.3.1 Framework Spring 19
1.3.2 Ekosystem Spring 20
1.3.3 Architektura aplikacji Spring 21
1.3.4 Bezpieczeństwo aplikacji Spring 22
1.4 Baza danych (Igor Kohsin) 23
1.5 Testowanie (Igor Kohsin, Oskar Krupa) 24
1.6 Współpraca frontend – backend (Oskar Krupa, Igor Kohsin) 25
2. Projekt aplikacji (Igor Kohsin, Oskar Krupa) 27
2.1 Analiza wymagań 27
2.1.1 Wymagania funkcjonalne 27
2.1.2 Wymagania pozafunkcjonalne 29
2.2 Schemat bazy danych 30
2.3 Projekt interfejsu użytkownika 32
2.4 Wybór technologii i narzędzii 39
3. Implementacja warstwy backend (Igor Kohsin) 43
3.1 Konfiguracja środowiska projektowego 43
3.2 Struktura systemu 45
3.3 Model przechowywania danych 47
3.3.1 Encje 47
3.3.2 Command i DTO 49
3.4 Dostęp do danych 51
3.5 Warstwa prezentacji dla klienta zewnętrznego 53
3.6 Logika biznesowa 55
3.7 Klasy pomocnicze 58
3.7.1 Mappery 58

3.7.2 Obsługa wyjątków ... 59
3.8 Bezpieczeństwo ... 62
3.8.1 Konfiguracja bezpieczeństwa ... 62
3.8.2 Autoryzacja ... 64
3.8.3 Uwierzytelnianie z wykorzystaniem JWT ... 66
3.9 Testowanie ... 68
3.9.1 Testy jednostkowe ... 68
3.9.2 Testy integracyjne ... 74
4. Implementacja warstwy frontend (Oskar Krupa) ... 78
4.1 Konfiguracja kodu aplikacji ... 78
4.2 Struktura aplikacji ... 80
4.3 Punkt wejściowy aplikacji ... 81
4.4 Schematy danych ... 82
4.5 Komunikacja z API ... 83
4.5.1 Instancja Axios ... 83
4.5.2 Funkcje dostępu do danych ... 83
4.5.3 Mutacje z optymistycznymi aktualizacjami ... 85
4.6 Autoryzacja i ochrona tras ... 86
4.6.1 Przepływ autoryzacji ... 87
4.6.2 Funkcja autoryzacji ... 88
4.6.3 Funkcja użytkownika ... 89
4.6.4 Trasy chronione ... 90
4.7 Struktura tras ... 91
4.8 Strony aplikacji ... 93
4.8.1 Strona magazynu ... 93
4.8.2 Strona główna ... 94
4.9 Stylizacja i motyw ... 96
4.10 Komponenty strony głównej ... 97
4.11 Responsywność ... 99
4.11.1 Adaptacyjne tabele ... 99
4.11.2 Punkty przerwania ... 100
4.12 Testowanie ... 100
4.12.1 Konfiguracja testów ... 101
4.12.2 Selektory ... 102
4.12.3 Struktura testów ... 103
4.12.4 Uruchomienie testów ... 104
Podsumowanie i wnioski końcowe ... 106

Bibliografia.....109
Streszczenie.....113
Abstract.....114

WYKAZ WAŻNIEJSZYCH OZNACZEŃ I SKRÓTÓW

API – Application Programming Interface

CRUD – Create, Read, Update, Delete

ERP – Enterprise Resource Planning

JSON – JavaScript Object Notation

JPA – Java Persistence API

JWT – JSON Web Token

ORM – Object-Relational Mapping

POS – Point of Sale

REST – Representational State Transfer

Spring Data JPA – Spring Data Java Persistence API

SQL – Structured Query Language

HTTP – HyperText Transfer Protocol

CSS – Cascading Style Sheets

HTML – HyperText Markup Language

JS – JavaScript

SoC – Separation of Concerns

JSX – JavaScript XML

UI – User Interface

XSS – Cross-Site Scripting

DOM – Document Object Model

CSRF – Cross-Site Request Forgery

CORS – Cross-Origin Resource Sharing

WSTĘP

W dzisiejszych czasach branża gastronomiczna wymaga coraz bardziej zaawansowanych narzędzi do zarządzania zasobami, pracownikami oraz procesami biznesowymi. Tradycyjne metody, oparte na ręcznej dokumentacji i systemach działających lokalnie, często okazują się niewystarczające w dynamicznym środowisku gastronomicznym. W odpowiedzi na te wyzwania powstała koncepcja webowego systemu zarządzania działalnością gastronomiczną, którego funkcjonalność odpowiada systemom ERP (Enterprise Resource Planning). Tego typu rozwiązania umożliwiają centralizację danych, automatyzację procesów oraz usprawnienie komunikacji między działami, co znacząco wpływa na efektywność zarządzania przedsiębiorstwem.

Celem pracy było zaprojektowanie oraz implementacja systemu webowego (aplikacji internetowej) do wspomagania działalności gastronomicznej. System ma ułatwić zarządzanie zakładem gastronomicznym, w tym pracownikami, magazynem i transakcjami. Odbiorcami systemu, dla którego przyjęto nazwę "GastroM" są przede wszystkim managerowie zakładów gastronomicznych, a więc osoby odpowiedzialne za codzienne funkcjonowanie zakładów gastronomicznych.

Praca została napisana przez dwóch autorów i składa się z 4 rozdziałów oraz Podsumowania i wniosków końcowych. W rozdziale pierwszym przedstawiono zagadnienia związane z aplikacjami internetowymi, ich projektowaniem i tworzeniem. Drugi rozdział zawiera projekt aplikacji GastroM, między innymi wybór technologii, opis architektury systemu oraz bazy danych, a także opis narzędzi zaplanowanych do wykorzystania przy tworzeniu aplikacji. Implementację warstwy frontend aplikacji, gdzie przedstawiono bibliotekę oraz ekosystem React, wraz z omówieniem architektury aplikacji, jak również zagadnienia związane z bezpieczeństwem w kontekście frontend opisał Oskar Krupa w rozdziale trzecim. W rozdziale 4 dotyczącym implementacji warstwy backend zostały przedstawione zagadnienia związane z frameworkiem Spring, wraz z opisem jego ekosystemu, architektury backend, jak również zagadnień dotyczących bezpieczeństwa aplikacji Spring, autorem całego rozdziału jest Igor Kohsin. Wstęp i podsumowanie zostały opracowane wspólnie przez obu autorów.

1. TWORZENIE APLIKACJI INTERNETOWYCH

Tworzenie aplikacji internetowych stanowi ważny obszar współczesnej informatyki integrując zaawansowane technologie, zasady projektowania, implementacji i wdrażania systemów.

W kolejnych podrozdziałach przedstawiono kluczowe zagadnienia związane z procesem tworzenia tego typu rozwiązań. Opisano tu podstawowe pojęcia dotyczące aplikacji internetowych oraz elementy ich projektowania. Omówiono część frontend, w tym bibliotekę React. Przedstawiono część backend, obejmującą charakterystykę frameworka Spring, elementy jego ekosystemu, architekturę aplikacji serwerowych oraz zagadnienia związane z bezpieczeństwem backend. W treści rozdziału opisano również warstwę bazodanową wraz ze sposobami przechowywania i zarządzania danymi, metody testowania aplikacji oraz zagadnienia dotyczące współpracy frontend i backend.

1.1 Aplikacje internetowe i ich projektowanie (Igor Kohsin, Oskar Krupa)

Aplikacje internetowe stanowią istotny element współczesnego Internetu, umożliwiając interakcję między użytkownikami a zasobami cyfrowymi w sposób dynamiczny i responsywny. Aplikacje internetowe działają w środowisku przeglądarek internetowych dzięki czemu nie wymagają od użytkowników instalowania dedykowanego oprogramowania na urządzeniach końcowych. Zapewniają uniwersalny dostęp do zasobów cyfrowych niezależnie od lokalizacji użytkownika, typu urządzenia oraz wykorzystywanego systemu operacyjnego [1].

Współczesne aplikacje internetowe charakteryzują się architekturą, która zakłada logiczne oddzielenie warstw, na przykład prezentacji, logiki biznesowej oraz dostępu do danych. Takie podejście umożliwia nie tylko optymalizację procesu przetwarzania informacji, ale również ułatwia rozwój systemu poprzez modularność i elastyczność implementacji poszczególnych komponentów. Aplikacje takie są również określane jako aplikacje wielowarstwowe [2, 3, 4].

Proces projektowania aplikacji internetowych zaczyna się od dogłębnej analizy kontekstu użytkowania oraz identyfikacji potrzeb odbiorców, co umożliwia sformułowanie wytycznych projektowych. Istotnym elementem tej analizy jest określenie funkcji, jakie system ma realizować, co wpływa na koncepcję modelu danych oraz mechanizmów przetwarzania informacji. Projektanci systemu muszą uwzględnić zarówno wymagania dotyczące operacyjności, funkcjonalności, szybkości działania i

reakcji na zdarzenia, jak i te związane z bezpieczeństwem danych oraz ochroną prywatności użytkowników [4].

Podczas projektowania aplikacji internetowych niezbędne jest uwzględnienie całego cyklu życia oprogramowania. Obejmuje on etapy od analizy wymagań, przez projektowanie i implementację, aż po testowanie, wdrożenie oraz późniejsze utrzymanie i rozwój systemu [1].

Aplikacje internetowe opierają się na współpracy dwóch kluczowych części nazywanych również warstwami: frontend, odpowiedzialnej za interfejs użytkownika oraz backend, realizującej logikę działania systemu. W kolejnych podrozdziałach omówiono technologie oraz rozwiązania wykorzystywane do budowy obu tych części.

1.2 Frontend (Oskar Krupa)

Warstwa frontend stanowi integralny element współczesnych aplikacji internetowych, odpowiedzialny za wizualną reprezentację aplikacji oraz interakcję z użytkownikiem. Komunikuje się ona najczęściej z warstwą backend za pośrednictwem dedykowanych interfejsów programistycznych API (Application Programming Interface), wykorzystując protokół HTTP (Hypertext Transfer Protocol) w celu wysyłania oraz odbierania danych. Jakość wykonania tej warstwy decyduje w dużej mierze o końcowych wrażeniach użytkowników, wpływając na intuicyjność, responsywność oraz poziom bezpieczeństwa aplikacji.

1.2.1 Technologie strony frontend

Obecnie oprócz podstawowych technologii, czyli HTML (Hypertext Markup Language), CSS (Cascading Style Sheets) oraz JavaScript, dostępnych jest wiele bibliotek i frameworków służących do tworzenia strony frontend aplikacji internetowych. Do popularnych rozwiązań należą m.in. React JS, Angular, Vue.js, Svelte czy Ember.js. Każda z nich oferuje mechanizmy umożliwiające tworzenie nowoczesnych, interaktywnych i responsywnych interfejsów użytkownika, a także zarządzanie stanem aplikacji i integrację z częścią backend poprzez protokoły sieciowe, takie jak HTTP czy WebSocket.

Spośród wielu dostępnych bibliotek oraz technologii służących do budowania warstwy frontend, szczególne znaczenie posiada React JS opisany dokładniej w podrozdziale 1.2.2. Popularność tego rozwiązania potwierdzają m.in. wyniki ankiety „State of JavaScript 2024” [5], zgodnie z którymi aż 67% respondentów wskazało, że

korzysta z React JS w codziennej pracy. Ze względu na szerokie zastosowanie oraz ciągły rozwój biblioteki, została ona dokładniej opisana oraz wykorzystana do realizacji praktycznej części pracy.

1.2.2 Frontend na przykładzie React

Warstwa klienta, nazywana również stroną frontend lub po prostu frontend, stanowi fundamentalny element w procesie tworzenia nowoczesnych aplikacji internetowych. Współczesne aplikacje internetowe opierają się najczęściej na architekturze klient-serwer, w której frontend komunikuje się z backendem poprzez zdefiniowane interfejsy API, przesyłając żądania i odbierając odpowiedzi.

React jako zaawansowana biblioteka JavaScript opracowana przez firmę Meta (dawniej Facebook), umożliwia konstruowanie dynamicznych oraz responsywnych interfejsów użytkownika poprzez zastosowanie wielokrotnie wykorzystywalnych komponentów [6]. Tworzenie części aplikacji w React wymaga użycia środowiska Node.js. Node.js jest to otwarte, wieloplatformowe środowisko uruchomieniowe JavaScript, które pozwala na wykonywanie kodu JS poza przeglądarką, głównie po stronie serwera, dzięki czemu można tworzyć szybkie, skalowalne aplikacje sieciowe i API.

Podstawą pracy aplikacji React JS jest sama przeglądarka internetowa, natomiast Node.js pełni rolę środowiska uruchomieniowego dla narzędzi developerskich. Obejmuje to m.in. tłumaczenie kodu źródłowego na postać zrozumiałą dla przeglądarki (transpilację) przy użyciu narzędzi takich jak Babel, a także łączenie plików i optymalizację zasobów (ang. bundling) realizowaną za pomocą narzędzi, takich jak Webpack czy Vite [6, 7]. W procesie tworzenia aplikacji frontend wykorzystywane są również narzędzia do automatyzacji konfiguracji środowiska, które upraszczają uruchomienie i budowę projektu. Do niedawna popularne było Create React App, obecnie stopniowo wycofywane na rzecz lżejszych i bardziej elastycznych rozwiązań, takich jak wcześniej wspomiane Vite, które oferują szybsze uruchamianie projektów oraz lepszą obsługę nowoczesnych funkcji języka.

Projektując strukturę aplikacji internetowych istotne jest stosowanie podejścia opartego na zasadzie separacji odpowiedzialności (ang. *Separation of Concerns*, SoC). Zasada ta zakłada podział systemu na odrębne części, z których każda odpowiada za konkretny zakres funkcjonalny. W kontekście takich aplikacji, jak te budowane w technologii React, oznacza to logiczne rozdzielenie warstw interfejsu użytkownika, logiki

biznesowej, obsługi komunikacji z serwerem oraz zarządzania stanem aplikacji [6, 7]. Architektura aplikacji React po stronie klienta została dokładniej opisana w podrozdziale 1.2.4.

Jedną z koncepcji projektowania interfejsów użytkownika jest Atomic Design, zaproponowana przez Brada Frosta [8]. Zakłada ona podział komponentów wizualnych na pięć hierarchicznych poziomów:

1. Atomy (atoms) – najmniejsze, niepodzielne elementy interfejsu, np. przycisk lub pole tekstowe,
2. Molekuły (molecules) – proste grupy komponentów, np. formularz logowania zawierający pole i etykietę,
3. Organizmy (organisms) – złożone sekcje, łączące wiele molekuł, np. nagłówek strony,
4. Szablony (templates) – struktury określające układ komponentów na stronie,
5. Strony (pages) – kompletne widoki zawierające konkretne dane.

Takie podejście pozwala tworzyć interfejs w sposób systematyczny, z możliwością ponownego wykorzystania zdefiniowanych elementów w różnych częściach aplikacji. Jest obecnie popularne w aplikacjach webowych i mobilnych, np. React czy Vue.

Alternatywnym i coraz częściej stosowanym podejściem jest struktura oparta na funkcjonalnościach (ang. *feature-based structure*). W tym modelu kod źródłowy organizowany jest według poszczególnych katalogów funkcjonalnych aplikacji (features), na przykład users, orders, auth. Każdy taki katalog może zawierać komponenty, logikę, style i testy związane wyłącznie z daną funkcją aplikacji. Dzięki temu możliwe jest ograniczenie zależności pomiędzy poszczególnymi modułami oraz łatwiejsze zarządzanie kodem. W praktyce w aplikacjach korzystających z React wyróżnia się również tzw. komponenty wspólne (określane często jako komponenty globalne lub współdzielone), czyli elementy interfejsu używane w wielu miejscach aplikacji – przykładowo: przyciski, okna dialogowe (modalne), nagłówki. Zazwyczaj umieszcza się je w osobnych katalogach, takich jak shared, common lub ui, co sprzyja porządkowi w strukturze projektu. Dodatkowo, zgodnie z zasadą separacji odpowiedzialności, komunikację z zewnętrznymi interfejsami API realizuje się za pomocą wydzielonej warstwy usługowej (ang. *service layer*), w której definiuje się funkcje odpowiedzialne za wysyłanie zapytań do serwera oraz przetwarzanie odpowiedzi [9, 10].

1.2.3 Ekosystem React

React stanowi obecnie jedną z najbardziej rozbudowanych i zróżnicowanych platform do tworzenia warstwy prezentacji w aplikacjach internetowych. Choć sam w sobie jest biblioteką służącą do budowania interfejsów użytkownika, to wokół niego powstał cały ekosystem — zbiór powiązanych narzędzi, bibliotek oraz praktyk programistycznych — umożliwiający tworzenie złożonych i nowoczesnych aplikacji internetowych [11].

Rozwój tego ekosystemu można przypisać przede wszystkim elastyczności i otwartości Reacta, która umożliwia jego integrację z wieloma dodatkowymi rozwiązaniami. Dzięki temu deweloperzy mają swobodę doboru odpowiednich narzędzi zgodnych z potrzebami projektu, co prowadzi do dynamicznego rozwoju nowych bibliotek i narzędzi odpowiadających na różnorodne wymagania, takie jak obsługa formularzy, zarządzanie stanem, czy optymalizacja wydajności [12].

W kontekście zarządzania stanem aplikacji szczególną rolę odgrywają biblioteki, które umożliwiają kontrolę i synchronizację danych w złożonych środowiskach, takie jak Jotai, Zustand czy Redux. Rozwiązania te pozwalają na śledzenie przepływu danych i aktualizacji na poziomie komponentów, co umożliwia precyzyjne diagnozowanie problemów i szybkie reagowanie na nieprzewidziane wcześniej zmiany w zachowaniu aplikacji [13].

Ważną częścią ekosystemu stały się również narzędzia dedykowane obsłudze formularzy, takie jak React Hook Form czy Formik. Potrafią one automatyzować walidację i przetwarzanie danych wejściowych bez konieczności rozbudowywania logiki wewnątrz komponentów. Implementacja tego typu rozwiązań zapewnia spójne i powtarzalne mechanizmy obsługi wyjątków i błędów użytkownika, co jest szczególnie istotne w aplikacjach przetwarzających dane wrażliwe lub często modyfikowane przez różne działy organizacji [13].

Współcześnie coraz większy nacisk kładzie się na poprawę wydajności oraz komfortu interakcji użytkowników. W ramach ekosystemu React stosowane są techniki optymalizacyjne tzw. *code splitting*, które pozwalają na dzielenie kodu na mniejsze części (chunks) i ładowanie tylko tych, które są aktualnie potrzebne. Dzięki temu użytkownik otrzymuje dostęp do kluczowych funkcji szybciej niż w przypadku ładowania całej aplikacji jednocześnie [14].

W kontekście szeroko pojętej optymalizacji coraz większe znaczenie mają również biblioteki umożliwiające kompresję i cache'owanie danych, a także narzędzia monitorujące czas odpowiedzi oraz zużycie zasobów przez poszczególne komponenty. React wykorzystuje także mechanizmy memoizacji, które pozwalają ograniczać liczbę zbędnych renderowań komponentów. Polegają one na zapamiętywaniu wcześniejszych wyników obciążających operacji i ponownym ich użyciu w sytuacji, gdy przekazywane dane nie uległy zmianie [14]. Warto też wspomnieć o inicjatywach związanych z rozwijaniem technik renderowania po stronie serwera (Server - Side Rendering, SSR) oraz generowania statycznych stron (Static Site Generation, SSG). Popularne frameworki takie jak Next.js i Gatsby integrują te podejścia z biblioteką React, oferując mechanizmy poprawiające wydajność, wspierające SEO (Search Engine Optimization) oraz zwiększające dostępność aplikacji na różnych urządzeniach [15].

Odrębnym obszarem ekosystemu React są biblioteki ukierunkowane na przygotowanie gotowych komponentów interfejsu użytkownika w ramach określonych standardów. Przykładami są Material UI czy Chakra UI, które oferują gotowe zestawy komponentów zgodnych z popularnymi systemami projektowymi [16, 17].

Podsumowując, dzięki pracy społeczności oraz producentów rozwiązań komercyjnych, ekosystem React nieustannie się rozwija. Dostarcza on szeroki zestaw narzędzi dostosowanych do różnorodnych potrzeb projektowych – od zarządzania stanem, przez testowanie, aż po optymalizację i stylizację interfejsów. W najbliższej przyszłości można spodziewać się dalszych udoskonaleń w zakresie wydajności, integracji z usługami chmurowymi oraz wspierania nowoczesnych modeli wdrażania aplikacji. React.js, mimo że pozostaje biblioteką do tworzenia interfejsów, odgrywa obecnie również ważną rolę w kształtowaniu architektury współczesnych aplikacji internetowych.

1.2.4 Architektura części frontend aplikacji React

Warstwowa architektura nowoczesnych aplikacji internetowych umożliwia jasny podział odpowiedzialności poszczególnych elementów systemu w myśl zasady *separation of concerns*, opisanej w podrozdziale 1.2.1. W przypadku aplikacji tworzonych w oparciu o bibliotekę React szczególnie istotne jest odpowiednie modelowanie struktury komponentów, które współpracują ze sobą w układzie hierarchicznym. Komponenty nadrzędne przekazują dane i funkcje obsługi zdarzeń do

komponentów podrzędnych za pomocą mechanizmu właściwości (props), co tworzy jednokierunkowy przepływ danych (ang. unidirectional data flow) [18].

Komponenty w React mogą być klasowe lub funkcyjne. Komponenty funkcyjne są łatwiejsze do napisania i czytelniejsze. Dostarczają tych samych możliwości co klasy dzięki możliwości wykorzystania funkcji nazywanych hookami. Hooki rozszerzają możliwości komponentów funkcyjnych np. o zarządzanie stanem [19].

W praktyce projektowej komponenty React dzielą się także na różne role pełnione w strukturze aplikacji. Podstawowy podział obejmuje komponenty prezentacyjne oraz komponenty kontenerowe.

Komponenty prezentacyjne (ang. presentational components) odpowiadają za wyświetlanie interfejsu użytkownika oraz prezentację danych koncentrując się na wyglądzie aplikacji. Otrzymują dane i funkcje zwrotne poprzez parametry (props) od komponentów nadrzędnych i renderują widok użytkownika. W aplikacjach korzystających z React są najczęściej implementowane jako funkcje wykorzystujące składnię JSX (JavaScript XML). JSX jest rozszerzeniem składni języka JavaScript umożliwiającym osadzanie w nim kodu HTML. Komponenty te komunikują się głównie z komponentami kontenerowymi, od których otrzymują dane i funkcje obsługi zdarzeń. Stanowią najniższą warstwę architektury, wykochrzystującą rezultaty działania warstw wyższych. W przypadku złożonych interfejsów są budowane hierarchicznie z mniejszych komponentów [10].

Komponenty kontenerowe (ang. container components, smart components) odpowiadają za działanie aplikacji: zarządzanie stanem, pobieranie danych i wywoływanie logiki biznesowej. W przeciwieństwie do komponentów prezentacyjnych, nie zajmują się wyglądem, a jedynie dostarczają dane do komponentów UI. W React często wykorzystują hooki, takie jak useState i useEffect. Kontenery mogą korzystać z komponentów prezentacyjnych, ale nie odwrotnie. Ich relacja jest hierarchiczna: kontener zarządza logiką, prezentacja odpowiada za wyświetlanie. Dobre praktyki zalecają, by kontenery były proste i czytelne. Ich stosowanie poprawia strukturę kodu, wspiera testowalność i ponowne wykorzystanie komponentów UI, pod warunkiem, że nie staną się zbyt rozbudowane [19].

W nowoczesnych aplikacjach tworzonych w oparciu o bibliotekę React, powszechnie stosuje się podejście inspirowane architekturą warstwową. Nie stanowi ono formalnego modelu narzuconego przez samą bibliotekę, lecz jest praktyką projektową

polegającą na podziale kodu na warstwy odpowiedzialne odpowiednio za usługi oraz logikę aplikacyjną. Taki podział umożliwia utrzymanie jednokierunkowego przepływu danych oraz wyraźne rozdzielenie odpowiedzialności pomiędzy poszczególnymi elementami systemu.

Warstwa usług odpowiada za komunikację aplikacji z zewnętrznymi systemami, zapewniając dostęp do danych i operacji w sposób izolowany i ustandaryzowany. Ukrywa szczegóły techniczne, takie jak adresy endpointów, metody HTTP czy obsługa błędów, dzięki czemu pozostałe warstwy aplikacji korzystają z prostego, spójnego interfejsu. Usługi organizowane są zwykle w osobnych folderach i są podzielone według funkcji domenowych, często opierają się na współdzielonym kliencie HTTP. Kod tej warstwy nie zawiera odwołań do interfejsu użytkownika i może korzystać z innych usług pomocniczych. Komponenty kontenerowe oraz moduły logiki biznesowej wywołują usługi, ale same usługi nie mają wiedzy o prezentacji, co zapewnia jednoznaczny kierunek zależności [10, 19].

Warstwa logiki aplikacyjnej określa zasady działania aplikacji wynikające z wymagań domenowych. Odpowiada za podejmowanie decyzji w reakcji na działania użytkownika, walidację danych, wykonywanie obliczeń, sterowanie nawigacją i koordynację wywołań usług. Część tej logiki może być realizowana po stronie klienta, bez udziału serwera. Logika biznesowa powinna być niezależna od interfejsu i źródeł danych – operuje na abstrakcyjnych modelach i korzysta z warstwy usług do komunikacji z zewnętrznymi systemami. Komponenty kontenerowe przekazują dane do funkcji warstwy logiki, a ta zwraca wynik, który trafia do komponentów prezentacyjnych. Dobre praktyki zakładają czystość funkcji (brak efektów ubocznych) oraz możliwość ich testowania w izolacji. Taki podział ułatwia ponowne wykorzystanie logiki, modyfikacje aplikacji oraz zachowanie modularnej, skalowalnej struktury projektu [10].

1.2.5 Bezpieczeństwo warstwy frontend

Bezpieczeństwo aplikacji internetowych stanowi istotny aspekt projektowania oraz implementacji strony klienta, która ze względu na swoją bezpośrednią interakcję z użytkownikiem narażona jest na liczne zagrożenia. Biblioteki i narzędzia frontend, w tym React, oferują programistom pewne mechanizmy minimalizujące ryzyko ataków, jednak ich skuteczność zależy od poprawności wdrożenia oraz przestrzegania rekomendowanych praktyk programistycznych.

Jednym z najczęściej występujących zagrożeń dla aplikacji internetowych jest atak typu Cross-Site Scripting (XSS). Polega on na wprowadzeniu przez atakującego szkodliwego kodu JavaScript do aplikacji, który ma zostać wykonany w przeglądarce użytkownika końcowego. React ogranicza podatność aplikacji na tego rodzaju ataki dzięki zastosowaniu składni JSX, automatycznie zabezpieczając treści przed nieuprawnionym wykonaniem kodu JavaScript. Pomimo tego, programiści muszą zachować szczególną ostrożność w sytuacjach, kiedy aplikacja bezpośrednio ingeruje w tzw. DOM (Document Object Model), czyli strukturę drzewa elementów HTML, reprezentującą treść strony w przeglądarce [20].

Kolejnym poważnym zagrożeniem bezpieczeństwa jest atak typu Cross-Site Request Forgery (CSRF), polegający na wykorzystaniu zalogowanego użytkownika do nieświadomego wysłania żądania na serwer aplikacji ze swojej przeglądarki. Chociaż główne zabezpieczenia przed tego rodzaju atakami znajdują się zwykle po stronie serwera, frontend również powinien wspierać ochronę np. poprzez odpowiednią obsługę tokenów uwierzytelniających. Tokeny uwierzytelniające (np. JSON Web Token – JWT) przechowywane są w odpowiednio zabezpieczonych miejscach, takich jak pamięć sesyjna przeglądarki (sessionStorage), unikając mniej bezpiecznych rozwiązań, takich jak przechowywanie ich w ciasteczkach lub lokalnych magazynach danych (localStorage). Dodatkowo frontend wymusza dołączanie odpowiednich nagłówków HTTP w każdym żądaniu wysyłanym do serwera, zapewniając tym samym dodatkową warstwę weryfikacji pochodzenia żądań [21].

Bardzo ważną praktyką zwiększającą bezpieczeństwo aplikacji frontend jest właściwa walidacja oraz sanitacja danych użytkownika. Walidacja danych polega na sprawdzeniu poprawności oraz zgodności danych wejściowych użytkownika z ustalonymi regułami, podczas gdy sanitacja oznacza usuwanie lub neutralizację potencjalnie niebezpiecznych treści.

Dodatkowo, zwiększenie bezpieczeństwa aplikacji React można osiągnąć poprzez odpowiednie skonfigurowanie serwera, w tym użycie specjalnych nagłówków HTTP określających politykę bezpieczeństwa aplikacji [22]. Do najważniejszych należą:

1. Content Security Policy (CSP) – polityka określająca, z jakich źródeł mogą pochodzić zasoby ładowane przez przeglądarkę.
2. X-Frame-Options – zabezpieczenie przed tzw. atakami clickjacking, czyli umieszczeniem aplikacji w ramce (iframe) w celu manipulacji użytkownikiem.

3. Strict-Transport-Security (HSTS) – wymuszanie na przeglądarce komunikacji z serwerem wyłącznie za pomocą bezpiecznego protokołu HTTPS.

Zastosowanie wyżej opisanych praktyk oraz świadomość potencjalnych zagrożeń pozwalają istotnie podnieść poziom bezpieczeństwa aplikacji React, chroniąc tym samym dane użytkowników oraz zapewniając stabilność działania całego systemu.

1.3 Backend (Igor Kohsin)

Backend w aplikacjach internetowych odpowiada za realizację logiki biznesowej, przetwarzanie danych oraz zarządzanie komunikacją z systemami zewnętrznymi, w tym z bazami danych i usługami sieciowymi. W literaturze warstwa ta jest definiowana jako część systemu działająca po stronie serwera, niewidoczna bezpośrednio dla użytkownika końcowego, której zadaniem jest zapewnienie poprawnego funkcjonowania aplikacji poprzez obsługę żądań pochodzących z interfejsu użytkownika [23].

Podstawową funkcją backendu jest realizacja logiki biznesowej, rozumianej jako zbiór reguł i procesów determinujących działanie aplikacji zgodnie z jej przeznaczeniem. W części backend implementuje się również mechanizmy uwierzytelniania (ang. authentication) i autoryzacji (ang. authorization), które odpowiadają odpowiednio za identyfikację użytkownika oraz nadawanie mu odpowiednich uprawnień do zasobów systemu.

We współczesnych aplikacjach backend jest zwykle podzielony na warstwy o wyraźnie określonych odpowiedzialnościach. Wyróżnia się najczęściej warstwę kontrolerów (ang. controllers), obsługującą żądania przychodzące z interfejsu użytkownika, warstwę serwisów (ang. services), realizującą logikę biznesową, oraz warstwę repozytoriów (ang. repositories), zarządzającą dostępem do danych przechowywanych w bazach danych [24].

Do realizacji funkcji backendowych wykorzystywane są nowoczesne technologie i frameworki, które wspierają tworzenie aplikacji skalowalnych, bezpiecznych i wydajnych. Przykładami dla języka Java może być np. Spring Framework, Grails, Vaadiin, Play i inne. Spring jest jednym z najpopularniejszych frameworków Java, a większość deweloperów pozytywnie ocenia pracę w tym środowisku. [25, 26]. W ekosystemie języka C# analogiczną rolę pełni platforma ASP.NET Core, umożliwiająca tworzenie usług backendowych oraz interfejsów REST o wysokiej wydajności. Alternatywne podejście oferuje język Python, w którym powszechnie wykorzystywane są frameworki Django oraz Flask.

1.3.1 Framework Spring

Framework Spring jest systematycznie rozwijanym zbiorem nowoczesnych rozwiązań dla strony backend aplikacji. Jednym z paradygmatów stosowanych w Spring jest zastosowanie zasady odwrócenia kontroli sterowania (Inversion of Control, IoC). Jedną z możliwości realizacji IoC jest wzorzec wstrzykiwania zależności (Dependency Injection, DI). Dzięki temu komponenty aplikacji nie są odpowiedzialne za samodzielne tworzenie swoich zależności – są one dostarczane z zewnątrz przez kontener Spring, co umożliwia lepsze zarządzanie cyklem życia obiektów, ułatwia testowanie oraz promuje luźne powiązanie pomiędzy klasami. Poza wzorcami IoC i DI, Spring wyróżnia się również modularną architekturą, dzięki której aplikacja może wykorzystywać jedynie te komponenty, które są faktycznie potrzebne. Framework wspiera także programowanie aspektowe (Aspect Oriented Programming, AOP), umożliwiające wydzielenie logiki przekrojowej, takiej jak logowanie czy zarządzanie transakcjami. Istotnym elementem jest również ujednolicony model obsługi transakcji oraz bogaty ekosystem integracji z technologiami bazodanowymi, komunikacyjnymi i chmurowymi [27, 28].

Framework Spring odznacza się modularnością, co pozwala programistom na wykorzystanie jedynie tych elementów, które są niezbędne dla danej aplikacji. Ponadto wspiera szeroki zakres wzorców projektowych, takich jak warstwa serwisów (Service Layer), wzorzec kontrolera (Controller) czy wzorzec dostępu do danych (DAO – Data Access Object), co sprzyja tworzeniu aplikacji zgodnych z dobrymi praktykami inżynierii oprogramowania. Istotną zaletą Spring jest także łatwość integracji z zewnętrznymi bibliotekami i systemami, dzięki czemu może on pełnić rolę centralnego elementu architektury aplikacyjnej [29].

Istotnym elementem Spring są adnotacje, które znacząco upraszczają konfigurację i implementację aplikacji oraz zwiększają czytelność kodu. Dzięki nim możliwe jest deklaratywne definiowanie zachowania komponentów, bez konieczności tworzenia rozbudowanych konfiguracji XML. Adnotacje takie jak @Component, @Service czy @Repository pozwalają na automatyczne wykrywanie i rejestrowanie klas w kontenerze IoC, natomiast @Autowired lub @Inject wspomagają wstrzykiwanie zależności. Ważną rolę odgrywają również adnotacje związane z warstwą webową, takie jak @Controller czy @RestController oraz adnotacje transakcyjne, w tym @Transactional, umożliwiające deklaratywne zarządzanie transakcjami [28].

Rozszerzeniem podstawowego frameworka Spring jest Spring Boot, który stanowi odpowiedź na potrzebę uproszczenia konfiguracji aplikacji. Dzięki podejściu *convention over configuration*, Spring Boot eliminuje potrzebę definiowania wielu ustawień poprzez dostarczanie gotowych do użycia szablonów konfiguracyjnych. Aplikacje tworzone w oparciu o Spring Boot mogą działać jako samodzielne aplikacje tzw. standalone, dzięki wbudowanym serwerom, takim jak Tomcat czy Jetty, co upraszcza ich uruchamianie i wdrażanie. Rozwiązanie to znacząco przyspiesza proces rozwoju aplikacji, jednocześnie zachowując wszystkie zalety architektury frameworka Spring [30, 31].

1.3.2 Ekosystem Spring

Spring został opracowany jako lekka alternatywa dla ciężkich rozwiązań platformy J2EE (Java 2 Platform, Enterprise Edition), czyli platformy stworzonej przez firmę Sun Microsystems, obecnie rozwijanej przez Oracle, przeznaczonej do budowy aplikacji biznesowych i korporacyjnych w języku Java. Z czasem rozwinął się w pełnoprawny ekosystem technologiczny, który wspiera różne aspekty tworzenia aplikacji – od obsługi warstwy webowej, przez integrację z bazami danych, po zaawansowane rozwiązania chmurowe i mikrousługowe. Ekosystem ten składa się z szeregu modułów i rozszerzeń, oferując gotowe rozwiązania dla typowych problemów programistycznych.

Jedną z ważnych cech frameworka Spring jest wykorzystanie wzorca architektonicznego MVC (Model, View, Controller). Pozwala on na przejrzyste oddzielenie logiki aplikacyjnej od warstwy prezentacji oraz obsługi żądań HTTP [29].

Kolejnym ważnym elementem jest warstwa abstrakcji w postaci projektu Spring Data, który upraszcza integrację aplikacji z relacyjnymi i nierelacyjnymi bazami danych. Spring Data dostarcza m.in. mechanizmy automatycznego generowania zapytań na podstawie nazw metod w interfejsach repozytoriów, co ogranicza potrzebę pisania kodu SQL i pozwala skupić się na logice biznesowej. Spring Data zapewnia wsparcie nie tylko dla klasycznych baz SQL w oparciu o JPA, ale również dla rozwiązań typu NoSQL, takich jak MongoDB czy Redis [32].

W kontekście bezpieczeństwa aplikacji istotną rolę odgrywa Spring Security – kompleksowy zestaw narzędzi w postaci frameworku umożliwiający implementację mechanizmów autoryzacji i uwierzytelniania. Framework ten wspiera wiele standardów bezpieczeństwa, w tym JWT, OAuth2, SAML (Security Assertion Markup Language) oraz LDAP (Lightweight Directory Access Protocol), a także oferuje ochronę przed popularnymi zagrożeniami, takimi jak ataki CSRF (Cross-Site Request Forgery) czy XSS

(Cross-Site Scripting). Jego modularna architektura umożliwia dostosowanie zabezpieczeń do specyficznych potrzeb aplikacji [33].

Integralną częścią ekosystemu Spring są również narzędzia wspierające testowanie aplikacji, takie jak *Spring Test* czy *MockMvc*. Umożliwiają one przeprowadzanie zarówno testów jednostkowych, jak i integracyjnych, w pełnym kontekście aplikacji. Adnotacja @SpringBootTest pozwala na uruchomienie środowiska aplikacyjnego zbliżonego do produkcyjnego, co zwiększa wiarygodność i wartość testów. Takie podejście znacząco wpływa na jakość końcowego produktu, umożliwiając szybkie wykrywanie i eliminację błędów [34].

1.3.3 Architektura aplikacji Spring

Architektura aplikacji zbudowanej w oparciu o framework Spring opiera się na koncepcji wyraźnego podziału odpowiedzialności pomiędzy poszczególne komponenty systemu. Taki model sprzyja przejrzystości kodu, ułatwia jego testowanie, rozwój oraz utrzymanie, a także umożliwia łatwą integrację z zewnętrznymi usługami.

W typowej aplikacji wykorzystującej Spring, backend organizowany jest wokół kilku głównych grup komponentów, z których każda realizuje ściśle określoną rolę [24] [29].

1. Modele (Models) – reprezentują strukturę danych, którą odwzorowują encje JPA. Składają się z klas oznaczonych adnotacją @Entity, które odwzorowują strukturę tabel w relacyjnej bazie danych. Modele odzwierciedlają pojęcia i relacje istotne z punktu widzenia logiki aplikacji, stanowiąc podstawę dla operacji realizowanych przez serwisy i komponenty dostępu do danych.
2. Komponenty dostępu do danych (Data Access Components) – odpowiadają za interakcję z bazą danych. W przypadku zastosowania modułu Spring Data JPA, komponenty te przyjmują postać interfejsów repozytoriów oznaczonych adnotacją @Repository. Spring automatycznie generuje ich implementacje, umożliwiając wykonywanie zapytań bez konieczności ręcznego pisania kodu SQL. Takie podejście przyspiesza rozwój i redukuje ryzyko błędów związanych z operacjami na danych.
3. Kontrolery (Controllers) – stanowią punkt wejścia do systemu. Odpowiadają za obsługę żądań przychodzących od klientów (np. przeglądarek internetowych lub aplikacji mobilnych) m.in. poprzez interfejs REST API. Komponenty te przetwarzają zapytania HTTP, przekazują je do odpowiednich serwisów oraz

przygotowują dane do zwrócenia w odpowiedzi. Dzięki adnotacjom takim jak @Controller, @RestControler i @RequestMapping, możliwe jest przejrzyste definiowanie tras i metod obsługujących różne operacje.

4. Serwisy (Services) – działają zgodnie z logiką biznesową aplikacji. To w tej warstwie realizowane są operacje związane z przetwarzaniem danych, regułami domenowymi, walidacją czy koordynacją pracy różnych komponentów. Komponenty serwisowe oznaczane są zwykle adnotacją @Service, a ich metody często korzystają z @Transactional, co umożliwia automatyczne zarządzanie transakcjami w kontekście dostępu do bazy danych.

Opisana architektura pozwala na osiągnięcie niskiego poziomu sprzężenia, tzw. *loose coupling* pomiędzy komponentami, co ułatwia ich niezależny rozwój, testowanie i ponowne wykorzystanie. Kluczową rolę w realizacji tej separacji pełni IoC i Dependency Injection, opisane w podrozdziale 1.3.1, które umożliwiają automatyczne zarządzanie zależnościami między obiektami. Dzięki nim komponenty systemu są konfigurowane deklaratywnie, a nie tworzone ręcznie, co znacząco wpływa na elastyczność i testowalność kodu [31].

1.3.4 Bezpieczeństwo aplikacji Spring

Spring dostarcza rozbudowane narzędzia do zapewnienia bezpieczeństwa aplikacji, a kluczową rolę w tym zakresie odgrywa moduł Spring Security. Zapewnia on kompleksowe mechanizmy do autoryzacji i uwierzytelniania, w tym integrację z zewnętrznymi dostawcami tożsamości (np. LDAP, OAuth2, Single Sign-On), co umożliwia dopasowanie poziomu zabezpieczeń do konkretnych wymagań projektu.

Z perspektywy warstwowej architektury aplikacji opisanej wcześniej w podrozdziale 1.3.3, komponenty odpowiedzialne za bezpieczeństwo funkcjonują głównie w warstwie domenowej i dostępu do danych. Ich głównym zadaniem jest ochrona punktów wejścia do systemu, w szczególności kontrolerów obsługujących żądania HTTP, poprzez filtrowanie i kontrolę przepływu danych jeszcze przed rozpoczęciem przetwarzania właściwej logiki biznesowej.

Mechanizmy bezpieczeństwa w Spring Security oparte są na zestawie współdziałających komponentów, które przetwarzają każde przychodzące żądanie. Do najważniejszych z nich należą [35, 36, 37]:

- Filtry – wszystkie żądania przechodzą przez tzw. łańcuch filtrów (filter chain), w którym w ustalonej kolejności wywoływane są komponenty odpowiadające m.in.

za uwierzytelnianie, autoryzację czy sprawdzanie tokenów. Filtr to element przechwytujący żądanie HTTP jeszcze przed jego obsłużeniem przez kontroler, pozwalając na przykład na prewencyjne odrzucenie nieautoryzowanego ruchu.

- AuthenticationManager – centralny punkt odpowiedzialny za proces autentykacji. Współpracuje z dostawcami poświadczeń (providerami), którzy na podstawie przekazanych danych logowania weryfikują tożsamość użytkownika.
- UserDetailsService – interfejs, którego implementacja dostarcza dane o użytkowniku na potrzeby autentykacji. W aplikacjach integrujących się z bazą danych komponent ten umożliwia wczytanie informacji o użytkownikach bezpośrednio z repozytoriów domenowych;
- SecurityContextHolder, mechanizm przechowujący informacje o bieżącym kontekście bezpieczeństwa, w tym dane uwierzytelnionego użytkownika. Pozwala to na dostęp do tożsamości i ról użytkownika w różnych częściach aplikacji, bez konieczności ponownego logowania lub przekazywania dodatkowych danych.

1.4 Baza danych (Igor Kohsin)

Baza danych stanowi zorganizowany zbiór informacji przechowywanych w formie elektronicznej, umożliwiający efektywne zarządzanie i przetwarzanie danych. System zarządzania bazą danych (SZBD), znany również jako Database Management System (DBMS), to oprogramowanie pośredniczące między użytkownikami a bazą danych, które umożliwia tworzenie, modyfikowanie, zarządzanie oraz dostęp do danych w sposób zorganizowany i bezpieczny [38].

Podstawowe funkcje SZBD obejmują [39, 40, 38]:

1. Definiowanie danych - określenie struktury danych oraz relacji między nimi.
2. Wykonywanie operacji CRUD (Create, Read, Update, Delete) na danych, takich jak dodawanie, modyfikacja, usuwanie oraz wyszukiwanie danych.
3. Zarządzanie transakcjami - zgodnie z zasadami ACID (Atomicity, Consistency, Isolation, Durability).
4. Kontrolowanie dostępu - zarządzanie uprawnieniami użytkowników do odczytu, zapisu oraz modyfikacji danych.
5. Zarządzanie współbieżnością - koordynacja dostępu wielu użytkowników do danych w sposób zapewniający spójność i integralność.

W kontekście aplikacji webowych, SZBD odgrywają kluczową rolę w przechowywaniu i zarządzaniu danymi, zapewniając ich integralność, bezpieczeństwo oraz dostępność. Aplikacje webowe często korzystają z relacyjnych baz danych, które umożliwiają przechowywanie danych w tabelach powiązanych za pomocą kluczy.

Przykładem może być PostgreSQL. Jest on zaawansowanym, obiektowo-relacyjnym systemem zarządzania bazą danych typu open-source, który oferuje wysoką dostępność oraz zgodność z różnymi platformami. Przechowuje dane w tabelach, które mogą być połączone za pomocą kluczy obcych, wspierając integralność referencyjną oraz umożliwiając skomplikowane operacje na danych.

System ten umożliwia korzystanie z różnych typów danych, w tym standardowych, takich jak liczby całkowite czy łańcuchy znaków oraz zaawansowanych, takich jak JSON, XML czy dane przestrzenne. System ten oferuje również wsparcie dla replikacji oraz klasteryzacji, co umożliwia zapewnienie wysokiej dostępności oraz skalowalności systemu [41, 39].

PostgreSQL, mimo że jest relacyjnym systemem zarządzania bazą danych, oferuje również funkcjonalności charakterystyczne dla rozwiązań typu NoSQL. System wspiera przechowywanie i przetwarzanie danych półustrukturyzowanych dzięki natywnym typom danych JSON oraz JSONB, umożliwiając wykonywanie zapytań na dokumentach JSON, indeksowanie ich zawartości oraz łączenie danych relacyjnych z dokumentowymi w ramach jednej bazy danych. Takie podejście pozwala na elastyczne modelowanie danych oraz stopniową ewolucję schematu bez konieczności rezygnacji z mechanizmów relacyjnych, takich jak transakcje czy integralność danych.

Dzięki swojej niezawodności, elastyczności i rozbudowanej funkcjonalności, PostgreSQL jest jednym z częściej wybieranych systemów zarządzania bazą danych, do aplikacji wymagających zarówno dużej wydajności, jak i zaawansowanych operacji na danych [39]

1.5 Testowanie (Igor Kohsin, Oskar Krupa)

Testowanie jest ważnym elementem procesu tworzenia aplikacji internetowych, zapewniającym wysoką jakość, niezawodność oraz bezpieczeństwo systemu. Jego głównym celem jest identyfikacja i eliminacja błędów na różnych etapach cyklu życia oprogramowania. W ramach testowania wyróżniamy różne poziomy i metody, które pozwalają na wczesne wykrywanie błędów, zapobieganie regresji oraz optymalizację działania aplikacji [42, 43, 44].

Testy, które można wykonać na oprogramowaniu dzieli się najczęściej na cztery rodzaje [43, 42]:

1. Testy jednostkowe (Unit Tests) polegają na testowaniu pojedynczych jednostek kodu, takich jak metody czy klasy, w izolacji od reszty aplikacji. Służą do weryfikacji poprawności działania najmniejszych jednostek kodu.
2. Testy integracyjne (Integration Tests) mają na celu sprawdzenie współpracy między różnymi komponentami systemu, np. warstwą backend i frontend, bazą danych czy usługami zewnętrznymi.
3. Testy systemowe (System Tests) obejmują testowanie całego systemu jako całości, weryfikując takie aspekty jak funkcjonalność, niezawodność, skalowalność itd.
4. Testy akceptacyjne (Acceptance Tests) przeprowadzane są w celu potwierdzenia, że system spełnia wymagania biznesowe i jest gotowy do wdrożenia.

Ze względu na sposób analizy systemu testowanie można podzielić na dwie podstawowe kategorie (42; 44):

1. Testowanie statyczne, które polega na analizie kodu źródłowego, dokumentacji oraz przeglądach technicznych bez uruchamiania aplikacji.
2. Testowanie dynamiczne, które wymaga uruchomienia systemu i obserwacji jego zachowania w trakcie wykonywania.

W zależności od sposobu realizacji testów wyróżnia się następujące techniki (42; 43):

1. Testowanie manualne, polegające na ręcznym wykonywaniu scenariuszy testowych przez testerów.
2. Testowanie automatyczne, w którym wykorzystywane są narzędzia umożliwiające automatyczne uruchamianie testów, co zwiększa ich powtarzalność i efektywność.

Ze względu na zakres weryfikowanych cech systemu wyróżnia się (42):

F10. Testowanie funkcjonalne, którego celem jest sprawdzenie zgodności działania systemu z wymaganiami funkcjonalnymi.

F11. Testowanie niefunkcjonalne, obejmujące m.in. testy wydajnościowe, bezpieczeństwa, niezawodności oraz użyteczności.

1.6 Współpraca frontend – backend (Oskar Krupa, Igor Kohsin)

Sprawna współpraca pomiędzy warstwą frontend a backend stanowi podstawę funkcjonowania systemów informatycznych, zapewniając poprawną wymianę danych

oraz spójność informacji w obrębie całej aplikacji. Komunikacja ta realizowana jest najczęściej za pośrednictwem protokołu HTTP lub jego rozszerzeń, z wykorzystaniem stylu architektonicznego REST lub podobnych podejść zorientowanych na usługi.

Styl architektoniczny REST opiera się na koncepcji zasobów identyfikowanych za pomocą adresów URI oraz standardowych metod protokołu HTTP, takich jak GET, POST, PUT, PATCH oraz DELETE. Podejście to zakłada bezstanowość komunikacji, jednoznaczne wykorzystanie kodów statusu HTTP oraz rozdzielenie warstwy klienckiej od serwerowej.

Wymiana danych pomiędzy klientem a serwerem zazwyczaj odbywa się w formacie tekstowym umożliwiającym łatwą serializację i deserializację danych, często wykorzystywanym formatem jest JSON (JavaScript Object Notation). Standaryzacja struktury przesyłanych danych pozwala na ich jednoznaczną interpretację, co ma istotne znaczenie dla walidacji i przetwarzania informacji po obu stronach komunikacji [45].

Oprócz komunikacji synchronicznej opartej na protokole HTTP, w nowoczesnych aplikacjach coraz częściej stosowane są również inne mechanizmy komunikacji, takie jak WebSocket, umożliwiający dwukierunkową komunikację w czasie rzeczywistym, czy protokół gRPC, który bazuje na wysoce wydajnym modelu komunikacji opartym na wywołaniach zdalnych procedur. Rozwiązania te znajdują zastosowanie m.in. w aplikacjach wymagających natychmiastowej wymiany danych, takich jak systemy czatowe czy aplikacje czasu rzeczywistego.

Integralną częścią tego mechanizmu jest obsługa błędów, której celem jest zapewnienie odporności systemu na sytuacje wyjątkowe oraz umożliwienie klientowi odpowiedniej reakcji. W razie nieprawidłowości, warstwa serwerowa zwraca odpowiednie kody statusu HTTP wraz z informacją diagnostyczną, pozwalając na identyfikację typu błędu (np. błąd walidacji, brak zasobu, naruszenie uprawnień). Po stronie klienta stosowane są mechanizmy interpretacji tych odpowiedzi, które umożliwiają np. prezentację komunikatów użytkownikowi końcowemu lub wykonanie określonych procedur naprawczych. Tego rodzaju współpraca stanowi fundament działania współczesnych aplikacji, zarówno webowych, jak i mobilnych, w których interfejs użytkownika jest oddzielony logicznie i technologicznie od logiki biznesowej i warstwy danych [46, 47, 48].

2. PROJEKT APLIKACJI (IGOR KOHSIN, OSKAR KRUPA)

W ramach pracy opracowano projekt i dokonano implementacji systemu o nazwie „GastroM”, którego celem jest wspomaganie działalności gastronomicznej poprzez monitorowanie danych w zakresie zarządzania restauracją, w tym pracownikami, magazynem i transakcjami. W niniejszym rozdziale zamieszczono projekt aplikacji obejmujący: analizę wymagań, schemat bazy danych, wybór technologii i narzędzi oraz schemat interfejsu użytkownika.

2.1 Analiza wymagań

Projektowana aplikacja jest przeznaczona dla menadżerów przedsiębiorstw zajmujących się przygotowywaniem oraz sprzedawaniem napojów i ma na celu usprawnienie i zintegrowanie procesów związanych z funkcjonowaniem działalności gastronomicznej. Założono, że przedsiębiorstwo zatrudnia pracowników oraz posiada magazyn. Głównym zadaniem aplikacji jest umożliwienie kontrolowania stanów magazynowych, monitorowanie kosztów oraz generowania raportów finansowych z funkcjonowania działalności.

Dzięki aplikacji możliwe będzie:

1. automatyczne rozliczanie sprzedaży i kosztów,
2. kontrola zużycia surowców,
3. analiza rentowności poszczególnych produktów sprzedażowych lub okresów sprzedaży.

System zakłada istnienie dwóch poziomów uprawnień — **Menadżera** i **Administratora**, przy czym rolę administratora odróżnia jedynie możliwość dodawania kolejnych kont menadżerów lub administratorów. Za wyjątkiem tej funkcji, funkcjonalność aplikacji dla tych dwóch ról pozostaje taka sama.

Dla aplikacji zdefiniowano wymagania funkcjonalne i poza funkcjonalne opisane poniżej.

2.1.1 Wymagania funkcjonalne

W celu zdefiniowania wymagań funkcjonalnych dla projektowanej aplikacji określono najważniejsze obszary działania systemu, uwzględniając charakterystykę branży gastronomicznej oraz kontekst zadań stawianych przed aplikacją. W tym celu

zidentyfikowano obszary działania przedsiębiorstwa, które będą wspomagane przez aplikację: zarządzanie magazynem, zarządzanie personelem, monitorowanie operacji handlowych, wyświetlanie bieżących statystyk i generowanie raportów. Poniżej przedstawiono najważniejsze wymagania funkcjonalne podzielone według obszarów działania aplikacji. Opis Użytkownik oznacza tutaj Menadżera lub Administratora.

Zarządzanie magazynem

F1. Użytkownik zarządza składnikami produktów sprzedażowych – dodaje nowe składniki, usuwa istniejące i modyfikuje ich dane opisowe i parametry stałe, takie jak nazwa, ilość, cena jednostkowa czy jednostka miary, bez bezpośredniej ingerencji w aktualny stan magazynowy.

F2. Użytkownik uzupełnia zapasy magazynowe – realizuje operację polegającą na zwiększeniu aktualnego stanu ilościowego wybranego składnika o określoną wartość, co odzwierciedla fizyczne przyjęcie towaru do magazynu.

F3. Użytkownik zarządza produktami sprzedażowymi – dodaje, usuwa i modyfikuje produkty sprzedażowe.

F4. Użytkownik ma możliwość przeglądania listy wszystkich składników oraz produktów sprzedażowych oraz szczegółowych informacji o każdym z nich.

Zarządzanie personelem

F5. Użytkownik może dodawać, edytować i usuwać dane pracowników, takie jak imię, nazwisko, oraz stawka godzinowa.

F6. System umożliwia ewidencjonowanie czasu pracy pracowników poprzez rejestrowanie liczby godzin, w których dany pracownik był zalogowany w systemie w danym dniu co pozwala użytkownikowi obliczyć wysokości wynagrodzeń na podstawie zadeklarowanej stawki godzinowej.

F7. Dane o wynagrodzeniach są automatycznie uwzględniane w zestawieniach kosztów stałych działalności - użytkownik inicjalizuje wypłatę pracownika na podstawie ilości przepracowanych godzin oraz stawki godzinowej.

F8. Użytkownik ma możliwość przeglądania listy wszystkich pracowników oraz szczegółowych informacji o każdym z nich.

F9. Administrator może dodawać oraz usuwać istniejących użytkowników aplikacji.

Monitorowanie operacji handlowych

F10. Użytkownik ma możliwość przeglądania historii dokonanych transakcji wraz z ich szczegółami.

F11. Użytkownik ma dostęp do aktualnych informacji na temat operacji handlowych w każdej chwili działania systemu.

Wyświetlanie bieżących statystyk

F12. Użytkownik może wyświetlić podstawowe statystyki przychodów, kosztów oraz bilansu zysków w określonym przedziale czasowym.

F13. Użytkownik może wyświetlić szczegółowe statystyki obejmujące dodatkowo, poza podstawowymi, statystyki dotyczące poszczególnych produktów sprzedażowych, takich jak ilość sprzedanych produktów czy rentowność poszczególnych produktów.

F14. Użytkownik ma możliwość wyświetlenia produktów, których ilość przekroczyła ustalony próg ostrzeżenia o niskim stanie magazynowym.

Generowanie raportów

F15. Użytkownik może wygenerować raport finansowy obejmujące szczegółowe statystyki za wybrany okres oraz pobrać go w formie pliku o formacie PDF.

2.1.2 Wymagania pozafunkcjonalne

Przy projektowaniu rozwiązań wspierających działalność gastronomiczną szczególny nacisk położono na zapewnienie stabilności, wysokiej dostępności danych oraz elastyczności w rozszerzaniu funkcjonalności w miarę rozwoju przedsiębiorstwa

Zdefiniowane wymagania pozafunkcjonalne to:

PF1. System obsługuje pojedyncze żądanie użytkownika w czasie nie dłuższym niż 2 sekundy.

PF2. System zapewnia ciągłość działania i bezpieczeństwo danych w przypadku awarii.

PF3. Hasła użytkowników są haszowane i zabezpieczone przed nieautoryzowanym dostępem.

PF4. Dostęp do systemu wymaga logowania i autoryzacji zgodnej z rolą użytkownika.

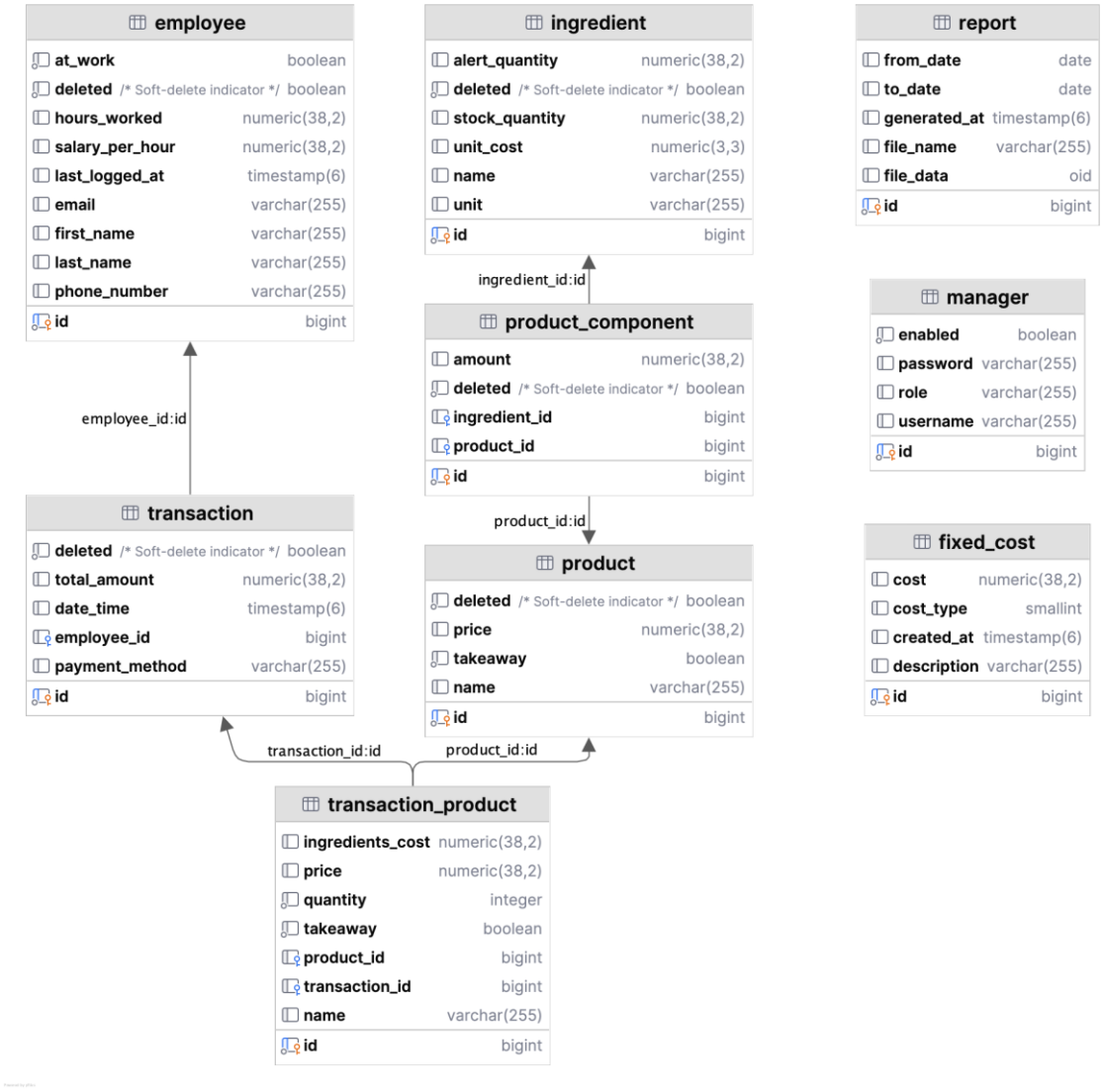
PF5. Interfejs użytkownika jest intuicyjny.

PF6. System jest skalowalny i umożliwia łatwe rozszerzanie funkcjonalności w przyszłości.

PF7. Aplikacja działa w architekturze klient serwer korzystając z protokołu HTTP i przeglądarki internetowej i jest kompatybilna z popularnymi systemami operacyjnymi.

2.2 Schemat bazy danych

Częścią projektu jest schemat bazy danych przedstawiony na Rys. 2.1. Baza danych została zaprojektowana w oparciu o model relacyjny, a każda z tabel zawiera klucz główny w postaci kolumny id.



Rys. 2.1. Schemat bazy danych aplikacji. Źródło: Opracowanie własne.

W tabeli **employee** znajdują się dane pracowników, w tym numer telefonu (phone_number), imię (first_name), nazwisko (last_name), email (email), godziny przepracowane dotychczasowo (hours_worked), stawka godzinowa (salary_per_hour),

informacja czy pracownik jest aktualnie w pracy (at_work) oraz data ostatniego logowania (last_logged_at). Tabela employee jest powiązana relacją jeden-do-wielu (1:N) z tabelą tabeli transaction, co oznacza, że jeden pracownik może obsłużyć wiele transakcji, natomiast każda transakcja przypisana jest do dokładnie jednego pracownika.

Tabela **transaction** obejmuje transakcje dokonane przez pracowników, w tym całkowitą kwotę (total_amount), metodę płatności (payment_method) dzieląca się na płatności kartą lub gotówką, datę i czas transakcji (date_time) oraz identyfikator pracownika (employee_id), który stanowi klucz obcy wskazujący na tabelę employee. Tabela ta pozostaje również w relacji jeden-do-wielu (1:N) z tabelą transaction_product, która przechowuje szczegółowe pozycje sprzedażowe składające się na daną transakcję.

Produkty sprzedażowe opisane są w tabeli **product**, z kolumnami takimi jak nazwa (name), cena (price), informacja czy produkt jest na wynos (takeaway) oraz wskaźnik usunięcia (deleted). Tabela product jest powiązana relacją wiele-do-wielu (N:M) zarówno z tabelą transaction, jak i z tabelą ingredient, przy czym obie relacje są realizowane za pomocą tabel pośrednich. W przypadku relacji z transakcjami rolę tę pełni tabela transaction_product, natomiast relację ze składnikami realizuje tabela product_component.

Składniki produktów znajdują się w tabeli **ingredient** i obejmują nazwę (name), jednostkę miary (unit), koszt jednostkowy (unit_cost), ilość w magazynie (stock_quantity) oraz minimalną dopuszczalną ilość (alert_quantity), po przekroczeniu której pojawia się alert o niskim stanie magazynowym. Tabela ingredient przechowująca dane magazynowe systemu GastroM jest powiązana relacją jeden-do-wielu (1:N) z tabelą product_component, co oznacza, że jeden składnik może być wykorzystywany w wielu różnych produktach.

Tabela **product_component** łączy produkty ze składnikami, określając ilość potrzebną do przygotowania produktu (amount), wskaźnik usunięcia (deleted) oraz identyfikatory składnika (ingredient_id) i produktu (product_id). Każdy produkt składa się z jednego lub wielu rekordów w tabeli product_component, które jednoznacznie definiują jego recepturę. Tabela ta realizuje relację wiele-do-wielu (N:M) pomiędzy tabelami product i ingredient.

Związki między transakcjami a produktami zapisano w tabeli **transaction_product,** która zawiera cenę produktu w transakcji (price), koszt składników (ingredients_cost), ilość zamówionych produktów (quantity), informację czy

produkt był na wynos (takeaway), nazwę produktu (name) oraz identyfikatory transakcji (transaction_id) i produktu (product_id). Tabela transaction_product pełni rolę tabeli historycznej, przechowując dane sprzedażowe w stanie niezmiennym, niezależnym od późniejszych modyfikacji cen produktów lub kosztów składników.

Informacje o użytkownikach systemu zapisano w tabeli **manager**, obejmujące nazwę użytkownika (username), hasło (password), rolę (role) oraz informację czy konto jest aktywne (enabled). Tabela ta funkcjonuje niezależnie od pozostałych encji i odpowiada za warstwę autoryzacji oraz zarządzania dostępem do systemu.

Koszty stałe są rejestrowane w tabeli **fixed_cost**, z kolumnami takimi jak typ kosztu (cost_type), wartość (cost), opis (description) oraz data utworzenia (created_at). Tabela ta nie posiada bezpośrednich relacji z innymi tabelami i służy do ewidencjonowania kosztów niezależnych od liczby transakcji, wykorzystywanych m.in. w analizach finansowych i raportach.

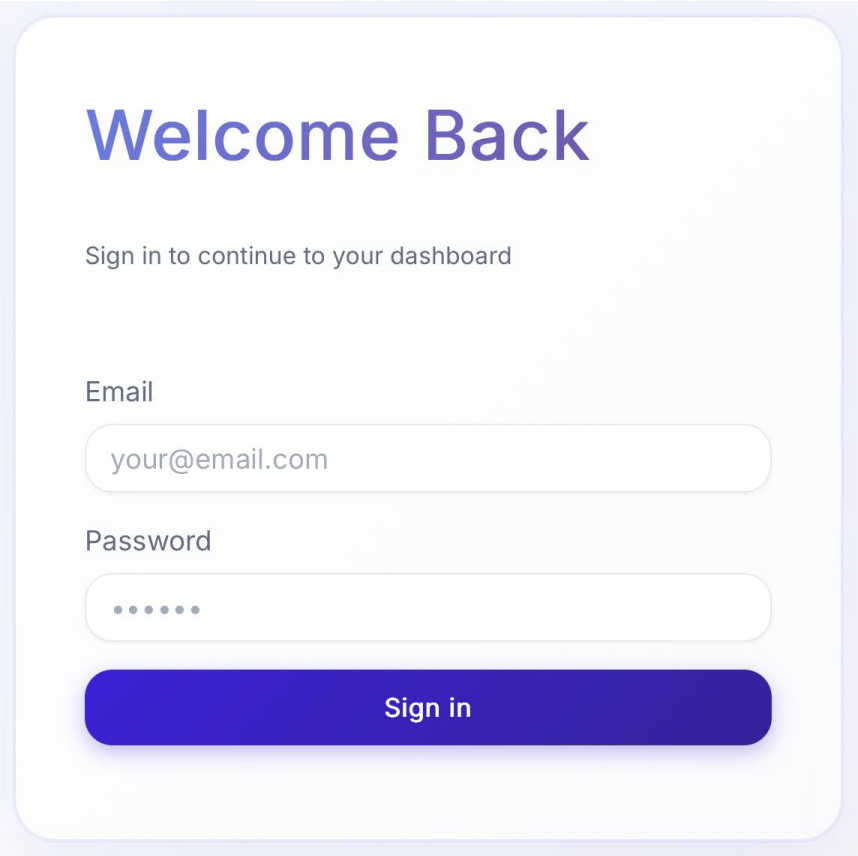
Raporty generowane przez system zapisywane są w tabeli **report**, która zawiera zakres dat obejmowanych przez raport (from_date, to_date), czas wygenerowania (generated_at), nazwę pliku (file_name) oraz dane pliku (file_data).

Tabela report stanowi niezależny element systemu, umożliwiający archiwizację wygenerowanych zestawień bez bezpośrednich powiązań relacyjnych z danymi źródłowymi.

2.3 Projekt interfejsu użytkownika

Zgodnie ze zdefiniowanymi wymaganiami poza funkcjonalnymi, celem przy opracowywaniu interfejsu użytkownika było zapewnienie przejrzystości, intuicyjności oraz funkcjonalności dla użytkowników końcowych. W ramach projektu zdecydowano się na implementację interfejsu użytkownika w architekturze Single Page Application (SPA). SPA to podejście do budowy aplikacji internetowych, w którym cała logika interfejsu jest ładowana do przeglądarki użytkownika jednorazowo, a kolejne interakcje z systemem nie wymagają przeładowywania całej strony.

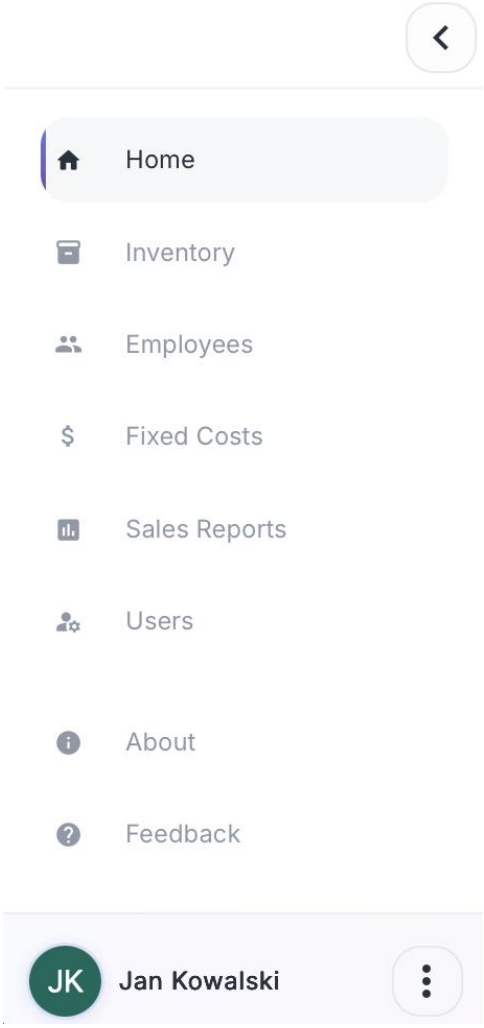
Struktura graficzna została zaplanowana na zasadzie minimalizmu, z ograniczoną paletą kolorów i wyraźną hierarchią informacji, co ułatwia szybkie odnajdywanie kluczowych danych i wykonywanie podstawowych operacji.



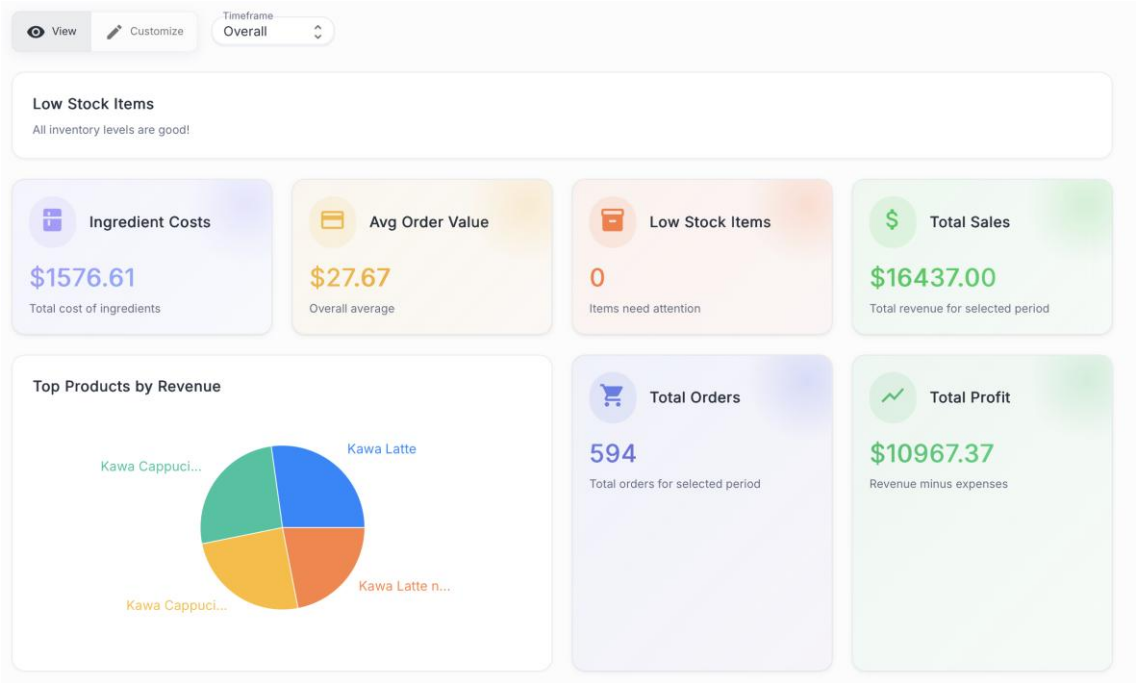
Rys. 2.2.Projekt widoku ekranu logowania. Źródło: Opracowanie własne.

Ekran logowania widoczny na Rys. 2.2. składa się z prostego formularza umożliwiającego uwierzytelnienie użytkownika za pomocą adresu e-mail oraz hasła. Całość umieszczona została w centralnej części ekranu, co pozwala na maksymalne skupienie uwagi użytkownika na logowaniu. W projekcie przewidziano również przycisk zatwierdzający logowanie, który został wyeksponowany kolorystycznie względem tła.

Menu aplikacji będzie umieszczone w panelu nawigacyjnym po lewej stronie ekranu. Przykład wyglądu takiego menu przedstawiono na Rys. 2.3. Zawiera ono opcje prowadzące do najważniejszych sekcji aplikacji, takich jak strona główna (Home), stan magazynowy (Inventory), zarządzanie pracownikami (Employees), koszta stałe (Fixed Costs), raporty sprzedaży (Sales Reports), oraz w przypadku roli admina dla zalogowanego użytkownika – sekcję do zarządzania użytkownikami aplikacji (Users). W panelu zaplanowano również odrębną sekcję dolną przeznaczoną na elementy pomocnicze, takie jak informacje o aplikacji oraz formularz opinii. Stylizacja komponentu nawiązuje do klasycznych wzorców projektowych spotykanych w aplikacjach administracyjnych.

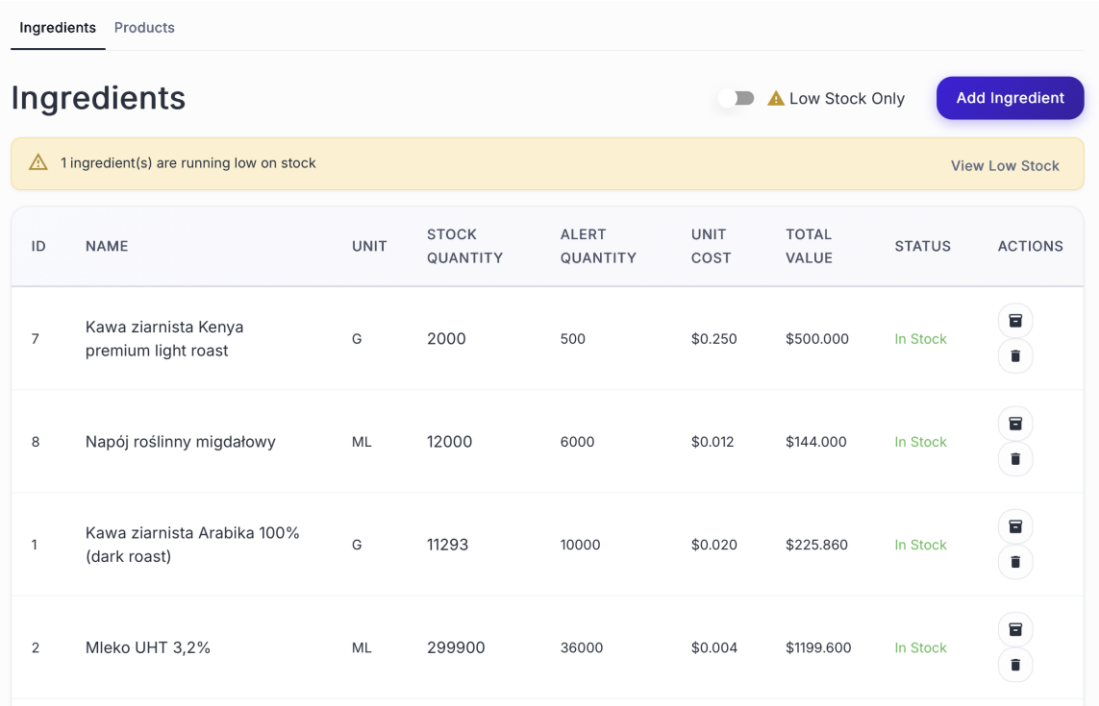


Rys. 2.3.Projekt widoku menu nawigacyjnego. Źródło: Opracowanie własne.



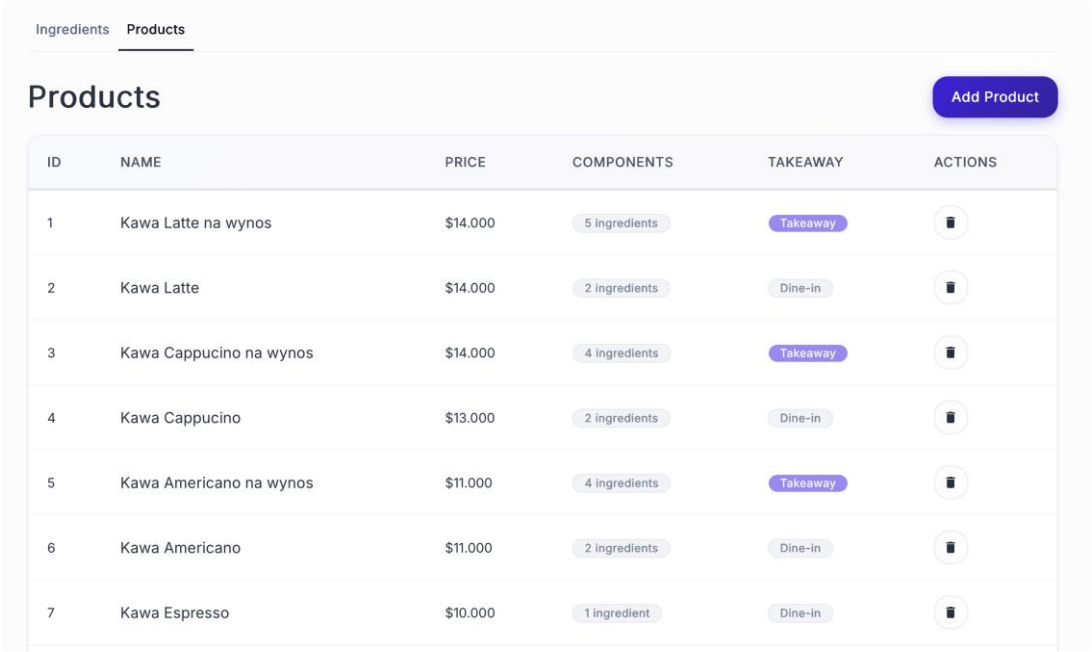
Rys. 2.4.Projekt widoku głównego panelu. Źródło: Opracowanie własne.

Projekt widoku głównego panelu przedstawiono na Rys. 2.4. Na prostych i czytelnych kartach znajdują się istotne dla działalności wartości oraz wykresy ilustrujące statystyki. Użytkownik ma możliwość wyświetlania danych w danym przedziale czasowym, korzystając z przycisku „TimeFrame”, znajdującego się na górze panelu, gdzie może wybrać dane zliczane od początku działania aplikacji, jak również w przedziale dziennym, tygodniowym lub miesięcznym licząc od dnia bieżącego. Istnieje również możliwość dostosowania wyglądu panelu głównego według własnych preferencji, korzystając z zakładki „Customize”, na co składa się między innymi dodanie, usuwanie lub zmiana położenia poszczególnych kart.



Rys. 2.5.Projekt widoku panelu składników stanu magazynowego. Źródło: Opracowanie własne.

Projekty dedykowanych widoków do zarządzania stanem magazynowym przedstawiono na Rys. 2.5. oraz Rys. 2.6. Tabela składników stanu magazynowego (Ingredients) przedstawia listę wszystkich produktów wraz z ich identyfikatorami, kategorią, aktualnym stanem magazynowym, ceną jednostkową, progiem minimalnym oraz przyciskami umożliwiającymi edycję lub usunięcie pozycji. W górnej części interfejsu zaplanowany został również pasek informacyjny ostrzegający o niskim stanie magazynowym.



Rys. 2.6.Projekt widoku panelu produktów stanu magazynowego. Źródło: Opracowanie własne.

Tabela produktów stanu magazynowego (Products) przedstawia natomiast dane produktów do sprzedaży. W kolumnach umieszczono kolejno identyfikator, nazwę produktu, cenę, liczbę komponentów z jakich się składa, oznaczenie czy produkt jest na wynos (takeaway) czy w lokalu (dine-in), a także przycisk pozwalający dany produkt usunąć.

Na górze tabel umieszczone zostały również przyciski umożliwiające dodanie nowego składnika bądź produktu – kolejno „Add Ingredient" oraz „Add Product".

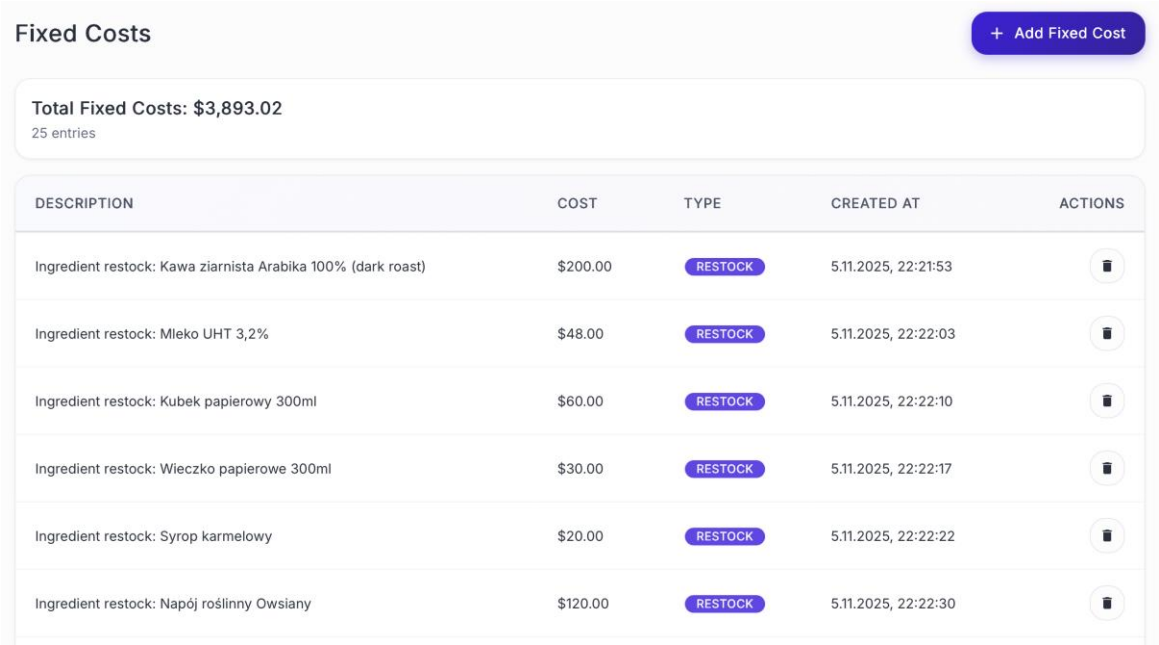
Employees

Add Employee

ID	NAME	EMAIL	PHONE	SALARY/HOUR	HOURS WORKED	ACTIONS
1	Anna Kowalska	anna.kowalska@coffeeshop.pl	600123456	$25.00	160	
2	Piotr Nowak	piotr.nowak@coffeeshop.pl	601987654	$26.50	145	
3	Katarzyna Wiśniewska	katarzyna.wisniewska@coffeeshop.pl	602543210	$27.00	158	
4	Tomasz Lewandowski	tomasz.lewandowski@coffeeshop.pl	603876543	$24.50	172	
5	Julia Wójcik	julia.wojcik@coffeeshop.pl	604112233	$25.00	150	

Rys. 2.7.Projekt widoku panelu pracowników. Źródło: Opracowanie własne.

Tabela zawierająca dane wszystkich pracowników, która została przedstawiona na Rys. 2.7. ukazuje kolumny takie jak identyfikator, nazwa pracownika, email, numer telefonu, stawkę godzinową, liczbę przepracowanych godzin oraz przyciski pozwalające na wykonanie akcji – zaktualizowania liczby godzin o odpowiednią wartość, wypłatę wynagrodzenia oraz usunięcie pracownika. Podobnie jak w przypadku panelu stanu magazynowego – na górze umieszczony został przycisk umożliwiający dodanie nowego pracownika „Add Employee".

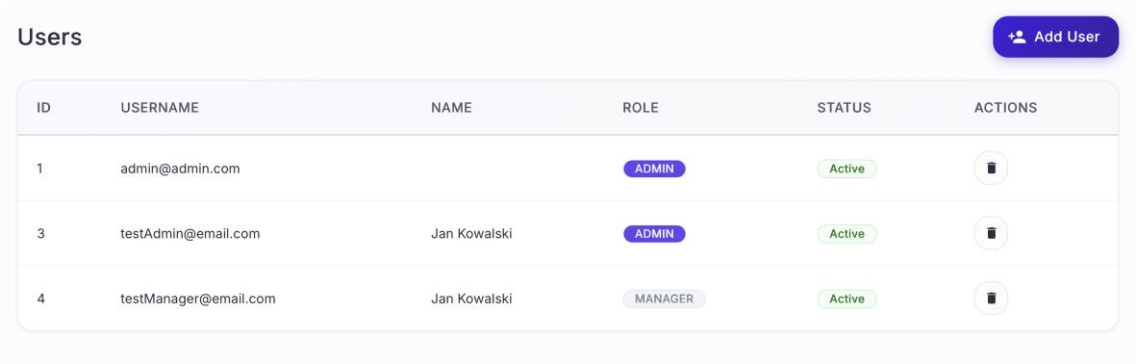


Rys. 2.8.Projekt widoku panelu kosztów stałych. Źródło: Opracowanie własne.

Na Rys. 2.8 przedstawiono koszty stałe, gdzie poza opisem danego kosztu, wyszczególniono również kwotę kosztu, typ, znacznik czasowy oraz przycisk usuwania danego kosztu. Przycisk służący do dodania nowego kosztu stałego znajduję się na górze tabeli i nosi nazwę „Add Fixed Cost”.

Rys. 2.9.Projekt widoku panelu raportów sprzedażowych. Źródło: Opracowanie własne.

Rys. 2.9. przedstawia raporty sprzedażowe, które użytkownik ma możliwość wygenerować za dany okres korzystając z przycisku na górze tabeli „Generate New Report”. Na tabelę składa się nazwa raportu, przedział czasowy, znacznik czasowy - kiedy dany raport został wygenerowany, nazwa pliku oraz przycisk służący do pobrania danego raportu.



Rys. 2.10.Projekt widoku panelu użytkowników aplikacji. Źródło: Opracowanie własne.

Zalogowany użytkownik, do którego konta przypisana została rola admina ma również możliwość korzystania z panelu użytkowników (Users) ukazanego na Rys. 2.10., gdzie ma możliwość dodania nowego użytkownika aplikacji, lub usunięcia już istniejących, korzystając kolejno z przycisków „Add User", oraz przycisku do usuwania danego użytkownika znajdującego się w tabeli. Poza przyciskiem usuwania, tabela składa się z kolumn takich jak identyfikator, nazwa użytkownika – email, imię i nazwisko użytkownika, jego rola oraz status.

Wykonany projekt interfejsu powinien zapewnić jak największy minimalizm, a jednocześnie umożliwić łatwą nawigację. Większość widoków będzie mogła zostać zaimplementowana z wykorzystaniem komponentów biblioteki Material UI, przy jednoczesnym zachowaniu spójności wizualnej i funkcjonalnej całej aplikacji.

2.4 Wybór technologii i narzędzii

Do implementacji systemu GastroM zaplanowano wykorzystanie określonych technologii i narzędzi. Wybór konkretnych rozwiązań wynika z wymagań projektowych oraz wcześniejszych doświadczeń autorów.

Do implementacji warstwy backend zaplanowano użycie języka Java oraz frameworka Spring Boot, którego szczegółowy opis znajduje się w podrozdziale 1.3. Wybór tej technologii został uzasadniony następującymi cechami:

1. Wsparcie dla architektury REST, co umożliwia komunikację z warstwą klienta za pomocą formatu JSON.
2. Wbudowany serwer HTTP (Tomcat), umożliwiający uruchamianie aplikacji bez konieczności konfiguracji zewnętrznego środowiska serwerowego.
3. Strukturalny podział aplikacji na warstwy (kontrolery, serwisy, repozytoria), co pozwala na rozdzielenie odpowiedzialności poszczególnych komponentów.

4. Automatyczna współpraca z frameworkiem ORM HIbernate, który jest implementacją JPA, umożliwiająca realizację operacji na bazie danych z zachowaniem zgodności z podejściem obiektowym.
5. Zintegrowane mechanizmy uwierzytelniania i autoryzacji (Spring Security), które umożliwiają kontrolę dostępu do zasobów systemu.
6. Możliwość integracji z systemami baz danych, w tym PostgreSQL, w sposób zgodny z założeniami projektu.

Dodatkowo, w projekcie zaplanowano wykorzystanie narzędzia Gradle jako systemu automatyzacji budowania aplikacji. Wybór Gradle został podyktowany następującymi właściwościami:

1. Możliwość deklaratywnego zarządzania zależnościami oraz pluginami, co usprawnia konfigurację projektu zarówno po stronie backend, jak i części wspólnych modułów,
2. Wsparcie dla wielomodułowej struktury aplikacji, co pozwala na przejrzyste organizowanie kodu oraz oddzielenie warstwy backendowej od pozostałych komponentów systemu,
3. Wysoka wydajność procesu budowania dzięki mechanizmom takim jak cache buildów oraz kompilacja przyrostowa,
4. Pełna integracja ze Spring Boot poprzez dedykowane pluginy (m.in. spring-boot oraz dependency-management), umożliwiająca łatwe uruchamianie aplikacji oraz generowanie artefaktów wykonawczych,
5. Zgodność z popularnymi środowiskami programistycznymi, w tym IntelliJ IDEA, umożliwiająca automatyczne importowanie konfiguracji projektu,
6. Możliwość automatyzacji uruchamiania testów jednostkowych i integracyjnych (JUnit, Mockito) dzięki wbudowanemu wsparciu dla ekosystemu testowego JVM.

Do przechowywania danych zaplanowano wykorzystanie systemu zarządzania relacyjną bazą danych PostgreSQL opisaną szczegółowo w podrozdziale 1.3.8, a którego wybór został uzasadniony następującymi właściwościami:

1. Zdolność obsługi zwiększającej się liczby danych i użytkowników,
2. Wsparcie dla transakcji ACID, umożliwiające zapewnienie integralności i spójności danych.
3. Obsługa zapytań SQL, indeksów oraz funkcji użytkownika,

4. Możliwość współpracy ze Spring Boot poprzez framework Hibernate ORM, umożliwiająca implementację warstwy dostępu do danych zgodnie z przyjętymi założeniami.

Realizacja interfejsu użytkownika została zaplanowana z użyciem biblioteki React.js, opisanej w podrozdziale 1.2. Wybór ten został uzasadniony poniższymi właściwościami:

1. Komponentowa architektura umożliwiająca modularne budowanie struktury interfejsu,
2. Zastosowanie wirtualnego DOM, umożliwiające aktualizację widoku bez przeładowania całej strony,
3. Zarządzanie stanem aplikacji za pomocą Context API,
4. Możliwość komunikacji z backend poprzez REST API.

Z uwagi na planowane użycie biblioteki React, do implementacji warstwy frontend zaplanowano również użycie języka programowania TypeScript, czyli nadzbioru JavaScript. React obsługuje oba wyżej wymienione języki programowania, jednak TypeScript wybrano ze względu na:

1. Możliwość stosowania statycznego typowania danych, co redukuje liczbę błędów w fazie programowania oraz usprawnia proces debugowania aplikacji
2. Lepszą współpracę z narzędziami wspomagającymi proces wytwórczy takimi jak środowiska zintegrowane IDE, które dzięki typom oferują funkcje autouzupełniania oraz analizę składniową.

W projekcie przewidziano wykorzystanie Hibernate jako implementacji standardu JPA (Java Persistence API). Umożliwia to mapowanie obiektowo-relacyjne i realizację operacji na danych. W szczególności uwzględniono:

- mapowanie encji na tabele bazy danych,
- obsługę operacji CRUD z wykorzystaniem podejścia obiektowego,
- dostępność mechanizmów takich jak lazy loading, które ograniczają wykonywanie zbędnych zapytań do bazy danych.

W celu przeprowadzenia testów funkcjonalnych i jednostkowych zaplanowano użycie następujących narzędzi:

1. JUnit, czyli framework do testów jednostkowych w języku Java, który umożliwia łatwe testowanie logiki aplikacji oraz integruje się z popularnymi narzędziami programistycznymi.

2. Mockito, czyli biblioteka ułatwiająca testowanie jednostkowe dzięki możliwości wykorzystania tzw. atrap zależności (mocków), pozwalających na izolowanie komponentów podczas testowania i kontrolowanie zachowań zależnych obiektów bez potrzeby implementowania ich pełnej funkcjonalności.
3. Playwright, tzn. framework do automatyzacji testowania aplikacji frontend umożliwiający symulację zachowań użytkownika i automatyczne sprawdzanie poprawności działania interfejsu.

Do realizacji projektu przewidziano także użycie następujących narzędzi deweloperskich:

1. IntelliJ IDEA jako środowisko programistyczne dla kodu w języku Java oraz TypeScript, a także do testowania zapytań do bazy danych.
2. Springdoc OpenAPI do testowania komunikacji z interfejsem REST API, a także automatycznego generowania dokumentacji API.
3. Postman do testowania komunikacji z interfejsem REST API.
4. Git oraz Github jako narzędzia kontroli wersji.
5. Render.com jako platformy chmurowej do udostępnienia aplikacji w środowisku publicznym.
6. Neon.com jako platformy chmurowej hostującej bazę danych PostgreSQL.

3. IMPLEMENTACJA WARSTWY BACKEND (IGOR KOHSIN)

Celem warstwy backend systemu GastroM jest implementacja funkcji odpowiedzialnych za pobieranie i przetwarzanie danych związanych z zarządzaniem składnikami, produktami sprzedaży oraz użytkownikami, w tym obsługę bazy danych, a także przygotowanie odpowiednich informacji dla strony klienta w postaci REST API. Zgodnie z projektem, do implementacji warstwy backend wykorzystano język i technologie Java oraz framework Spring. W kolejnych podrozdziałach przedstawiono konfigurację środowiska projektowego, struktury systemu, model przechowywania danych, dostęp do danych, warstwę prezentacji dla klienta zewnętrznego, logikę biznesową, klasy pomocnicze, bezpieczeństwo oraz testowanie aplikacji.

3.1 Konfiguracja środowiska projektowego

Zgodnie z proektem do implementacji warstwy backend wykorzystano język Java, wersja 17, oraz framework Spring Boot, wersja 3.5.6. Do automatyzacji budowania aplikacji wykorzystano Gradle opisany w podrozdziale 2.3. Plik konfiguracyjny projektu build.gradle został przedstawiony na Rys. 3.1. Zdefiniowano tu zależności projektu oraz parametry kompilacji.

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.6'
    id 'io.spring.dependency-management' version '1.1.7'
}

group = 'pl.obrona'
version = '0.0.1-SNAPSHOT'
description = 'management-api'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(17)
    }
}

configurations {
    compileOnly {
        extendsFrom annotationProcessor
    }
}

repositories {
    mavenCentral()
}

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-data-
jpa'
```

```
    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springdoc:springdoc-openapi-starter-webmvc-
ui:2.8.5'
    implementation 'com.itextpdf:itextpdf:5.5.13.3'
    implementation 'io.jsonwebtoken:jjwt-api:0.11.5'
    implementation 'io.jsonwebtoken:jjwt-impl:0.11.5'
    implementation 'io.jsonwebtoken:jjwt-jackson:0.11.5'
    implementation 'org.springframework.boot:spring-boot-starter-web-
services'
    implementation 'org.springframework.boot:spring-boot-starter-
security'
    testImplementation 'io.projectreactor:reactor-test'
    testImplementation 'org.springframework.security:spring-security-
test'
    runtimeOnly 'com.mysql:mysql-connector-j'
    compileOnly 'org.projectlombok:lombok'
    runtimeOnly 'org.postgresql:postgresql'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-
test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

tasks.named('test') {
    useJUnitPlatform()
}
```

Rys. 3.1. Kod skryptu budowania dla Gradle w pliku buid.gradle. Źródło: Opracowanie własne.

W projekcie zastosowano następujące wtyczki (plugins):

- org.springframework.boot, która umożliwia budowanie i zarządzanie aplikacją Spring Boot w Gradle,
- io.spring.dependency-management, zarządza wersjami zależności,
- java, obsługuje kompilację kodu języka Java.

Jako źródło zależności wskazano repozytorium Maven Central, natomiast użyte zależności w projekcie to:

- Spring Boot Starter Web, do tworzenia aplikacji webowych i REST API,
- Spring Boot Starter Data JPA, w celu ułatwienia interakcji z bazą danych,
- Spring Boot Starter Security, do uwierzytelniania i autoryzacji,
- Springdoc OpenAPI, użyte do generowania dokumentacji API,
- iTextPDF, do generowania dokumentów PDF,
- JJWT, do obsługi tokenów JWT,
- Lombok, do automatycznego generowania kodu szablonowego (boilerplate code).

W projekcie wykorzystano również sterowniki bazy danych PostgreSQL. Do pisania i uruchamiania testów dołączona została biblioteka JUnit 5.

Głównym plikiem konfiguracyjnym aplikacji jest application.yml, który zawiera parametry połączenia z bazą danych, jego zawartość została przedstawiona na Rys. 3.2.

```
spring:
  datasource:
    url: ${JDBC_URL}
  jpa:
    hibernate:
      ddl-auto: update
```

Rys. 3.2. Konfiguracja połączenia z bazą danych i ustawień JPA w pliku application.yml.
Źródło: Opracowanie własne.

Jak widać, aplikacja korzysta z dynamicznej konfiguracji połączenia z bazą danych przy użyciu zmiennej środowiskowej JDBC_URL, której wartość definiuje pełny adres połączenia w formacie JDBC.

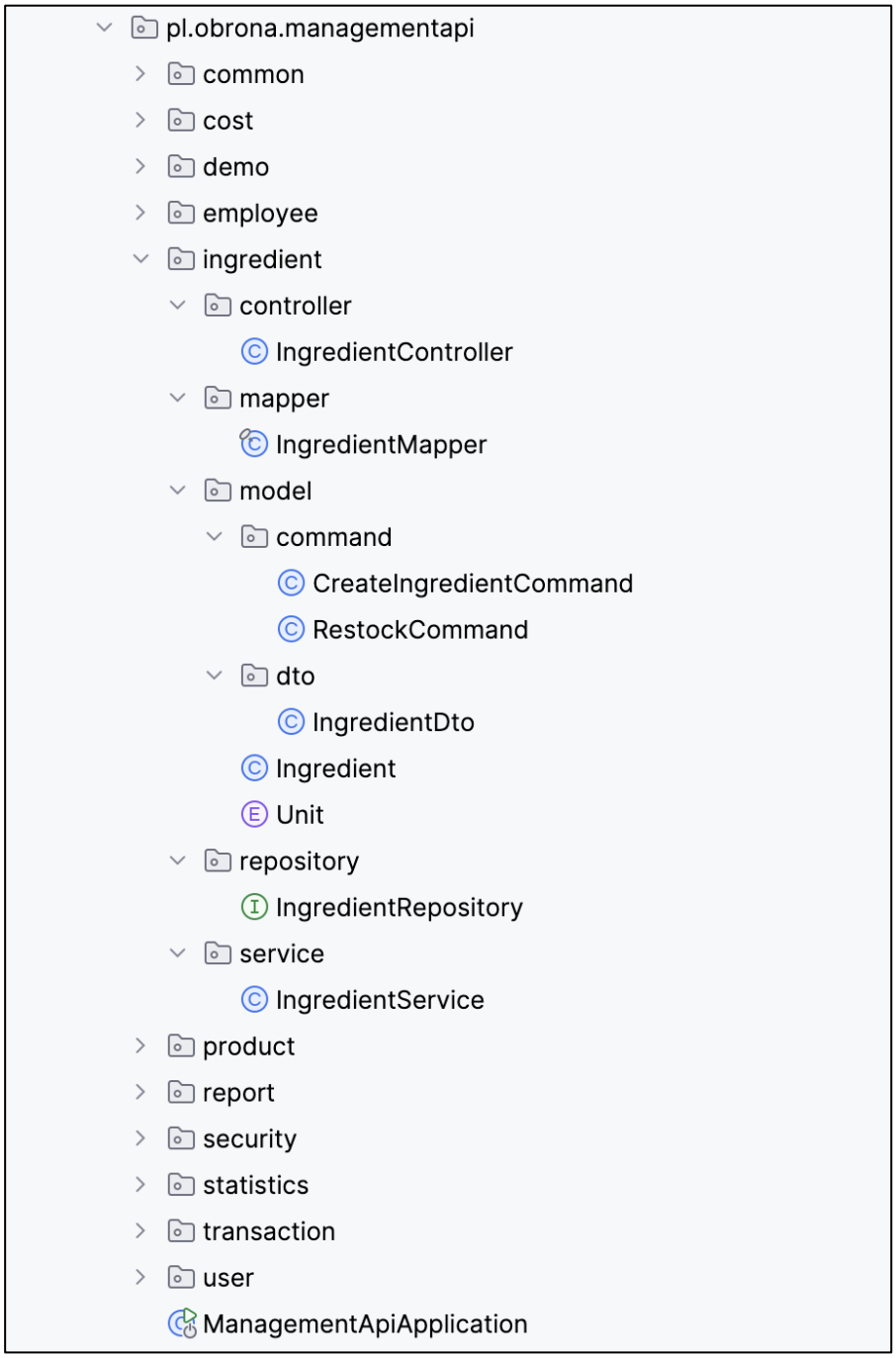
*JDBC_URL=jdbc:postgresql://ep-frosty-tooth-adka9o0r-pooler.c-2.us-east-1.aws.neon.tech/neondb?user=neondb_owner&password=********&sslmode=require&channelBinding=require*

Dzięki zastosowaniu zmiennej środowiskowej, wrażliwe dane dostępowe, takie jak hasło użytkownika, nie są przechowywane bezpośrednio w plikach konfiguracyjnych projektu. Zmienna środowiskowa ustawiona została bezpośrednio na platformie hostingowej Render.com.

Ustawienie `spring.jpa.hibernate.ddl-auto=update` pozwala na automatyczne aktualizowanie schematu bazy danych na podstawie encji zdefiniowanych w kodzie.

3.2 Struktura systemu

Struktura aplikacji została zaprojektowana w oparciu o architekturę warstwową z jednoczesnym zastosowaniem podejścia feature-based. Kod źródłowy został zorganizowany w pakiety odpowiadające poszczególnym obszarom funkcjonalnym systemu, takim jak ingredient, product, user czy transaction. W ramach każdego pakietu wydzielono elementy warstwy prezentacji, logiki biznesowej, dostępu do danych oraz modelu, co umożliwia jednoznaczne przypisanie komponentów do konkretnej funkcjonalności systemu.



Rys. 3.3. Struktura aplikacji. Źródło: Opracowanie własne.

Każdy obszar może być rozwijany w sposób niezależny, zachowując przy tym spójność z ogólną architekturą aplikacji. Na każdy z nich składają się warstwy:

- *model*, zawierający klasy obszaru funkcjonalnego, w tym m.in. encje odwzorowujące tabele w bazie danych, ale również typy wyliczeniowe definiujące zbiór nazwanych stałych;
- *repository*, definiujący interfejsy odpowiedzialne za komunikację z bazą danych z wykorzystaniem mechanizmów Spring Data JPA;

- *service*, który implementuje logikę biznesową, przetwarza dane oraz koordynuje współpracę pomiędzy kontrolerami a repozytoriami;
- *controller*, odpowiadający za obsługę żądań HTTP w ramach interfejsu REST API oraz za przekazywanie odpowiedzi do warstwy klienta;

W przypadku niektórych obszarów funkcjonalnych, wyróżnić można również dodatkowo:

- *command*, opisujący działania wykorzystywane przy tworzeniu lub modyfikacji danych;
- *dto*, obejmujący obiekty do przekazywania danych (Data transfer objects);
- *mapper*, zawierający klasy pomocnicze odpowiedzialne za przekształcanie danych między encjami, dto i komendami;

3.3 Model przechowywania danych

Model przechowywania danych, nazywany również modelem trwałości (persistance model) stanowi odwzorowanie struktury danych przechowywanych w bazie danych w kodzie programu. W ramach modelu dla aplikacji GastroM zdefiniowano klasy reprezentujące dane aplikacji, takie jak składniki (klasa Ingredient), produkty (klasa Products), użytkownicy (klasa User) czy transakcje (klasa Transaction).

Dodatkowo w modelu zdefiniowano typy wyliczeniowe (enum), które służą do przechowywania stałych wartości, takich jak jednostki miary składników – Unit, dopuszczający wartości ML, G oraz PCS.

3.3.1 Encje

Odpowiednie klasy zostały oznaczone adnotacją *@Entity*, tworząc tym samym encje, co umożliwia ich mapowanie na odpowiadające tabele w relacyjnej bazie danych z wykorzystaniem mechanizmu ORM (ang. Object-Relational Mapping) dostarczanego przez bibliotekę Hibernate.

Encje zawierają pola odpowiadające kolumnom tabel, wraz z adnotacjami definiującymi m.in. typy danych, klucze główne i relacje między tabelami. Przykładowo, klasa Ingredient, ukazana na Rys. 3.4. reprezentująca składnik produktu, zawiera m.in. nazwę czy jednostkę miary.

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
@SoftDelete
@Entity
public class Ingredient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @Enumerated(EnumType.STRING)
    private Unit unit;

    private BigDecimal stockQuantity;
    @Column(precision = 3, scale = 5)
    private BigDecimal unitCost;
    private BigDecimal alertQuantity;

}
```

Rys 3.4. Kod klasy encji Ingredient. Źródło: Opracowanie własne.

Tak jak zostało wcześniej wspomniane, klasa została oznaczona adnotacją *@Entity* co pozwala Hibernate automatycznie mapować ją na tabelę w bazie danych. Klasa oznaczona została również adnotacjami z biblioteki Lombok *(@Getter, @Setter, @AllArgsConstructor, @NoArgsConstructor, @Builder*), które eliminują potrzebę ręcznego tworzenia konstruktorów, metod dostępowych i budowniczych. Dodatkowo, dzięki adnotacji *@SoftDelete*, encja wspiera miękkie usuwanie (soft delete), co oznacza, że rekord nie jest trwale usuwany z bazy, lecz jedynie oznaczany jako usunięty.

Z uwagi na adnotację *@Entity*, encja musi mieć klucz główny, oznaczony adnotacją *@Id*, a w tym przypadku jest on generowane automatycznie dzięki adnotacji *@GeneratedValue(strategy = GenerationType.IDENTITY)* i przechowywany w polu typu Long o nazwie id.

Wśród pól klasy Ingredient, wyróżnić można także:

1. name, typu String, zawierające nazwę danego składnika,
2. unit, gdzie zastosowana adnotacja @Enumerated(EnumType.STRING) określa, że pole unit jest typem wyliczeniowym i jego wartość będzie zapisywana w bazie danych jako tekst (np. G, ML, PCS) i oznacza jednostkę miary,

3. stockQuantity typu BigDecimal oznaczające ilość danego składnika w magazynie,
4. unitCost typu BigDecimal oznaczające cenę jednostkową danego składnika, gdzie przykładowo cena wynosząca 0.4 z jednostką PCS oznacza cenę 40 gr. za każdą sztukę składnika,
5. alertQuantity typu BigDecimal oznaczające ustaloną ilość składnika wskazującą na niski poziom względem ilości całkowitej i wymagającej uwagi.

Zastosowanie typu BigDecimal gwarantuje precyzyjne obliczenia, unikane są tym samym zaokrąglenia mogące mieć wpływ na końcowy rezultat operacji co ma znaczenie w kontekście kosztów i stanów magazynowych. Pole unitCost zostało dodatkowo opisane adnotacją *@Column(precision = 3, scale = 5),* co określa precyzję i dokładność wartości liczbowych przechowywanych w bazie danych, które ma zastosowanie jedynie w przypadku użycia typu BigDecimal.

W systemie GastroM oprócz opisanego Ingredient zdefiniowane zostały następujące modele: FixedCost, Employee, Product, ProductComponent, Report, Transaction, TransactionProduct, User. Zdefiniowano również następujące typy wyliczeniowe: CostType, Unit, StatisticsRange, PaymentMethod, Role.

3.3.2 Command i DTO

W celu zapewnienia rozdzielenia warstw logiki biznesowej od warstwy prezentacji oraz wprowadzenia większej czytelności kodu, w aplikacji wykorzystano wzorce Command i DTO (Data Transfer Object). Ich zastosowanie pozwala na ograniczenie bezpośredniego dostępu do encji oraz umożliwia określenie zakresu danych przekazywanych między poszczególnymi warstwami systemu.

Obiekty komend (ang. commands) w opisywanej aplikacji służą do przekazywania danych wejściowych do logiki biznesowej podczas tworzenia lub modyfikowania encji. Przykładem może być obiekt klasy CreateIngredientCommand, której kod przedstawiono na Rys. 3.5. Odpowiada ona za przekazywanie danych wymaganych do utworzenia nowego składnika w systemie.

```
@Data
@Builder
public class CreateIngredientCommand {
    private String name;
    private Unit unit;
    private BigDecimal stockQuantity;
    private BigDecimal alertQuantity;
```

```
    private BigDecimal unitCost;
}
```

Rys. 3.5. Kod klasy CreateIngredientCommand. Źródło: Opracowanie własne.

Klasa ta zawiera pola odpowiadające informacjom niezbędnym do zarejestrowania nowego składnika: nazwę, jednostkę miary, ilość początkową, ilość ostrzegawczą oraz koszt jednostkowy. Dzięki zastosowaniu adnotacji z biblioteki Lombok (@Data, @Builder), klasa zyskuje automatycznie generowane metody dostępowe oraz budownicze. Pozwala to na tworzenie obiektów komend w prosty i czytelny sposób. Komenda ta jest wykorzystywana m.in. w kontrolerze przy obsłudze żądania HTTP POST odpowiedzialnego za dodanie nowego składnika.

Drugim przykładem komendy w aplikacji jest obiekt klasy RestockCommand, której kod przedstawiono na Rys. 3.6. Obiekt taki służy do przekazywania informacji o ilości składnika, o jaką należy zwiększyć jego aktualny stan magazynowy. Pole quantity reprezentuje liczbę jednostek, o które należy zwiększyć zapas danego składnika, jego wartość jest typu BigDecimal.

```
@Data
@Builder
public class RestockCommand {
    @PositiveOrZero
    private BigDecimal quantity;
}
```

Rys. 3.6. Kod klasy RestockCommand. Źródło: Opracowanie własne.

Zastosowana adnotacja @PositiveOrZero weryfikuje poprawność danych wejściowych, gwarantując, że przekazana wartość nie będzie ujemna, co chroni system przed błędami logicznymi. Oprócz wymienionych w systemie zaimplementowano następujące obiekty komend: CreateFixedCostCommand, CreateEmployeeCommand, CreateIngredientCommand, RestockCommand, ComponentCommand, CreateProductCommand, CreateReportCommand, CreateTransactionCommand oraz CreateUserCommand.

Obiekty DTO służą przede wszystkim do przekazywania danych między warstwą aplikacji a klientem (np. interfejsem użytkownika lub systemem zewnętrznym). Ich zaletą jest możliwość oddzielenia logiki wewnętrznej aplikacji od sposobu prezentacji danych oraz ograniczenie ilości informacji zwracanych w odpowiedziach API. Przykładem jest klasa IngredientDto, zaprezentowana na Rys. 3.7., która reprezentuje dane zwracane w odpowiedzi na zapytania dotyczące składników.

```
@Getter
@Builder
public class IngredientDto {
    private Long id;
    private String name;
    private Unit unit;
    private BigDecimal stockQuantity;
    private BigDecimal alertQuantity;
    private BigDecimal unitCost;
}
```

Rys. 3.7. Kod klasy IngredientDto. Źródło: Opracowanie własne.

Obiekt IngredientDto zawiera wyłącznie dane niezbędne do prezentacji składnika, dzięki czemu nie dochodzi do przekazywania zbędnych danych ani wewnętrznych struktur encji.

W praktyce, obiekty klas Ingredient i IngredientDto są konwertowane przy użyciu klasy Mapper, która odpowiada za konwersję danych pomiędzy różnymi reprezentacjami obiektów, np. między obiektami encji i dto. W systemie GastroM zaimplementowane zostały następujące klasy Dto: EmployeeDto, IngredientDto, ProductComponentDto, ProductDto, StatisticsDto, ReportStatisticsDto, ExpandedStatisticsDto i TransactionDto.

3.4 Dostęp do danych

Dostęp do danych w aplikacji realizowany jest z wykorzystaniem mechanizmu Spring Data JPA, który stanowi część frameworka Spring i umożliwia obsługę operacji na bazie danych w sposób deklaratywny, bez konieczności pisania rozbudowanego kodu SQL. Spring Data JPA implementuje klasyczny wzorzec Repository, którego zadaniem jest oddzielenie logiki biznesowej od warstwy dostępu do danych.

Podstawową jednostką odpowiedzialną za komunikację z bazą danych są interfejsy repozytoriów, które rozszerzają wbudowany interfejs JpaRepository dostarczany przez Spring Data. Przykład kodu można zobaczyć na Rys. 3.8. Dzięki JpaRepository aplikacja automatycznie zyskuje dostęp do gotowych metod takich jak m.in. *save(), findById(), findAll(), deleteById*() czy *existsById*(), bez potrzeby ich ręcznej implementacji.

```
public interface IngredientRepository extends
JpaRepository<Ingredient, Long> {
```

```
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    Optional<Ingredient> findWithLockingById(Long id);

    @Query("""
            select i
            from Ingredient i
            where ((:lowStock is null or :lowStock is false) or
i.stockQuantity <= i.alertQuantity)
            """)
    List<Ingredient> findAllByLowStock(@Param("lowStock") Boolean
lowStock);

    @Query("SELECT i FROM Ingredient i WHERE i.stockQuantity <
i.alertQuantity")
    List<Ingredient> findLowStock();
}
```

Rys. 3.8. Kod interfejsu IngredientRepository. Źródło: Opracowanie własne.

Przedstawiony na Rys. 3.8. interfejs IngredientRepository rozszerza JpaRepository, definiując parametry jako klasę encji (Ingredient) oraz typ klucza głównego tej klasy (Long).

Pierwszą z metod zaimplementowanych w interfejsie jest metoda *findWithLockingById(),* która umożliwia pobranie składnika z bazy danych filtrując go po identyfikatorze (id), z zastosowaniem blokady zapisu. Oznacza to, że rekord zostaje zablokowany do momentu zwolnienia blokady, co zapobiega równoczesnym modyfikacjom tego samego składnika przez różne wątki. Zastosowanie adnotacji @Lock(LockModeType.PESSIMISTIC_WRITE) zapobiega wystąpieniu problemów współbieżności, takich jak utrata aktualizacji (lost update) czy niespójność stanów magazynowych, które mogłyby pojawić się w przypadku równoczesnych operacji aktualizacji tego samego składnika. Mechanizm ten znajduje zastosowanie w operacjach wymagających zachowania spójności danych, np. podczas aktualizacji stanów magazynowych.

W interfejsie zdefiniowano metody realizujące bardziej zaawansowane operacje, wykorzystujące język zapytań JPQL (Java Persistence Query Language). Jedną z nich jest metoda *findAllByLowStock(),* która służy do pobierania listy składników, które osiągnęły niski poziom zapasów w magazynie. Jeżeli parametr lowStock przyjmuje wartość true, metoda zwraca jedynie te składniki, których aktualna ilość (stockQuantity) jest mniejsza lub równa wartości progowej (alertQuantity). W przeciwnym wypadku zwracane są wszystkie składniki.

Ostatnia metoda, *findLowStock()* jest uproszczoną wersją poprzedniego zapytania i zawsze zwraca tylko składniki, których poziom zapasów znajduje się poniżej wartości

ostrzegawczej. Inne repozytoria zaimplementowane w systemie GastroM to: FixedCostRepository, EmployeeRepository, ProductComponentRepository, ProductRepository, ReportRepository, TransactionProductRepository, TransactionRepository, UserRepository.

3.5 Warstwa prezentacji dla klienta zewnętrznego

Warstwa prezentacji strony backend stanowi w przypadku tej aplikacji interfejs komunikacyjny pomiędzy warstwą frontend i backend. Odpowiada za obsługę żądań HTTP kierowanych do aplikacji, w tym za zwracanie odpowiednich odpowiedzi w formacie JSON, co realizują odpowiednie kontrolery.

Każdy kontroler odpowiada za określony obszar funkcjonalny aplikacji. Klasa IngredientController, której kod przedstawiono na Rys 3.9. obsługuje operacje związane ze składnikami, takie jak dodawanie nowych rekordów, pobieranie listy składników, usuwanie czy aktualizowanie stanów magazynowych.

```
@RestController
@RequestMapping("/api/v1/ingredients")
@RequiredArgsConstructor
public class IngredientController {
    private final IngredientService ingredientService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public IngredientDto create(@RequestBody @Valid
CreateIngredientCommand command) {
        return ingredientService.create(command);
    }

    @GetMapping
    @ResponseStatus(HttpStatus.OK)
    public List<IngredientDto> getAll(@RequestParam(required = false)
Boolean lowStock) {
        return ingredientService.getAll(lowStock);
    }

    @GetMapping("/{id}")
    @ResponseStatus(HttpStatus.OK)
    public IngredientDto getById(@PathVariable Long id) {
        return ingredientService.getById(id);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteById(@PathVariable Long id) {
        ingredientService.deleteById(id);
    }

    @PatchMapping("/restock/{id}")
    @ResponseStatus(HttpStatus.OK)
    public IngredientDto restockById(
```

```
            @PathVariable Long id,
            @RequestBody @Valid RestockCommand command) {
        return ingredientService.restock(id, command);
    }
}
```

Rys. 3.9. Kod klasy IngredientController. Źródło: Opracowanie własne.

Klasa została oznaczona adnotacją *@RestController*, co oznacza, że klasa jest kontrolerem oraz, że odpowiedzi zwracane przez metody tej klasy są automatycznie serializowane do formatu JSON lub XML i wysyłane jako odpowiedzi HTTP. Adnotacja *@RequestMapping("/api/v1/ingredients")* określa prefiks adresu URL, pod którym dostępne są wszystkie endpointy tego kontrolera. Dzięki adnotacji @RequiredArgsConstructor konstruktor z wymaganymi parametrami definiowany jest automatycznie. Ponieważ w tym przypadku jedynym wymaganym parametrem jest klasa IngredientService, obiekt tej klasy jest wstrzykiwany automatycznie poprzez mechanizm injection, co eliminuje potrzebę definiowania kodu konstruktora.

Poszczególne mapowania żądań HTTP, zdefiniowane za pomocą adnotacji Spring, wiążą operacje protokołu HTTP z logiką biznesową aplikacji:

- *@PostMapping* – adnotacja mapuje żądania typu POST, a oznaczona nią metoda umożliwia tworzenie nowych składników w bazie. Dane wejściowe przekazywane są w postaci obiektu CreateIngredientCommand dzięki adnotacji *@RequestBody*, a poprawność danych weryfikowana jest dzięki adnotacji *@Valid*. W przypadku powodzenia operacji zwracany jest kod HTTP 201 Created oraz obiekt IngredientDto reprezentujący utworzony rekord.
- *@GetMapping* – adnotacja mapuje żądanie typu GET, a metoda, która została nią oznaczona pozwala na pobranie wszystkich składników lub tylko tych o niskim stanie magazynowym (poprzez opcjonalny parametr lowStock).
- *@GetMapping("/{id}")* – mapuje żądanie typu GET z parametrem id, a oznaczona nią metoda zwraca obiekt o podanym id na podstawie identyfikatora przekazanego w ścieżce URL.
- *@DeleteMapping("/{id}")* – mapuje żądanie typu DELETE, a metoda, która została nią oznaczona usuwa składnik o wskazanym identyfikatorze, zwracając kod odpowiedzi 204 No Content, co oznacza pomyślne wykonanie operacji bez zwracania treści.
- *@PatchMapping("/restock/{id}")* – mapuje żądania typu PATCH dotyczące uzupełnienia stanu magazynowego. Metoda, która jest opisana tą adnotacją

przyjmuje dane w formie obiektu RestockCommand, w którym określana jest ilość składnika do dodania.

Kontrolery, które zostały zaimplementowane w aplikacji to: FixedCostController, EmployeeController, IngredientController, ProductComponentController, ProductController, ReportController, StatisticsController, TransactionController, UserController.

3.6 Logika biznesowa

Warstwa logiki biznesowej stanowi centralny element backendu aplikacji, odpowiedzialny za przetwarzanie danych, realizację operacji domenowych oraz koordynację współpracy pomiędzy warstwą prezentacji (kontrolerami) a warstwą dostępu do danych (repozytoriami).

W aplikacji GastroM każdy obszar funkcjonalny posiada dedykowaną klasę usługową (serwisową) oznaczaną adnotacją *@Service*. Taka klasa definiuje operacje wykonywane na danym zbiorze danych. Klasą odpowiedzialną za wykonywanie operacji na danych dotyczących składników jest IngredientService, której kod zaprezentowano na Rys. 3.10.

```
@Service
@RequiredArgsConstructor
public class IngredientService {
    private final IngredientRepository ingredientRepository;
    private final FixedCostRepository fixedCostRepository;

    public IngredientDto create(CreateIngredientCommand command) {
        return
mapToDto(ingredientRepository.save(mapFromCommand(command)));
    }

    public List<IngredientDto> getAll(Boolean lowStock) {
        return
ingredientRepository.findAllByLowStock(lowStock).stream()
                .map(IngredientMapper::mapToDto)
                .toList();
    }

    public IngredientDto getById(Long id) {
        Ingredient ingredient = ingredientRepository.findById(id)
                .orElseThrow(() -> new NotFoundException("Ingredient
not found with id: " + id));

        return mapToDto(ingredient);
    }

    public void deleteById(Long id) {
        ingredientRepository.deleteById(id);
    }
```

```
    @Transactional
    public IngredientDto restock(Long id, RestockCommand command) {
        Ingredient ingredient =
ingredientRepository.findWithLockingById(id)
                .orElseThrow(() -> new NotFoundException("Ingredient
not found with id: " + id));

        BigDecimal newQuantity =
ingredient.getStockQuantity().add(command.getQuantity());
        ingredient.setStockQuantity(newQuantity);
        ingredientRepository.save(ingredient);

        BigDecimal costAmount =
command.getQuantity().multiply(ingredient.getUnitCost());

        fixedCostRepository.save(FixedCost.builder()
                .description("Ingredient restock: " +
ingredient.getName())
                .cost(costAmount)
                .costType(CostType.RESTOCK)
                .createdAt(LocalDateTime.now())
                .build()
        );
        return mapToDto(ingredient);
    }

}
```

Rys. 3.10. Kod klasy IngredientService. Źródło: Opracowanie własne.

Tak jak wcześniej wspomniano, klasa IngredientService została oznaczona adnotacją *@Service*, co oznacza, że pełni rolę komponentu warstwy serwisów zarządzanego przez kontener Spring. Dzięki adnotacji *@RequiredArgsConstructor* odpowiednio zdefiniowane pola klasy IngredientRepository oraz FixedCostRepository są automatycznie wstrzykiwane poprzez konstruktor.

W klasie zdefiniowano metody realizujące podstawowe operacje biznesowe:

- create() – tworzy nowy składnik na podstawie danych przekazanych w obiekcie CreateIngredientCommand. Dane te są mapowane na encję Ingredient, zapisywane w bazie danych, a następnie przekształcane w obiekt IngredientDto, który zwracany jest do obiektu wywołującego, w tym przypadku kontrolera;
- getAll() – zwraca listę składników, z możliwością filtrowania po niskim stanie magazynowym poprzez parametr lowStock;
- getById() – wyszukuje składnik po identyfikatorze, a w przypadku jego braku rzuca wyjątek NotFoundException, co umożliwia kontrolerowi zwrócenie odpowiedniego komunikatu błędu;

- deleteById() – usuwa składnik z bazy danych na podstawie przekazanego identyfikatora;

Szczególną rolę pełni metoda *restock*(), oznaczona adnotacją *@Transactional*. Metoda ta realizuje proces uzupełnienia zapasu składnika w następujących krokach:

1. Pobiera składnik z wykorzystaniem blokady zapisu, tzw. pesymistycznej (*findWithLockingById*) , opisanej w podrozdziale 4.4.
2. Aktualizuje ilość dostępnego składnika (stockQuantity) o wartość zadaną w obiekcie RestockCommand.
3. Oblicza koszt uzupełnienia (costAmount) na podstawie ceny jednostkowej i ilości dodanej do magazynu.
4. Zapisuje nowy koszt za pośrednictwem repozytorium FixedCostRepository jako wydatek typu RESTOCK, korzystając z typu wyliczeniowego CostType.

Adnotacja *@Transactional* zdefiniowana w Spring zapewnia, że wszystkie operacje wykonywane w ramach danej metody są traktowane jako jedna, niepodzielna. Oznacza to, że jeśli w trakcie jej realizacji wystąpi jakikolwiek błąd (np. problem z zapisem do bazy danych lub wyjątek w trakcie obliczeń), cała transakcja zostanie automatycznie wycofana (nastąpi tzw. rollback), a stan bazy danych przywrócony do wartości sprzed rozpoczęcia operacji.

W przypadku metody *restock*(), oznacza to, że jeśli np. zapis składnika zakończy się sukcesem, ale zapis kosztu w FixedCostRepository nie powiedzie się, żadne z tych działań nie zostanie ostatecznie zatwierdzone. Zapobiega to powstawaniu niespójności danych — np. sytuacji, w której stan magazynowy zostałby zwiększony, ale koszt zakupu nie zostałby odnotowany.

Dodatkowo, w połączeniu z blokadą pesymistyczną adnotacja ta zapewnia bezpieczeństwo współbieżnego dostępu do danych, uniemożliwiając jednoczesne modyfikacje tego samego składnika przez wielu użytkowników. Dzięki temu aplikacja zachowuje integralność danych nawet w środowisku wielowątkowym i przy dużym obciążeniu.

Kontekst trwałości (ang. persistence context) udostępniany przez Hibernate odpowiada za zarządzanie cyklem życia encji w ramach danej transakcji, czyli metody oznaczonej adnotacją *@Transactional*. Obiekty znajdujące się w tym kontekście mają status managed, co oznacza, że Hibernate śledzi ich stan i automatycznie wykrywa wszelkie zmiany wprowadzane w ich polach. Warto również zaznaczyć, że specyfika

działania adnotacji *@Transactional* sprawia, iż wszystkie modyfikacje obiektów będących w stanie zarządzanym przez kontekst trwałości Hibernate są automatycznie synchronizowane z bazą danych w momencie zatwierdzenia transakcji. Oznacza to, że nie jest konieczne jawne wywoływanie metody *save*() na repozytorium po każdej zmianie pól encji - wystarczy zwrócić zaktualizowany obiekt w formie DTO (np. poprzez *mapToDto*(*ingredient*)), a Hibernate sam zadba o zapis zmian w bazie przy zamykaniu transakcji. Dzięki temu, po zakończeniu transakcji (np. na skutek wyjścia z metody oznaczonej @Transactional), Hibernate wykonuje tzw. dirty checking, porównując aktualny stan obiektów z wcześniejszym, a następnie generuje odpowiednie zapytania UPDATE, synchronizując stan encji z bazą danych bez konieczności ręcznej interwencji programisty. W aplikacji GastroM znajdują się serwisy: FixedCostService, EmployeeService, IngredientService, ProductComponentService, ProductService, PdfReportService, ReportService, StatisticsService, TransactionService oraz UserService.

3.7 Klasy pomocnicze

Klasy pomocnicze pełnią rolę wsparcia dla głównych warstw backend — logiki biznesowej i warstwy prezentacji. W aplikacji GastroM, zaimplementowany zostały klasy oznaczane przyrostkiem Mapper, oraz klasy odpowiadające za obsługę wyjątków z przyrostkiem Exception.

3.7.1 Mappery

Mappery są klasami odpowiedzialnymi za konwersję danych, najczęściej między różnymi warstwami aplikacji, np. encji, DTO i komend.

```
@UtilityClass
public class IngredientMapper {

    public static Ingredient mapFromCommand(CreateIngredientCommand
command) {
        return Ingredient.builder()
                .name(command.getName())
                .unit(command.getUnit())
                .stockQuantity(command.getStockQuantity())
                .alertQuantity(command.getAlertQuantity())
                .unitCost(command.getUnitCost())
                .build();
    }

    public static IngredientDto mapToDto(Ingredient ingredient) {
        return IngredientDto.builder()
                .id(ingredient.getId())
                .name(ingredient.getName())
```

```
                    .unit(ingredient.getUnit())
                    .stockQuantity(ingredient.getStockQuantity())
                    .alertQuantity(ingredient.getAlertQuantity())
                    .unitCost(ingredient.getUnitCost())
                    .build();
    }
}
```

Rys. 3.11. Kod klasy IngredientMapper. Źródło: Opracowanie własne.

Klasa IngredientMapper przedstawiona na Rys. 3.11., została oznaczona adnotacją @UtilityClass z biblioteki Lombok. Oznacza to, że jest statyczną klasą narzędziową, a więc jej wszystkie metody są statyczne i nie wymaga tworzenia instancji klasy.

Wyróżniamy dwie metody służące do konwersji danych, pierwszą z nich jest *mapFromCommand*(), która ma za zadanie konwertować obiekt typu CreateIngredientCommand na encję Ingredient. Metoda ta przypisuje wartości pól obiektu command do odpowiednich pól encji Ingredient. Wykorzystuje w tym celu Builder, którego adnotacją oznaczona została klasa Ingredient. Wykorzystanie wzorca Builder w klasie Ingredient jest istotne, ponieważ umożliwia czytelne i bezpieczne tworzenie obiektów bez konieczności używania rozbudowanych konstruktorów. Zmniejsza to ryzyko pomyłek przy przekazywaniu parametrów, a także ułatwia późniejsze rozszerzanie klasy o nowe pola bez ingerencji w istniejący kod. Drugą metodą klasy IngredientMapper jest *mapToDto*(), która konwertuje encję Ingredient na obiekt typu IngredientDto, który z kolei zwracany jest do warstwy prezentacji, zapewniając separację modelu domenowego od warstwy klienta. Mechanizmy działania tych metod nie różnią się, natomiast różni je to, że w przypadku mapToDto, zwracane jest również pole id. Mappery w GastroM to: EmployeeMapper, IngredientMapper, ProductComponentMapper, ProductMapper, StatisticsMapper i TransactionMapper.

3.7.2 Obsługa wyjątków

Na potrzeby systemu zdefiniowano 2 własne wyjątki w celu ułatwienia obsługi sytuacji niepożądanych. Wyjątkami zdefiniowanymi w aplikacji są ApiException oraz NotFoundException ukazanymi kolejno na Rys 3.12. oraz Rys. 3.13.

```
public class ApiException extends RuntimeException {
    public ApiException(String message) {
        super(message);
    }
}
```

Rys. 3.12. Kod klasy ApiException. Źródło: Opracowanie własne.

```
public class NotFoundException extends RuntimeException {
    public NotFoundException(String message) {
```

```
        super(message);
    }
}
```

Rys. 3.13. Kod klasy NotFoundException. Źródło: Opracowanie własne.

NotFoundException jest wyjątkiem niestandardowym rzucanym w przypadku, gdy wyszukiwany zasób (np. składnik) nie istnieje w bazie danych, natomiast ApiException dotyczy przypadków, gdy przykładowo nie spodziewamy się otrzymanych zasobów, lub też w przypadku, gdy dodawany do bazy danych zasób już istnieje, tak jak na przykładzie pokazanym na Rys. 3.14.

```
private LocalDateTime getFromRange(StatisticsRange statisticsRange) {
    LocalDateTime from;
    switch (statisticsRange) {
        case DAILY -> from = LocalDate.now().atStartOfDay();
        case WEEKLY -> from =
LocalDate.now().atStartOfDay().minusDays(7);
        case MONTHLY -> from =
LocalDate.now().atStartOfDay().minusMonths(1);
        case YEARLY -> from =
LocalDate.now().atStartOfDay().minusYears(1);
        case OVERALL -> from =
transactionRepository.findFirstTransactionDateTime()
                .orElseThrow(() -> new NotFoundException("Transactions
not found"));
        default -> throw new ApiException("Unexpected value: " +
statisticsRange);
    }
    return from;
}
```

Rys. 3.14. Przykład zastosowania niestandardowych wyjątków. Źródło: Opracowanie własne.

W celu centralizacji obsługi błędów w aplikacji zdefiniowano klasę, której kod przedstawiono na Rys. 3.15. GlobalExceptionHandler oznaczoną adnotacją @ControllerAdvice. Ta, zdefiniowana w Spring, adnotacja dostarcza mechanizmu globalnej obsługi błędów, dzięki czemu wszystkie wyjątki rzucone np. w kontrolerach mogą być przechwycone i przekształcone na spójne odpowiedzi HTTP.

```
@ControllerAdvice
@RequiredArgsConstructor
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ExceptionDto handleNotFoundException(
            NotFoundException exception) {
        return new ExceptionDto(exception.getMessage());
    }

    @ExceptionHandler(ApiException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ExceptionDto handleApiException(
```

```
                ApiException exception) {
        return new ExceptionDto(exception.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ExceptionDto handleMethodArgumentNotValidException(
                MethodArgumentNotValidException exception) {
        return new ExceptionDto(
        exception.getBindingResult()
        .getFieldError().getField(),
        exception.getBindingResult()
        .getAllErrors().get(0).getDefaultMessage());
    }

    @ExceptionHandler(MethodArgumentTypeMismatchException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public ExceptionDto handleMethodArgumentTypeMismatchException(
                MethodArgumentTypeMismatchException exception) {
        return new ExceptionDto(exception.getMessage());
    }

}
```

Rys. 3.15. Kod klasy GlobalExceptionHandler. Źródło: Opracowanie własne.

Zdefiniowana klasa obsługuje zarówno wyjątki niestandardowe, takie jak opisane wcześniej NotFoundException czy ApiException, jak również standardowe błędy walidacji dostarczane przez Spring, np. MethodArgumentNotValidException oraz MethodArgumentTypeMismatchException. Adnotacja @ResponseStatus pozwala na ustawienie odpowiedniego kodu HTTP w odpowiedzi (np. 404 dla zasobu nieznalezionego (HttpStatus.NOT_FOUND) lub 400 dla błędów walidacji (HttpStatus.BAD_REQUEST).

```
@Getter
@RequiredArgsConstructor
public class ExceptionDto {
    private String field;
    private final String message;

    @JsonFormat(pattern = "dd-MM-yyyy HH:mm:ss")
    private final LocalDateTime timestamp = LocalDateTime.now();

    public ExceptionDto(String field, String message) {
        this.field = field;
        this.message = message;
    }
}
```

Rys. 3.16. Kod klasy ExceptionDto. Źródło: Opracowanie własne.

W aplikacji GastroM, każdy przechwycony wyjątek jest mapowany na ujednolicony obiekt typu ExceptionDto, dla którego kod klasy pokazano na Rys. 3.16. Umożliwia on przesyłanie danych między warstwą backend a klientem w spójnej i

przewidywalnej formie. W kontekście globalnej obsługi wyjątków, ExceptionDto zawiera podstawowe informacje, które pozwalają klientowi REST API na ujednolicenie informacji o błędach, na przykład w przypadku błędów walidacji, wskazać pole, które spowodowało problem. W klasie wyróżnić można pola:

1. field, opcjonalne pole wskazujące nazwę elementu (np. pola DTO lub parametru żądania), którego dotyczy błąd; jest wykorzystywane głównie w przypadku wyjątków walidacyjnych, gdy klient wysyła niepoprawne dane;
2. message, zawiera komunikat opisujący przyczynę błędu; jest to pole wymagane, przekazywane w konstruktorze i zwracane w każdej odpowiedzi DTO;
3. timestamp, czyli automatycznie ustawiany czas utworzenia obiektu DTO, oznaczający moment wystąpienia błędu; adnotacja *@JsonFormat(pattern = "dd-MM-yyyy HH:mm:ss"*) zapewnia, że data i czas zostaną zapisane w podanym formacie w odpowiedzi JSON, np. "20-10-2025 14:35:10".

3.8 Bezpieczeństwo

Bezpieczeństwo aplikacji GastroM realizowane jest przy użyciu mechanizmów dostarczanych przez Spring Security, które pozwalają na uwierzytelnianie użytkowników, autoryzację dostępu do zasobów oraz zabezpieczenie komunikacji między klientem a serwerem.

3.8.1 Konfiguracja bezpieczeństwa

Konfiguracja bezpieczeństwa została zapisana w klasie SecurityConfig, oznaczonej adnotacjami @Configuration i @EnableWebSecurity, której kod widać na Rys. 3.17. Adnotacja @Configuration wskazuje, że ta klasa zawiera definicje beanów, z których Spring buduje kontekst aplikacji, czyli konfigurację aplikacji. Spring podczas uruchamiania aplikacji analizuje tę klasę i tworzy instancje komponentów zarządzanych przez kontener IoC. Adnotacja @EnableWebSecurity aktywuje mechanizmy Spring Security dla aplikacji webowej. Dzięki temu klasa umożliwia konfigurowanie zabezpieczeń HTTP, filtrowanie żądań, uwierzytelnianie i autoryzację użytkowników.

```
@Configuration
@RequiredArgsConstructor
@EnableWebSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final AuthenticationProvider authenticationProvider;
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
throws Exception {
        http.cors(cors -> cors
        .configurationSource(corsConfigurationSource()))
        .authorizeHttpRequests((authorizeRequests) ->
        authorizeRequests
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/api/v1/users").hasRole("ADMIN")
                .requestMatchers("/v3/api-docs/**").permitAll()
                .requestMatchers("/swagger-ui/**").permitAll()
                .anyRequest().authenticated())
        .authenticationProvider(authenticationProvider)
        .addFilterBefore(jwtAuthenticationFilter,
        UsernamePasswordAuthenticationFilter.class)
        .sessionManagement(session ->
        session
                .sessionCreationPolicy(
                SessionCreationPolicy.STATELESS))
        .csrf(AbstractHttpConfigurer::disable);
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();

        configuration.addAllowedOrigin("*");

        configuration.setAllowedMethods(
        Arrays.asList("GET", "POST", "PUT", "PATCH", "DELETE"));

        configuration.addAllowedHeader("*");

        UrlBasedCorsConfigurationSource source =
        new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);

        return source;
    }
}
```

Rys. 3.17. Kod klasy SecurityConfig. Źródło: Opracowanie własne.

W ramach konfiguracji dla tworzonej aplikacji określono, że wszystkie żądania kierowane na ścieżki związane z autoryzacją *(/api/v1/auth/**)* oraz dokumentacją API *(/v3/api-docs/**, /swagger-ui/**)* są publicznie dostępne. Wszystkie pozostałe żądania muszą być uwierzytelnione, jednak dostęp do zasobów wymagających podwyższonych uprawnień, takich jak zarządzanie użytkownikami (/api/v1/users), został ograniczony wyłącznie do użytkowników z rolą ADMIN. Proces uwierzytelniania opiera się na tokenach JWT, które są weryfikowane przy każdym żądaniu przez filtr jwtAuthenticationFilter. Filtr ten jest wstrzykiwany do łańcucha Spring Security przed standardowym filtrem UsernamePasswordAuthenticationFilter, co pozwala na

sprawdzenie poprawności tokenu zanim żądanie dotrze do kontrolera. Dzięki adnotacji *@RequiredArgsConstructor* zależności klasy, takie jak filtr JWT i dostawca uwierzytelnienia (AuthenticationProvider), są automatycznie wstrzykiwane przez Spring.

Ponadto w konfiguracji zastosowano politykę stateless poprzez ustawienie *SessionCreationPolicy.STATELESS*, co oznacza, że serwer nie przechowuje sesji użytkownika. W takim przypadku, każde żądanie musi zawierać ważny token JWT, który pozwala na weryfikację tożsamości użytkownika. W celu uproszczenia komunikacji z aplikacjami frontendowymi i zewnętrznymi systemami włączono również obsługę CORS. Konfiguracja CORS w klasie SecurityConfig umożliwia wykonywanie żądań z dowolnej domeny oraz ze wszystkimi standardowymi metodami HTTP, co w tym przypadku umożliwia komunikację z warstwą frontend.

3.8.2 Autoryzacja

W aplikacji GastroM mechanizm autoryzacji opiera się na przypisaniu użytkownikom określonych ról, które determinują dostęp do poszczególnych zasobów REST API. Role zostały zdefiniowane za pomocą typu wyliczeniowego Role, obejmującego wartości ADMIN i MANAGER. Proces uwierzytelniania i autoryzacji odbywa się poprzez dedykowany endpoint */api/v1/auth*, obsługiwany przez klasę AuthenticationController. Klasa ta korzysta z serwisu AuthenticationService, który odpowiada za weryfikację danych logowania oraz generowanie tokenu JWT dla użytkownika. Kod klasy serwisu widać na Rys. 3.18.

```
@Service
@RequiredArgsConstructor
public class AuthenticationService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final JwtService jwtService;
    private final AuthenticationManager authenticationManager;

    public AuthenticationResponse authenticate(AuthenticationRequest
request) {
        authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(
                        request.getEmail(),
                        request.getPassword())
        );
        User user = userRepository.findByUsername(request.getEmail())
                .orElseThrow(() -> new NotFoundException("User not
found with email: " + request.getEmail()));
        var jwtToken = jwtService.generateToken(
                user,
                user.getId(),
                user.getFirstName(),
```

```
                user.getLastName(),
                user.getRole().toString()
        );
        return AuthenticationResponse.builder()
                .token(jwtToken)
                .build();
    }

    public AuthenticationResponse changePassword(ChangePasswordRequest
changePasswordRequest) {
        var user =
userRepository.findByUsername(changePasswordRequest.getEmail())
                .orElseThrow(() -> new NotFoundException("User not
found with email: " + changePasswordRequest.getEmail()));

if(passwordEncoder.matches(changePasswordRequest.getOldPassword(),
user.getPassword())) {

user.setPassword(passwordEncoder.encode(changePasswordRequest.getNewPa
ssword()));
            userRepository.save(user);
            var jwtToken = jwtService.generateToken(
                    user,
                    user.getId(),
                    user.getFirstName(),
                    user.getLastName(),
                    user.getRole().toString());
            return AuthenticationResponse.builder()
                    .token(jwtToken)
                    .build();
        } else {
            throw new ApiException("Old password is incorrect");
        }
    }
}
```

Rys. 3.18. Kod klasy AuthenticationService. Źródło: Opracowanie własne.

Metoda authenticate przyjmuje obiekt AuthenticationRequest, zawierający login (adres email) oraz hasło użytkownika. Po poprawnej weryfikacji danych metoda zwraca AuthenticationResponse, w którym znajduje się wygenerowany token JWT. Token ten jest następnie wykorzystywany w nagłówku każdego żądania HTTP, umożliwiając serwerowi identyfikację użytkownika i sprawdzenie jego uprawnień.

Dodatkowo, aplikacja umożliwia zmianę hasła przez endpoint *api/v1/auth/change-password*, który przyjmuje obiekt ChangePasswordRequest. W ten sposób użytkownik może zaktualizować swoje dane uwierzytelniające bez konieczności ingerencji administratora.

Mechanizm autoryzacji jest powiązany z konfiguracją Spring Security, w której reguły dostępu do poszczególnych endpointów są określone na podstawie ról użytkowników. Dzięki temu np. dostęp do zasobów związanych z zarządzaniem użytkownikami jest ograniczony do roli ADMIN, podczas gdy inne zasoby mogą być

dostępne dla użytkowników posiadających rolę MANAGER – w przypadku GastroM, wszystkie pozostałe.

3.8.3 Uwierzytelnianie z wykorzystaniem JWT

W aplikacji GastroM uwierzytelnianie użytkowników realizowane jest przy użyciu tokenów JWT z biblioteki *io.jsonwebtoken:jjwt*, czyli popularnej biblioteki Java do tworzenia, parowania i weryfikowania JSON Web Tokens – standardu wymiany danych jako bezpiecznych, kompaktowych obiektów JSON, używanych głównie do uwierzytelniania i autoryzacji w aplikacjach webowych i REST API. Umożliwiają one bezstanowe uwierzytelnianie i weryfikację tożsamości użytkownika przy każdym żądaniu do serwera. Tokeny JWT zawierają zakodowane informacje o użytkowniku, takie jak identyfikator, imię, nazwisko oraz przypisaną rolę, co pozwala backendowi weryfikować uprawnienia bez konieczności przechowywania sesji. Generowanie i weryfikacja tokenów jest realizowana przez serwis JwtService w jego standardowej implementacji pokazanej na Rys. 3.19.

Serwis ten umożliwia:

1. Tworzenie tokenu na podstawie danych użytkownika (generateToken).
2. Odczytanie informacji zawartych w tokenie (np. nazwa użytkownika (extractUsername), identyfikator (extractUserId), rola (extractRole).
3. Sprawdzenie ważności tokenu (isTokenExpired).
4. Sprawdzanie poprawności tokenu (isTokenValid).

Token jest generowany w odpowiedzi na poprawne uwierzytelnienie użytkownika i przekazywany w obiekcie AuthenticationResponse. Klient umieszcza token w nagłówku Authorization przy każdym żądaniu HTTP, co pozwala backendowi na identyfikację użytkownika i określenie jego uprawnień.

Weryfikacja tokenu w każdej żądanej ścieżce realizowana jest przez klasę JwtAuthenticationFilter, dziedziczącą po OncePerRequestFilter. Filtr odczytuje nagłówek Authorization, wyodrębnia token JWT i sprawdza jego poprawność przy pomocy serwisu JwtService. Jeśli token jest prawidłowy, filtr tworzy obiekt UsernamePasswordAuthenticationToken, który przechowuje informacje o użytkowniku i jego rolach, a następnie ustawia go w kontekście bezpieczeństwa Spring Security (SecurityContextHolder). Dzięki temu kolejne komponenty aplikacji mogą korzystać z informacji o użytkowniku oraz jego uprawnieniach. Klucz prywatny w JWT służy do podpisywania tokenu, aby zapewnić jego integralność i autentyczność. Tylko właściciel

klucza prywatnego może wygenerować poprawny podpis, a serwer (lub każdy posiadający odpowiadający klucz publiczny przy algorytmach asymetrycznych) może weryfikować, że token nie został zmieniony.

```
@Service
public class JwtService {

    private static final String SECRET_KEY =
"eb3967ef280cda5c177d52de364e01db17c7f65e4d990c57da14c83cfd7a5b69";

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public Long extractUserId(String token) {
        return extractClaim(token, claims -> claims.get("id",
Long.class));

    }

    public String extractFirstName(String token) {
        return extractClaim(token, claims -> claims.get("firstName",
String.class));
    }

    public String extractLastName(String token) {
        return extractClaim(token, claims -> claims.get("lastName",
String.class));
    }

    public String extractRole(String jwt) {
        return extractClaim(jwt, claims -> claims.get("role",
String.class));
    }

    public <T> T extractClaim(String token, Function<Claims, T>
claimsResolver) {
        final Claims claims = extractAllClaims(token);
        return claimsResolver.apply(claims);
    }

    public String generateToken(UserDetails userDetails, Long userId,
String firstName, String lastName, String role) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("id", userId);
        claims.put("firstName", firstName);
        claims.put("lastName", lastName);
        claims.put("userRole", role);
        return generateToken(claims, userDetails);
    }


    public String generateToken(
            Map<String, Object> extraClaims,
            UserDetails userDetails
    ) {
        return Jwts
                .builder()
                .setClaims(extraClaims)
```

```
                .setSubject(userDetails.getUsername())
                .setIssuedAt(new Date(System.currentTimeMillis()))
                .setExpiration(new Date(System.currentTimeMillis() +
1000 * 60 * 24))
                .signWith(getSignInKey(), SignatureAlgorithm.HS256)
                .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails)
{
        final String username = extractUsername(token);
        return (username.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
    }

    private boolean isTokenExpired(String token) {
        return extractExpiration(token).before(new Date());
    }

    private Date extractExpiration(String token) {
        return extractClaim(token, Claims::getExpiration);
    }

    private Claims extractAllClaims(String token) {
        return Jwts
                .parserBuilder()
                .setSigningKey(getSignInKey())
                .build()
                .parseClaimsJws(token)
                .getBody();
    }

    private Key getSignInKey() {
        byte[] keyBytes = Decoders.BASE64.decode(SECRET_KEY);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

Rys. 3.19. Kod klasy JwtService. Źródło: Opracowanie własne.

3.9 Testowanie

Testowanie aplikacji GastroM zostało zrealizowane z wykorzystaniem testów jednostkowych oraz testów integracyjnych, których celem była weryfikacja poprawności działania poszczególnych komponentów systemu oraz ich współpracy w ramach całej aplikacji. Przyjęte podejście umożliwiło zarówno izolowane sprawdzenie logiki biznesowej za pomocą testów jednostkowych, jak i ocenę poprawności integracji warstw aplikacji z mechanizmami persystencji danych.

3.9.1 Testy jednostkowe

Do implementacji testów zastosowano bibliotekę JUnit 5, pełniącą rolę podstawowego frameworka testowego oraz Mockito, które umożliwia tworzenie atrap zależności,

symulowanie zachowania komponentów zewnętrznych oraz weryfikację interakcji pomiędzy obiektami.

Do tej pory zaimplementowanych zostało łącznie 31 testów jednostkowych w klasach FixedCostServiceTest, EmployeeServiceTest, IngredientServiceTest, ProductComponentServiceTest, ProductServiceTest, UserServiceTest; Kolejne testy implementowane będą w trakcie dalszego rozwoju aplikacji.

Klasa IngredientServiceTest przedstawiona na Rys 3.20. realizuje testy jednostkowe warstwy serwisowej odpowiedzialnej za obsługę składników.

```
@ExtendWith(MockitoExtension.class)
class IngredientServiceTest {

    @InjectMocks
    private IngredientService ingredientService;
    @Mock
    private IngredientRepository ingredientRepository;
    @Mock
    private FixedCostRepository fixedCostRepository;
    @Captor
    private ArgumentCaptor<Ingredient> ingredientCaptor;
    @Captor
    private ArgumentCaptor<FixedCost> fixedCostCaptor;
```

Rys. 3.20. Kod deklaracji i wstrzykiwanych zależności klasy IngredientServiceTest. Źródło: Opracowanie własne.

Adnotacja @ExtendWith(MockitoExtension.class) integruje funkcje z biblioteki Mockito z cyklem życia testów zdefiniowanym w JUnit, umożliwiając m.in. automatyczną inicjalizację atrap i korzystanie ze zdefiniowanych adnotacji. Dzięki adnotacji @InjectMocks tworzona jest instancja klasy IngredientService, do której wstrzykiwane są wszystkie oznaczone mocki. Repozytoria IngredientRepository oraz FixedCostRepository zostały oznaczone adnotacją @Mock, co powoduje zastąpienie ich atrapami i uniemożliwia wykonywanie rzeczywistych operacji na bazie danych.

Adnotacje @Captor definiują obiekty typu ArgumentCaptor, które umożliwiają przechwycenie argumentów przekazywanych do metod repozytoriów. Pozwala to na weryfikację stanu obiektów zapisywanych w trakcie wykonywania logiki biznesowej.

```
@Test
void create_SavesAndReturnsDto() {
    // given
    CreateIngredientCommand cmd = CreateIngredientCommand.builder()
            .name("Sugar")
            .unit(Unit.G)
            .stockQuantity(new BigDecimal("500"))
            .alertQuantity(new BigDecimal("50"))
            .unitCost(new BigDecimal("0.02"))
            .build();
```

```
when(ingredientRepository.save(any(Ingredient.class))).thenAnswer(invo
cation -> {
        Ingredient i = invocation.getArgument(0);
        i.setId(7L);
        return i;
    });

    // when
    IngredientDto dto = ingredientService.create(cmd);

    // then
    verify(ingredientRepository).save(ingredientCaptor.capture());
    Ingredient saved = ingredientCaptor.getValue();
    assertEquals("Sugar", saved.getName());
    assertEquals(Unit.G, saved.getUnit());
    assertEquals(new BigDecimal("500"), saved.getStockQuantity());
    assertEquals(new BigDecimal("50"), saved.getAlertQuantity());
    assertEquals(new BigDecimal("0.02"), saved.getUnitCost());

    assertEquals(7L, dto.getId());
    assertEquals("Sugar", dto.getName());
}
```

Rys. 3.21. Kod testu create_SavesAndReturnsDto() klasy IngredientServiceTest. Źródło: Opracowanie własne.

Test create_SavesAndReturnsDto ukazany na Rys. 3.21. weryfikuje poprawność tworzenia nowego składnika. W części przygotowawczej tworzony jest obiekt CreateIngredientCommand, a metoda save repozytorium zostaje zamockowana, co oznacza stworzenie obiektu – atrapy (mocka), który symuluje prawdziwy obiekt w teście jednostkowym, pozwalając na kontrolowanie jego zachowania (np. zwracanych wartości, zgłaszanych wyjątków) i izolację testowanego kodu, w taki sposób, aby symulować nadanie identyfikatora encji po zapisie.

Po wywołaniu metody create do tworzenia obiektu typu Ingredient sprawdzane jest:

- czy metoda save repozytorium została wywołana,
- czy zapisany obiekt Ingredient posiada poprawnie ustawione pola,
- czy zwrócony obiekt IngredientDto zawiera identyfikator oraz nazwę zgodne z zapisanym rekordem.

Instrukcje assertEquals służą do porównania oczekiwanych wartości z rzeczywistym stanem obiektów, co pozwala potwierdzić poprawność mapowania danych oraz logiki tworzenia encji.

```
@Test
void getAll_WithLowStockFlag_DelegatesToRepositoryAndMaps() {
    // given
    Ingredient i1 =
Ingredient.builder().id(1L).name("Milk").unit(Unit.ML)
            .stockQuantity(new BigDecimal("1"))
```

```
            .alertQuantity(new BigDecimal("2"))
            .unitCost(new BigDecimal("3.00")).build();

when(ingredientRepository.findAllByLowStock(true)).thenReturn(List.of(
i1));

    // when
    List<IngredientDto> result = ingredientService.getAll(true);

    // then
    assertEquals(1, result.size());
    assertEquals(1L, result.get(0).getId());
    assertEquals("Milk", result.get(0).getName());
    verify(ingredientRepository).findAllByLowStock(true);
}
```

Rys. 3.22. Kod testu getAll_WithLowStockFlag_DelegatesToRepositoryAndMaps() klasy IngredientServiceTest. Źródło: Opracowanie własne.

Test getAll_WithLowStockFlag_DelegatesToRepositoryAndMaps ukazany na Rys. 3.22. sprawdza działanie metody pobierającej listę składników z uwzględnieniem flagi niskiego stanu magazynowego. Repozytorium zostaje skonfigurowane tak, aby zwracało listę zawierającą jeden składnik spełniający warunek niskiego stanu.

Test weryfikuje:

- poprawność liczby zwróconych elementów,
- poprawność mapowania danych na obiekt DTO,
- wywołanie metody findAllByLowStock(true) w repozytorium.

```
@Test
void getById_WhenFound_ReturnsDto() {
    // given
    Ingredient i =
Ingredient.builder().id(2L).name("Coffee").unit(Unit.G)
            .stockQuantity(new BigDecimal("1000"))
            .alertQuantity(new BigDecimal("100"))
            .unitCost(new BigDecimal("0.05")).build();

when(ingredientRepository.findById(2L)).thenReturn(Optional.of(i));

    // when
    IngredientDto dto = ingredientService.getById(2L);

    // then
    assertEquals(2L, dto.getId());
    assertEquals("Coffee", dto.getName());
    verify(ingredientRepository).findById(2L);
}
```

Rys. 3.23. Kod testu getById_WhenFound_ReturnsDto() klasy IngredientServiceTest. Źródło: Opracowanie własne.

Przedstawiony na Rys. 3.23. test getById_WhenFound_ReturnsDto dotyczy pobierania pojedynczego składnika na podstawie identyfikatora. Repozytorium zwraca obiekt Ingredient, który następnie mapowany jest na IngredientDto.

Sprawdzane jest:

- czy zwrócony obiekt DTO zawiera poprawny identyfikator i nazwę,
- czy metoda findById repozytorium została wywołana z właściwym parametrem.

```
@Test
void getById_WhenNotFound_ThrowsNotFoundException() {

when(ingredientRepository.findById(9L)).thenReturn(Optional.empty());
    assertThrows(NotFoundException.class, () ->
ingredientService.getById(9L));
    verify(ingredientRepository).findById(9L);
}
```

Rys. 3.24. Kod testu getById_WhenNotFound_ThrowsNotFoundException() klasy IngredientServiceTest. Źródło: Opracowanie własne.

Test getById_WhenNotFound_ThrowsNotFoundExceptionTest ukazany na Rys. 3.24. weryfikuje poprawność obsługi sytuacji, w której żądany składnik nie istnieje w bazie danych. Repozytorium IngredientRepository zostaje skonfigurowane tak, aby metoda findById zwracała pusty obiekt Optional, co symuluje brak rekordu o podanym identyfikatorze. Instrukcja assertThrows sprawdza, czy wywołanie metody getById serwisu skutkuje zgłoszeniem wyjątku typu NotFoundException. Pozwala to potwierdzić, że logika serwisowa poprawnie reaguje na brak danych i nie zwraca wartości niepoprawnych lub pustych. Dodatkowo zastosowanie verify umożliwia potwierdzenie, że metoda findById repozytorium została wywołana dokładnie z przekazanym identyfikatorem, co świadczy o poprawnym delegowaniu operacji odczytu do warstwy persystencji.

```
@Test
void deleteById_DelegatesToRepository() {
    ingredientService.deleteById(4L);
    verify(ingredientRepository).deleteById(4L);
}
```

Rys. 3.25. Kod testu deleteById_DelegatesToRepository() klasy IngredientServiceTest. Źródło: Opracowanie własne.

Test deleteById_DelegatesToRepository ukazay na Rys. 4.25. sprawdza poprawność delegowania operacji usuwania składnika do repozytorium. Metoda deleteById serwisu jest wywoływana z przykładowym identyfikatorem, bez wcześniejszego przygotowywania zachowania repozytorium, ponieważ operacja nie zwraca wartości. Instrukcja verify służy do potwierdzenia, że metoda deleteById

repozytorium została wywołana z poprawnym parametrem. Test nie weryfikuje dodatkowej logiki biznesowej, ponieważ jego celem jest sprawdzenie czy warstwa serwisowa poprawnie przekazuje żądanie usunięcia do warstwy persystencji.

```
@Test
    void restock_UpdatesQuantity_AndCreatesFixedCost() {
        Ingredient i = Ingredient.builder()
                .id(3L)
                .name("Milk")
                .unit(Unit.ML)
                .stockQuantity(new BigDecimal("5"))
                .unitCost(new BigDecimal("2.50"))
                .alertQuantity(new BigDecimal("2"))
                .build();

when(ingredientRepository.findWithLockingById(3L)).thenReturn(Optional
.of(i));

when(ingredientRepository.save(any(Ingredient.class))).thenAnswer(inv
-> inv.getArgument(0));

when(fixedCostRepository.save(any(FixedCost.class))).thenAnswer(inv ->
inv.getArgument(0));

        RestockCommand cmd = RestockCommand.builder().quantity(new
BigDecimal("3"))
                .build();

        IngredientDto dto = ingredientService.restock(3L, cmd);

        verify(ingredientRepository).save(ingredientCaptor.capture());
        Ingredient updated = ingredientCaptor.getValue();
        assertEquals(0, new
BigDecimal("8.00").compareTo(updated.getStockQuantity()));

        verify(fixedCostRepository).save(fixedCostCaptor.capture());
        FixedCost fc = fixedCostCaptor.getValue();
        assertEquals(CostType.RESTOCK, fc.getCostType());
        assertTrue(fc.getDescription().contains("Milk"));
        assertEquals(new BigDecimal("7.50"), fc.getCost());
        assertNotNull(fc.getCreatedAt());

        assertEquals(3L, dto.getId());
        assertEquals(0, new
BigDecimal("8.00").compareTo(dto.getStockQuantity()));
    }
}
```

Rys. 3.26. Kod testu restock_UpdatesQuantity_AndCreatesFixedCost() klasy IngredientServiceTest. Źródło: Opracowanie własne.

Test restock_UpdatesQuantity_AndCreatesFixedCost, którego kod przedstawiono na Rys. 3.26. weryfikuje złożoną operację uzupełniania stanu magazynowego. Repozytorium składników zwraca obiekt pobrany z zastosowaniem blokady pesymistycznej (findWithLockingById), co odpowiada rzeczywistemu scenariuszowi aktualizacji stanu magazynowego.

Test sprawdza:

- czy ilość składnika została poprawnie zwiększona o podaną wartość,
- czy zapisany obiekt Ingredient zawiera zaktualizowany stan magazynowy,
- czy utworzony został rekord kosztu stałego (FixedCost) o typie RESTOCK,
- czy koszt został poprawnie obliczony na podstawie ilości i ceny jednostkowej,
- czy rekord kosztu zawiera opis oraz datę utworzenia,
- czy zwrócony obiekt DTO odzwierciedla zaktualizowany stan składnika.

Zastosowanie assertEquals, assertTrue oraz assertNotNull umożliwia precyzyjną weryfikację zarówno wartości liczbowych, jak i poprawności uzupełnienia pól opisowych i czasowych.

3.9.2 Testy integracyjne

Testy integracyjne w aplikacji GastroM zostały zaimplementowane w celu weryfikacji poprawności współdziałania poszczególnych warstw aplikacji w ramach pełnego kontekstu Spring. W przeciwieństwie do testów jednostkowych, testy integracyjne uruchamiane są z wykorzystaniem rzeczywistej konfiguracji aplikacji, co umożliwia sprawdzenie poprawności mapowania żądań HTTP, walidacji danych wejściowych, obsługi wyjątków oraz integracji warstwy kontrolerów z warstwą serwisową i repozytoriami.

```
@SpringBootTest
@AutoConfigureMockMvc
class IngredientControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Autowired
    private FixedCostRepository fixedCostRepository;

    @Test
    void create_lowStockFilter_and_restock() throws Exception {
        CreateIngredientCommand create =
CreateIngredientCommand.builder()
                .name("Milk")
                .unit(Unit.ML)
                .stockQuantity(new BigDecimal("1"))
                .alertQuantity(new BigDecimal("2"))
                .unitCost(new BigDecimal("0.50"))
                .build();

        String createdJson =
mockMvc.perform(post("/api/v1/ingredients")
```

```
                        .contentType(MediaType.APPLICATION_JSON)

.content(objectMapper.writeValueAsString(create)))
                .andExpect(status().isCreated())
                .andExpect(jsonPath("$.id").exists())
                .andExpect(jsonPath("$.name").value("Milk"))
                .andReturn().getResponse().getContentAsString();

        long id =
objectMapper.readTree(createdJson).get("id").asLong();

        mockMvc.perform(get("/api/v1/ingredients").param("lowStock",
"true"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$[0].id").value(id))
                .andExpect(jsonPath("$[0].name").value("Milk"));

        RestockCommand restock = RestockCommand.builder()
                .quantity(new BigDecimal("3"))
                .build();

        mockMvc.perform(patch("/api/v1/ingredients/restock/" + id)
                        .contentType(MediaType.APPLICATION_JSON)

.content(objectMapper.writeValueAsString(restock)))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.id").value(id))
                .andExpect(jsonPath("$.stockQuantity").value(4));

        assertThat(fixedCostRepository.findAll
                ())
                .anySatisfy(fc -> {

assertThat(fc.getCostType()).isEqualTo(CostType.RESTOCK);
                    assertThat(fc.getDescription()).contains("Milk");
                    assertThat(fc.getCost()).isEqualByComparingTo(new
BigDecimal("1.50"));
                });
    }
}
```

Rys. 3.27. Kod klasy IngredientControllerTest. Źródło: Opracowanie własne.

Klasa testowa testów integracyjnych, której kod przedstawiony został na Rys. 3.27. została opatrzona zestawem adnotacji konfiguracyjnych, które definiują sposób uruchamiania kontekstu testowego oraz mechanizm wykonywania żądań HTTP.

Adnotacja @SpringBootTest uruchamia pełny kontekst aplikacji Spring Boot na potrzeby testu. Oznacza to, że podczas wykonywania testów inicjalizowane są wszystkie beany aplikacji, w tym kontrolery, serwisy, repozytoria oraz konfiguracja JPA i bazy danych (najczęściej w wariancie testowym). Zastosowanie tej adnotacji umożliwia testowanie rzeczywistej integracji pomiędzy warstwami aplikacji, a nie jedynie pojedynczych komponentów w izolacji. Dzięki temu test odwzorowuje realne zachowanie aplikacji w środowisku uruchomieniowym.

Adnotacja @AutoConfigureMockMvc powoduje automatyczną konfigurację komponentu MockMvc, który umożliwia symulowanie żądań HTTP bez konieczności uruchamiania serwera aplikacyjnego.

Adnotacja @Autowired służy do wstrzykiwania zależności zarządzanych przez kontener Springa do pól klasy testowej. W testach integracyjnych wykorzystywana jest do uzyskania dostępu do rzeczywistych beanów aplikacji, takich jak:

- MockMvc – do wykonywania i weryfikowania żądań HTTP bez uruchamiania serwera HTTP,
- ObjectMapper – do serializacji i deserializacji obiektów Java do formatu JSON,
- FixedCostRepository – do bezpośredniej weryfikacji stanu bazy danych po wykonaniu operacji.

Zastosowanie @Autowired w połączeniu z @SpringBootTest zapewnia, że w testach wykorzystywane są te same komponenty, które działają w środowisku produkcyjnym aplikacji.

Test create_lowStockFilter_and_restock_flow weryfikuje pełny scenariusz biznesowy obejmujący utworzenie nowego składnika, filtrowanie składników o niskim stanie magazynowym oraz uzupełnienie stanu magazynowego wraz z zapisem kosztu stałego. Test obejmuje cały przepływ żądania przez warstwę kontrolera, serwisu oraz repozytoriów, co pozwala na ocenę poprawności integracji poszczególnych komponentów aplikacji.

W pierwszym etapie testu tworzony jest obiekt CreateIngredientCommand, zawierający dane nowego składnika, takie jak nazwa, jednostka miary, początkowy stan magazynowy, próg alertowy oraz koszt jednostkowy. Obiekt ten jest następnie serializowany do formatu JSON przy użyciu komponentu ObjectMapper i przekazywany jako treść żądania HTTP typu POST do endpointu /api/v1/ingredients.

Za pomocą obiektu MockMvc symulowane jest wykonanie żądania HTTP. Oczekiwany jest kod odpowiedzi 201 Created, co potwierdza poprawne utworzenie zasobu. Dodatkowo test weryfikuje obecność identyfikatora w odpowiedzi oraz poprawność nazwy utworzonego składnika, korzystając z wyrażeń JSONPath. Zwrócona odpowiedź jest następnie odczytywana jako tekst, a identyfikator nowo utworzonego składnika wyodrębniany w celu dalszego wykorzystania w kolejnych krokach testu.

W kolejnym etapie wykonywane jest żądanie GET do endpointu /api/v1/ingredients z parametrem zapytania lowStock=true. Celem tego kroku jest

sprawdzenie poprawności działania mechanizmu filtrowania składników o niskim stanie magazynowym. Test weryfikuje kod odpowiedzi 200 OK oraz sprawdza, czy w zwróconej liście znajduje się wcześniej utworzony składnik, co potwierdza poprawną integrację kontrolera z logiką filtrującą po stronie serwisu.

Następnie test realizuje operację uzupełnienia stanu magazynowego poprzez wykonanie żądania PATCH do endpointu /api/v1/ingredients/restock/{id}. Do żądania przekazywany jest obiekt RestockCommand, zawierający ilość składnika przeznaczoną do dodania. Test weryfikuje poprawność odpowiedzi, w szczególności zwracany identyfikator składnika oraz zaktualizowaną ilość w magazynie, która stanowi sumę stanu początkowego oraz ilości uzupełnienia.

W ostatnim etapie testu następuje bezpośrednia weryfikacja skutków ubocznych operacji uzupełnienia stanu magazynowego w warstwie persystencji danych. Przy użyciu repozytorium FixedCostRepository sprawdzane jest, czy w bazie danych został zapisany rekord kosztu stałego o typie RESTOCK. Test weryfikuje również poprawność opisu kosztu, który powinien zawierać nazwę składnika, oraz wartość kosztu obliczoną jako iloczyn ilości uzupełnienia i kosztu jednostkowego składnika.

W bieżącym stanie aplikacji zaimplementowane zostały 4 testy integracyjne w klasach FixedCostControllerTest, EmployeeControllerTest, IngredientControllerTest, ProductControllerTest. Opisany test integracyjny jest obecnie jedynym zaimplementowanym testem klasy IngredientControllerTest. Został on celowo zaprojektowany jako test przekrojowy, obejmujący kilka kluczowych operacji biznesowych w ramach jednego scenariusza. W kolejnych etapach rozwoju aplikacji planowane jest rozszerzenie zestawu testów integracyjnych o dodatkowe przypadki, w tym testy scenariuszy błędnych, walidacji danych wejściowych oraz obsługi wyjątków.

4. IMPLEMENTACJA WARSTWY FRONTEND (OSKAR KRUPA)

Warstwa frontend aplikacji GastroM stanowi interfejs użytkownika. Korzystając z niego pracownicy restauracji oraz administratorzy wykonują operacje na danych zarządzanych przez warstwę backend, której implementację opisano w rozdziale 3. Frontend realizuje m.in. logikę aplikacyjną: prezentację danych, obsługę formularzy, walidację po stronie klienta oraz nawigację między widokami.

4.1 Konfiguracja kodu aplikacji

Środowisko aplikacji oparto na narzędziu Vite 6. Główne zależności projektu dobrano pod kątem konkretnych zadań, co przedstawiono na Rys. 4.1.:

```
{
  "name": "gastrom",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "test:e2e": "npx playwright test",
    "test:e2e-ui": "npx playwright test --ui"
  },
  "dependencies": {
    "@dnd-kit/core": "^6.3.1",
    "@dnd-kit/sortable": "^10.0.0",
    "@dnd-kit/utilities": "^3.2.2",
    "@emotion/react": "^11.14.0",
    "@emotion/styled": "^11.14.0",
    "@fontsource/inter": "^5.1.0",
    "@fontsource/roboto": "^5.1.0",
    "@mui/icons-material": "^6.4.0",
    "@mui/lab": "^6.0.0-beta.24",
    "@mui/material": "^6.4.0",
    "@tanstack/react-query": "^5.64.1",
    "axios": "^1.7.9",
    "install": "^0.13.0",
    "notistack": "^3.0.2",
    "npm": "^11.0.0",
    "prettier": "^3.4.2",
    "react": "^18.3.1",
```

```
    "react-dom": "^18.3.1",
    "react-jwt": "^1.2.2",
    "react-router-dom": "^7.1.1",
    "recharts": "^2.15.1",
    "zod": "^3.24.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.17.0",
    "@playwright/test": "^1.50.1",
    "@types/node": "^25.0.2",
    "@types/react": "^18.3.18",
    "@types/react-dom": "^18.3.5",
    "@vitejs/plugin-react": "^4.3.4",
    "dotenv": "^17.2.3",
    "eslint": "^9.17.0",
    "eslint-plugin-react-hooks": "^5.0.0",
    "eslint-plugin-react-refresh": "^0.4.16",
    "globals": "^15.14.0",
    "typescript": "~5.6.2",
    "typescript-eslint": "^8.18.2",
    "vite": "^6.0.5"
  }
}
```

Rys. 4.1. Kod pliku package.json. Źródło: Opracowanie własne.

Do implementacji wykorzystano następujące główne zależności:

- **React 18** – biblioteka opisana w podrozdziale 1.2.2.
- **Vite 6** – nowoczesne narzędzie do budowania aplikacji z Hot Module Replacement.
- **Material-UI v6** (`@mui/material`, `@mui/icons-material`) – kompleksowa biblioteka komponentów UI zgodna z Material Design.
- **TanStack Query v5** (`@tanstack/react-query`) – biblioteka upraszczająca m. in. zarządzanie stanem serwera, buforowanie i synchronizację danych.
- **React Router v7** (`react-router-dom`) – biblioteka do tworzenia ścieżek po stronie klienta.
- **Axios** – klient HTTP do komunikacji z REST API.
- **Zod** – biblioteka do walidacji schematów i wnioskowania typów.
- **Notistack** – system powiadomień.

- **Recharts** – biblioteka do tworzenia wykresów.
- **@dnd-kit** – biblioteka do obsługi podnoszenia i opuszczania elementów HTML.

Kod konfiguracji Vite przedstawiono na Rys. 4.2. Jest ona minimalistyczna i wykorzystuje oficjalną wtyczkę @vitejs/plugin-react do obsługi React z JSX.

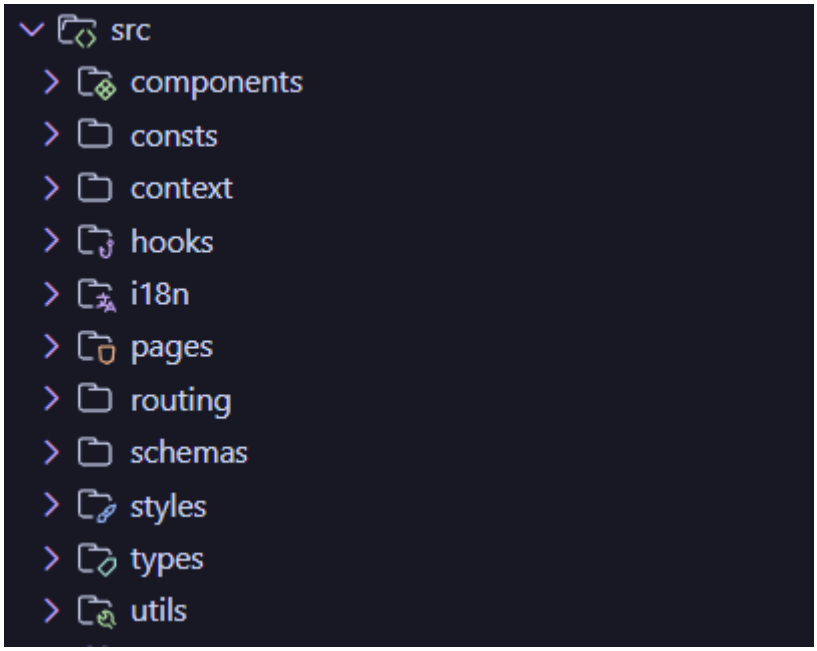
```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
});
```

Rys. 4.2. Kod konfiguracji Vite w pliku vite.config.ts. Źródło: Opracowanie własne.

4.2 Struktura aplikacji

Struktura aplikacji w warstwie frontend opiera się na grupowaniu według typu i przeznaczenia. Główny katalog src zawiera następujące podkatalogi:



Rys. 4.3. Struktura aplikacji. Źródło: Opracowanie własne.

Każdy z katalogów pełni określoną rolę w architekturze aplikacji:

- **components** – zawiera komponenty wielokrotnego użytku, takie jak menu boczne, kontenery główne czy kafelki dashboardu.

- **hooks** – zawiera własne funkcje pomocnicze enkapsulujące logikę biznesową i komunikację z API.
- **pages** – zawiera komponenty reprezentujące poszczególne strony aplikacji, mapowane na ścieżki URL.
- **schemas** – zawiera schematy biblioteki Zod definiujące strukturę danych i zapewniające walidację w czasie wykonania.

4.3 Punkt wejściowy aplikacji

Głównym plikiem wejściowym aplikacji jest main.tsx, który inicjalizuje i renderuje drzewo komponentów React. Kod tego pliku został przedstawiony na Rys. 4.4.

```
    createRoot(document.getElementById('root')!).render(
      <QueryClientProvider client={queryClient}>
        <AuthProvider>
          <AppTheme>
            <SnackbarProvider
              maxSnack={3}
              anchorOrigin={{ vertical: 'bottom', horizontal: 'right'
}}
            >
              <CssBaseline enableColorScheme />
              <Routes />
              <ColorModeSelect
                sx={{ position: 'fixed', top: '1rem', right: '1rem' }}
              />
            </SnackbarProvider>
          </AppTheme>
        </AuthProvider>
      </QueryClientProvider>
    );
```

Rys. 4.4. Kod pliku main.tsx. Źródło: Opracowanie własne.

Kod widoczny na Rys. 4.4. konfiguruje hierarchię dostawców kontekstu, które udostępniają wspólne funkcjonalności wszystkim komponentom aplikacji. Jako najważniejsze można wyróżnić:

1. **QueryClientProvider** – umożliwia komponentom korzystanie z buforowanych danych API bez konieczności przekazywania ich przez właściwości.
2. **AuthProvider** – przechowuje stan zalogowanego użytkownika, dostępny w dowolnym miejscu aplikacji do weryfikacji uprawnień.

3. **AppTheme** – dostarcza motyw Material-UI.
4. **SnackbarProvider** – zapewnia jednolity sposób wyświetlania powiadomień o wynikach operacji (sukces, błąd) w całej aplikacji.

4.4 Schematy danych

Schematy biblioteki Zod pełnią podwójną rolę: definiują strukturę danych wymienianą z API oraz walidują dane wejściowe formularzy. Każdy schemat odpowiada klasie DTO lub Command z warstwy backend, które są opisane w podrozdziale 4.3.2., co zapewnia spójność między warstwami.

Schemat dla Ingredient przedstawiono na Rys. 4.5.

```
import { z } from 'zod';

// Unit enum for ingredients
export const unitEnum = z.enum(['G', 'ML', 'PCS']);
export type Unit = z.infer<typeof unitEnum>;

// Ingredient DTO matching the API specification
export const ingredientDtoSchema = z.object({
  id: z.number(),
  name: z.string(),
  unit: unitEnum,
  stockQuantity: z.number(),
  alertQuantity: z.number(),
  unitCost: z.number(),
});

export type IngredientDto = z.infer<typeof ingredientDtoSchema>;

// Create Ingredient Command for POST requests
export const createIngredientCommandSchema = z.object({
  name: z.string(),
  unit: unitEnum,
  stockQuantity: z.number(),
  alertQuantity: z.number(),
  unitCost: z.number(),
});
```

Rys. 4.5. Kod schematu Ingredient. Źródło: Opracowanie własne.

Schemat ingredientDtoSchema odzwierciedla strukturę IngredientDto zwracanego przez endpoint GET /api/ingredients. Dzięki deklarowaniu typów przy

pomocy konstrukcji z.infer<typeof schema> typ TypeScript jest automatycznie wnioskowany ze schematu, eliminując duplikację definicji typów.

Schematy komend, np. CreateIngredientCommandSchema, zawierają reguły walidacji – minimalną długość nazwy, dozwolone wartości jednostek – które są sprawdzane przed wysłaniem żądania do API. Pozwala to wyświetlić użytkownikowi komunikat o błędzie bez wykonywania zbędnego żądania sieciowego.

4.5 Komunikacja z API

Komunikacja z warstwą backend realizowana jest za pomocą biblioteki Axios. Podstawowym, a jednocześnie centralnym elementem jest tutaj instancja Axios apiInstance, która przechowuje gotową konfigurację HTTP do komunikacji z serwerem. W kolejnych podrozdziałach omówione zostaną elementy, dzięki którym warstwa frontend komunikuje się z warstwą backend.

4.5.1 Instancja Axios

Konfiguracja instancji Axios przedstawiona jest na Rys. 4.6.

```
import axios from 'axios';
import { API_ROOT_URL } from '../consts/api.ts';

export const apiInstance = axios.create({
  baseURL: API_ROOT_URL,
});
```

Rys. 4.6. Konfiguracja instancji Axios. Źródło: Opracowanie własne.

W obiekcie zdefiniowano bazowy adres URL API po stronie backend, wykorzystując do tego celu stałą API_ROOT_URL zdefiniowaną w pliku konfiguracyjnym:

```
export const API_ROOT_URL = 'https://management-api-irsm.onrender.com/api/v1';
export const LOGIN_URL = '/auth/authenticate';
```

Rys. 4.7. Definicje stałych do komunikacji z serwerem w pliku api.ts. Źródło: Opracowanie własne.

Bazowy URL wskazuje na serwer API uruchomiony na platformie Render. Nagłówek autoryzacji zwrócony przez serwer jest automatycznie dodawany przez funkcję useUser po zalogowaniu użytkownika.

4.5.2 Funkcje dostępu do danych

Dostęp do danych serwera, które są potrzebne do wyświetlenia odpowiednich metryk oraz statystyk w aplikacji realizowany jest z wykorzystaniem biblioteki TanStack

Query. Implementację funkcji pomocniczych przedstawiono na przykładzie pliku use-inventory.ts, którego kod jest widoczny na Rys. 4.8.

```
// Query keys for cache management
export const ingredientKeys = {
  all: ['ingredients'] as const,
  lists: () => [...ingredientKeys.all, 'list'] as const,
  list: (filters: Record<string, string | number | boolean>) =>
    [...ingredientKeys.lists(), filters] as const,
  details: () => [...ingredientKeys.all, 'detail'] as const,
  detail: (id: number) => [...ingredientKeys.details(), id] as
const,
};

// Fetch all ingredients
export const useIngredients = (options = {}) => {
  return useQuery({
    queryKey: ingredientKeys.lists(),
    queryFn: async () => {
      const response = await
apiInstance.get<IngredientDto[]>('/ingredients');
      return response.data;
    },
    ...options,
  });
};
```

Rys. 4.8. Implementacja funkcji pomocniczej useIngredients w pliku use-inventory.ts. Źródło: Opracowanie własne.

Funkcja pomocnicza useIngredients pobiera listę składników z adresu GET /api/ingredients. Jest to funkcja typu hook, co pozwala jej m. in. korzystać z hooka useQuery zdefiniowanego w bibliotece TanStack. Hook useQuery pozwala w tym przypadku pobrać listę składników oraz automatycznie:

- buforuje odpowiedź i udostępnia ją wszystkim komponentom bez ponownego żądania,
- śledzi stan ładowania i błędów, eliminując ręczne zarządzanie flagami,
- odświeża dane po powrocie użytkownika do aplikacji.

Dzięki temu komponent wyświetlający tabelę składników nie musi wiedzieć, skąd pochodzą dane – otrzymuje je gotowe do wyświetlenia.

4.5.3 Mutacje z optymistycznymi aktualizacjami

Mutacje, czyli operacje modyfikujące dane na serwerze, takie jak tworzenie, edycja czy usuwanie, wykorzystują tzw. optymistyczne aktualizacje. Polegają one na natychmiastowym zaktualizowaniu interfejsu przed otrzymaniem potwierdzenia z serwera z założeniem, że operacja na serwerze się powiedzie:

```
    // Create a new ingredient
    export const useCreateIngredient = () => {
      const queryClient = useQueryClient();

      return useMutation({
        mutationFn: async (newItem: CreateIngredientCommand) => {
          const response = await apiInstance.post<IngredientDto>(
            '/ingredients',
            newItem
          );
          return response.data;
        },
        onMutate: async (newItem: CreateIngredientCommand) => {
          // Cancel any outgoing refetches
          await          queryClient.cancelQueries({          queryKey:
ingredientKeys.lists() });

          // Snapshot the previous value
          const                  previousIngredients                  =
queryClient.getQueryData<IngredientDto[]>(
            ingredientKeys.lists()
          );

          // Optimistically update to the new value
          if (previousIngredients) {
            const optimisticIngredient: IngredientDto = {
              id: Date.now(), // Temporary ID
              ...newItem,
            };

queryClient.setQueryData<IngredientDto[]>(ingredientKeys.lists(), [
              ...previousIngredients,
              optimisticIngredient,
            ]);
          }

          return { previousIngredients };
        },
```

```
        onError: (_err, _newItem, context) => {
          // Rollback on error
          if (context?.previousIngredients) {
            queryClient.setQueryData(
              ingredientKeys.lists(),
              context.previousIngredients
            );
          }
        },
        onSettled: () => {
          queryClient.invalidateQueries({                    queryKey:
ingredientKeys.lists() });
        },
      });
    };
```

Rys. 4.9. Kod mutacji z optymistycznymi aktualizacjami z pliku use-inventory.ts. Źródło: Opracowanie własne.

Widoczna na Rys. 4.9. mutacja useCreateIngredient implementuje optymistyczne aktualizacje:

1. **onMutate** – przed wysłaniem żądania, anuluje aktywne zapytania, zapisuje poprzedni stan i natychmiast dodaje nowy element do tymczasowego zapisu.
2. **onError** – w przypadku błędu przywraca poprzedni stan z kopii zapasowej.
3. **onSettled** – po zakończeniu operacji (sukces lub błąd) unieważnia bufor danych, wymuszając ponowne pobranie aktualnych danych.

Sposób działania optymistycznych aktualizacji w aplikacji:

1. Użytkownik klika "Dodaj składnik" – interfejs natychmiast pokazuje nowy element w tabeli.
2. W tle wysyłane jest żądanie `POST /api/ingredients` do backendu.
3. Jeśli serwer zwróci błąd, interfejs przywraca poprzedni stan i wyświetla komunikat o problemie.

Takie podejście sprawia, że aplikacja reaguje natychmiast na działania użytkownika.

4.6 Autoryzacja i ochrona tras

Mechanizm autoryzacji w warstwie frontend opiera się na standardzie JWT, który współpracuje z systemem JWT zaimplementowanym w warstwie backend. Przeglądarka zapisuje token otrzymany z serwera po zalogowaniu i dołącza go do każdego żądania

HTTP, co pozwala uwierzytelnić użytkownika oraz zabezpieczyć dane, do których ten użytkownik nie powinien mieć dostępu.

4.6.1 Przepływ autoryzacji

Kontekst autoryzacji AuthContext zarządza stanem tokenu i użytkownika udostępniając swoje dane i funkcje w całym drzewie komponentów aplikacji. Jego implementację w postacji interfejsu przedstawiono na Rys. 4.10.

```
interface AuthContextProps {
  user: User | null;
  token: string | null;
  setUser: (user: User | null) => void;
  setToken: (token: string | null) => void;
}

// eslint-disable-next-line react-refresh/only-export-components
export const AuthContext = createContext<AuthContextProps>({
  user: null,
  token: null,
  setUser: () => {},
  setToken: () => {},
});

export const AuthProvider = ({ children }: { children: ReactNode })
=> {
  const [user, setUser] = useState<User | null>(null);
  const [token, setTokenState] = useState<string | null>(() => {
    return localStorage.getItem('token');
  });

  useEffect(() => {
    if (token) {
      localStorage.setItem('token', token);
    } else {
      localStorage.removeItem('token');
    }
  }, [token]);

  const setToken = (newToken: string | null) => {
    setTokenState(newToken);
  };

  return (
```

```
        <AuthContext.Provider value={{ user, token, setUser, setToken
}}>
          {children}
        </AuthContext.Provider>
      );
    };
```

Rys. 4.10. Kontekst autoryzacji z pliku auth-context.tsx. Źródło: Opracowanie własne.

Kontekst ten pozwala zapewnić dostęp do danych użytkownika innym elementom aplikacji frontend, które mogą na bieżąco wykorzystywać dane takie jak obiekt użytkownika oraz token JWT.

4.6.2 Funkcja autoryzacji

Funkcja useAuth enkapsuluje logikę logowania i wylogowania. Jej implementację przedstawiono na Rys. 4.11.

```
    export const useAuth = () => {
      const { user, addUser, removeUser, token } = useUser();
      const navigate = useNavigate();

      const login = async (email: string, password: string) => {
        const { data, status } = await
apiInstance.post<AuthResponse>(LOGIN_URL, {
          email,
          password,
        });

        if (status === 200) {
          addUser(data.token);
          navigate('/');
        } else {
          return { error: true };
        }

        return { error: false };
      };

      const logout = () => {
        removeUser();
        navigate('/login');
      };

      return {
        user,
```

```
        token,
        login,
        logout,
      };
    };
```

Rys. 4.11. Kod funkcji useAuth w pliku use-auth.ts. Źródło: Opracowanie własne.

Widoczna tu metoda login wysyła żądanie POST do endpointu autoryzacji, a po pomyślnym uwierzytelnieniu zapisuje token i przekierowuje użytkownika na stronę główną. Metoda logout usuwa dane użytkownika i przekierowuje na stronę logowania. Proces logowania w aplikacji przebiega następująco:

- Użytkownik wprowadza dane w formularzu logowania.
- Frontend wysyła żądanie POST /api/auth/login z danymi uwierzytelniającymi.
- Backend weryfikuje dane i zwraca token JWT.
- Frotnend zapisuje token w localStorage przeglądarki i aktualizuje kontekst autoryzacji.
- Kolejne żądania HTTP do serwera dołączają token w nagłówku Authorization.

4.6.3 Funkcja użytkownika

Funkcja useUser odpowiada za dekodowanie tokenu JWT i ekstrakcję danych użytkownika pobranych z opisanej w podrozdziale 4.6.2. funkcji useAuth. Jej implementację przedstawiono na Rys. 4.12.

```
    export const useUser = () => {
      const { user, setUser, token, setToken } =
useContext(AuthContext);

      useEffect(() => {
        if (token) {
          addUser(token);
        } else {
          removeUser();
        }
        // eslint-disable-next-line react-hooks/exhaustive-deps
      }, [token]);

      const addUser = (token: string) => {
        setToken(token);
```

```
        apiInstance.defaults.headers.common['Authorization'] = 'Bearer
' + token;
        const decodedToken = decodeToken<Token>(token);
        const parsed = tokenSchema.safeParse(decodedToken);

        if (parsed.success) {
          setUser({
            id: parsed.data.id,
            email: parsed.data.sub,
            firstName: parsed.data.firstName,
            lastName: parsed.data.lastName,
            userRole: parsed.data.userRole,
          });
        } else {
          console.error(
            'There was an error while trying to validate token: ',
            parsed.error
          );
        }
      };

      const removeUser = () => {
        setUser(null);
        setToken(null); // This should trigger localStorage removal in
AuthContext
        delete apiInstance.defaults.headers.common['Authorization'];
      };

      return { user, addUser, removeUser, setUser, token, setToken };
    };
```

Rys. 4.12. Kod funkcji useUser w pliku use-user.ts. Źródło: Opracowanie własne.

Funkcja wykorzystuje bibliotekę react-jwt do dekodowania tokenu oraz schemat Zod tokenSchema do walidacji struktury. Dzięki temu aplikacja ma dostęp do danych użytkownika takich jak: identyfikator, email, imię, nazwisko oraz rola.

4.6.4 Trasy chronione

Mechanizm tras chronionych kontroluje dostęp do określonych stron lub komponentów na podstawie stanu autoryzacji użytkownika. Jest on realizowany przez komponent ProtectedRoute. Jego implementację przedstawiono na Rys. 4.13.

```
    export default function ProtectedRoute() {
      const { token, logout } = useAuth();
```

```
    const navigate = useNavigate();

    useEffect(() => {
      if (!token || isExpired(token)) {
        logout();
      }
    }, [token, navigate, logout]);

    // If authenticated, render the child routes
    return <Outlet />;
  }
```

Rys. 4.13. Kod komponentu ProtectedRoute w pliku protected-route.tsx. Źródło: Opracowanie własne.

Komponent ten zabezpiecza widoki wymagające autoryzacji. Przy każdym wejściu na chronioną ścieżkę sprawdza:

- Czy token istnieje w magazynie localStorage.
- Czy token nie wygasł - sprawdzenie pola exp w obiekcie tokenu.

W przypadku gdy warunki nie są spełnione, użytkownik jest przekierowywany na stronę logowania. Mechanizm ten stanowi uzupełnienie walidacji po stronie backend – nawet jeśli użytkownik ręcznie przejdzie do chronionego URL, serwer odrzuci żądania bez ważnego tokenu.

4.7 Struktura tras

Kod implementujący strukturę tras dla całej aplikacji przedstawiono na Rys. 4.14. Zdefiniowane tu trasy określają, jakie ścieżki w aplikacji są dostępne dla użytkowników oraz jakie komponenty zostaną wyświetlone po wejściu na określoną podstronę.

```
  // Public routes accessible to all users
  const routesForPublic = [
    {
      path: '/service',
      element: <div>Service Page</div>,
    },
    {
      path: '/about-us',
      element: <div>About Us</div>,
    },
  ];

  // Routes accessible only to NON-authenticated users (ALWAYS
included)
```

```
const routesForNotAuthenticatedOnly = [
  {
    path: '/login',
    element: <SignIn />,
  },
];

// Routes accessible only to authenticated users
const routesForAuthenticatedOnly = [
  {
    path: '/',
    element: <ProtectedRoute />,
    children: [
      {
        path: '/',
        element: <Home />,
      },
      {
        path: '/inventory',
        element: <Inventory />,
      },
      {
        path: '/employees',
        element: <EmployeesPage />,
      },
      {
        path: '/users',
        element: <UsersPage />,
      },
      {
        path: '/fixed-costs',
        element: <FixedCostsPage />,
      },
      {
        path: '/sales-reports',
        element: <SalesReportsPage />,
      },
      {
        path: '/about',
        element: <About />,
      },
      {
        path: '/account',
        element: <MyAccount />,
```

```
        },
        {
          path: '/feedback',
          element: <Feedback />,
        },
      ],
    },
  ];

  // Combine routes
  const router = createBrowserRouter([
    ...routesForPublic,
    ...routesForNotAuthenticatedOnly,
    ...routesForAuthenticatedOnly,
  ]);

  export default function Routes() {
    return <RouterProvider router={router} />;
  }
```

Rys. 4.14. Kod z definicjami tras aplikacji w pliku Routes.tsx. Źródło: Opracowanie własne.

Aplikacja definiuje trzy grupy tras odpowiadające różnym poziomom dostępu:

1. Trasy publiczne - dostępne bez logowania (np. strona informacyjna)
2. Trasy dla niezalogowanych – strona logowania, niedostępna po zalogowaniu
3. Trasy chronione - główne funkcjonalności aplikacji, wymagające autoryzacji

Taki podział sprawia, że zalogowany użytkownik nie zobaczy formularza logowania, a niezalogowany nie uzyska dostępu do panelu zarządzania.

4.8 Strony aplikacji

Każda strona aplikacji jest komponentem React renderowanym dla określonej ścieżki URL. Strony wykorzystują własne funkcje pomocnicze do pobierania danych oraz komponenty Material-UI do budowy interfejsu, co pozwala zachować komponentom charakter prezentacyjny ułatwiając tym zarządzanie kodem aplikacji.

4.8.1 Strona magazynu

Strona Inventory stanowi przykład pełnej integracji warstwy frontend z serwerem. Realizuje operacje CRUD na składnikach, odpowiadające ścieżkom API z kontrolera IngredientController, który został opisany w podrozdziale 3.5.:

- Wyświetlenie listy – GET /api/ingredients

- Dodanie składnika - POST /api/ingredients
- Usunięcie składnika - DELETE /api/ingredients
- Uzupełnienie stanu magazynu – POST /api/ingredients/{id}/restock

```
    const handleIngredientSubmit = (): void => {
        // Close dialog immediately for optimistic UX
        handleCloseIngredientDialog();

        // Trigger mutation (optimistic update will show it immediately)
        createIngredient.mutate(ingredientFormState, {
          onSuccess: () => {
            setSnackbarMsg('Ingredient created successfully');
          },
          onError: err => {
            const error = err as Error;
            setError(error.message || 'Failed to create ingredient');
          },
        });
      };
```

Rys. 4.15. Kod komponentu Inventory w pliku inventory.tsx. Źródło: Opracowanie własne.

Strona wykorzystuje optymistyczne aktualizacje zaimplementowane w kodzie widocznym na Rys. 4.15 – po kliknięciu “Dodaj” dialog zamyka się natychmiast, a nowy składnik pojawia się w tabeli jeszcze przed potwierdzeniem przez serwer. Jeśli backend zwróci błąd, np. duplikat nazwy, interfejs przywraca poprzedni stan i wyświetla komunikat.

4.8.2 Strona główna

Strona główna prezentuje kafelki ze statystykami pobieranymi ze ścieżki GET /api/statistics. Użytkownik może dostosować układ przez przeciąganie kafelków, co zostało zaimplementowane przy pomocy biblioteki @dnd-kit, która znacząco ułatwiła pracę z ruchomymi elementami strony. Konfiguracja kafelków zapisywana jest w localStorage przeglądarki, dzięki czemu układ jest zachowany między sesjami.Fragment kodu implementacji strony głównej przedstawiono na Rys. 4.16.

```
    const [isCustomizeMode, setIsCustomizeMode] = useState(false);
      const [snackbarMsg, setSnackbarMsg] = useState('');
      const [timeframe, setTimeframe] =
useState<StatisticsRange>('OVERALL');
```

```
      const  {  config,  addTile,  removeTile,  updateTileOrder,
resetToDefault } =
        useDashboardConfig();

      const sensors = useSensors(
        useSensor(PointerSensor, {
          activationConstraint: {
            distance: 8,
          },
        }),
        useSensor(KeyboardSensor, {
          coordinateGetter: sortableKeyboardCoordinates,
        })
      );

      const handleDragEnd = (event: DragEndEvent) => {
        const { active, over } = event;

        if (over && active.id !== over.id) {
          const oldIndex = config.tiles.findIndex(tile => tile.id ===
active.id);
          const newIndex = config.tiles.findIndex(tile => tile.id ===
over.id);

          const newTiles = arrayMove(config.tiles, oldIndex, newIndex);
          updateTileOrder(newTiles);
        }
      };

      const handleAddTile = (type: DashboardTileType, width: number) =>
{
        addTile(type, width);
        setSnackbarMsg(`Tile added to dashboard`);
      };

      const handleRemoveTile = (id: string) => {
        removeTile(id);
        setSnackbarMsg('Tile removed from dashboard');
      };

      const handleResetDashboard = () => {
        resetToDefault();
        setSnackbarMsg('Dashboard reset to default layout');
      };
```

Rys. 4.16. Fragment kodu komponentu HomePage w pliku home.tsx. Źródło: Opracowanie własne.

Strona główna oferuje:

1. **Tryb edycji** – przełącznik umożliwiający dostosowanie układu, co pozwala użytkownikom dostosować widoczne metryki sprzedażowe
2. **Przeciąganie i upuszczanie** (handleDragEnd) – przenoszenie kafelków ułatwiające dostosowanie widoku do własnych potrzeb
3. **Dynamiczne kafelki** – możliwość dodawania i usuwania widżetów.

4.9 Stylizacja i motyw

Aplikacja wykorzystuje system motywów Material-UI z własnymi dostosowaniami. Zdefiniowano paletę kolorów marki oraz obsługę trybu jasnego i ciemnego. Centralizacja stylów w motywie zapewnia spójność wizualną - zmiana koloru głównego w jednym miejscu automatycznie aktualizuje wszystkie komponenty. Kod motywu AppTheme jest widoczny na Rys. 4.17.

```
export default function AppTheme({
  children,
  disableCustomTheme,
  themeComponents,
}: AppThemeProps) {
  const theme = React.useMemo(() => {
    return disableCustomTheme
      ? {}
      : createTheme({
          cssVariables: {
            colorSchemeSelector: 'data-mui-color-scheme',
            cssVarPrefix: 'template',
          },
          colorSchemes,
          typography: {
            ...typography,
            button: {
              ...typography.button,
              textTransform: 'none' as const,
            },
          },
          shadows,
          shape,
          components: {
            ...inputsCustomizations,
            ...dataDisplayCustomizations,
            ...feedbackCustomizations,
```

```
                ...navigationCustomizations,
                ...surfacesCustomizations,
                ...themeComponents,
              },
            });
    }, [disableCustomTheme, themeComponents]);
    if (disableCustomTheme) {
      return <React.Fragment>{children}</React.Fragment>;
    }
    return (
      <ThemeProvider theme={theme} disableTransitionOnChange>
        {children}
      </ThemeProvider>
    );
  }
```

Rys. 4.17. Kod komponentu motywu AppTheme w pliku AppTheme.tsx. Źródło: Opracowanie własne.

Jak widać w kodzie implementacja obejmuje następujące elementy:

1. **Zmienne CSS** (cssVariables) – wykorzystanie zmiennych CSS do dynamicznego przełączania motywów.
2. **Schematy kolorów** (colorSchemes) – obsługa trybu jasnego i ciemnego.
3. **Typografia** (typography) **– konfiguracja typografii z wyłączeniem automatycznej transformacji na wielkie litery dla przycisków.**
4. **Dostosowania komponentów** (components) – modularne modyfikacje wyglądu dla poszczególnych kategorii komponentów.

4.10 Komponenty strony głównej

Kafelki (tile) strony głównej są samodzielnymi komponentami wyświetlającymi metryki i dane sprzedażowe. Wykorzystują one pobrane dane statystyk i prezentują je w formie graficznej. Każdy kafelek reaguje na zmianę przedziału czasowego pobierania danych wybranego przez użytkownika, co powoduje ponowne pobranie danych z odpowiednimi parametrami. Fragment kodu komponentu Dashboard przedstawiono na Rys. 4.18.

```
  export const DashboardTileComponent: React.FC<DashboardTileProps> = ({
    type,
  }) => {
    switch (type) {
      case 'total-sales':
        return <TotalSalesTile />;
```

```
        case 'total-orders':
          return <TotalOrdersTile />;
        case 'avg-order-value':
          return <AvgOrderValueTile />;
        case 'highest-order':
          return <HighestOrderTile />;
        case 'total-profit':
          return <TotalProfitTile />;
        case 'total-expense':
          return <TotalExpenseTile />;
        case 'ingredient-costs':
          return <IngredientCostsTile />;
        case 'fixed-costs':
          return <FixedCostsTile />;
        case 'cash-income':
          return <CashIncomeTile />;
        case 'card-income':
          return <CardIncomeTile />;
        case 'last-transaction':
          return <LastTransactionTile />;
        case 'avg-margin':
          return <AvgMarginTile />;
        case 'avg-items-per-order':
          return <AvgItemsPerOrderTile />;
        case 'low-stock-alert':
          return <LowStockAlertTile />;
        case 'payment-methods':
          return <PaymentMethodsTile />;
        case 'top-products-revenue':
          return <TopProductsRevenueTile />;
        case 'top-products-units':
          return <TopProductsUnitsTile />;
        case 'low-stock-items':
          return <LowStockItemsTile />;
        case 'recent-transactions':
          return <RecentTransactionsTile />;
        default:
          return (
            <Paper sx={{ p: 2 }}>
              <Typography>Unknown tile type: {type}</Typography>
            </Paper>
          );
      }
    };
```

Rys. 4.18. Fragment kodu komponentu DashboardTiles w pliku DashboardTiles.tsx. Źródło: Opracowanie własne.

Każdy kafelek:

1. **Wykorzystuje funkcje danych** – np. `useStatistics` do pobierania statystyk
2. **Obsługuje kontekst czasowy** – `useTimeframe` umożliwia filtrowanie pobieranych danych po przedziale czasowym
3. **Wyświetla stan ładowania** – pokazuje wskaźnik ładowania podczas pobierania danych
4. **Zawiera stylizację gradientową** – wizualne rozróżnienie typów kafelków.

4.11 Responsywność

Aplikacja dostosowuje się do rozmiaru ekranu dzięki funkcji useMediaQuery zdefiniowanej w bibliotece Material-UI. Na urządzeniach mobilnych:

- Tabele zamieniają się w widok kart
- Dialogi wyświetlają się w trybie pełnoekranowym
- Menu boczne jest domyślnie zwinięte

Takie podejście zapewnia dostosowanie interfejsu aplikacji do urządzeń o różnych rozmiarach wyświetlaczy.

4.11.1 Adaptacyjne tabele

Komponent ResponsiveTable automatycznie przełącza się między widokiem tabelarycznym na dużych ekranach a widokiem kart na urządzeniach mobilnych. Kluczowe cechy komponentu wykorzystane w opisywanej aplikacji to:

1. **Wykrywanie rozmiaru ekranu** – funkcja `useMediaQuery` widoczna na Rys. 4.19. sprawdza, czy szerokość ekranu jest mniejsza niż punkt przerwania `md` (960px).
2. **Warunkowe renderowanie** – na urządzeniach mobilnych dane wyświetlane są w formie kart zamiast wierszy tabeli, co zostało ukazane na Rys. 4.20.
3. Ukrywanie kolumn – właściwość hideOnMobile przedstawiona oraz użyta na Rys. 4.21. pozwala pomijać mniej istotne kolumny na małych ekranach.
4. **Pełnoekranowe dialogi** – formularze modalne automatycznie przełączają się w tryb pełnoekranowy przy pomocy zmiennej fullScreenDialog przedstawionej na Rys. 4.22. na urządzeniach mobilnych.

```
const isMobile = useMediaQuery(theme.breakpoints.down('md'));
```

Rys. 4.19. Kod definicji zniennej isMobile w pliku responsive-table.tsx. Źródło: Opracowanie własne

```
if (isMobile) {
```

```
        return (
          <Stack spacing={2} data-testid={testId}>
    ...
```

Rys. 4.20. Fragment kodu warunkowego wyświetlania elementów z komponentu ResponsiveTable w pliku responsive-table.tsx. Źródło: Opracowanie własne.

```
    <Stack spacing={1}>
                        {columns
                          .filter(col => !col.hideOnMobile)
    ...
```

Rys. 4.21. Fragment kodu ukrywania kolumn komponentu ResponsiveTable w pliku responsive-table.tsx. Źródło: Opracowanie własne.

```
    const                        fullScreenDialog                    =
useMediaQuery(theme.breakpoints.down('sm'));
```

Rys. 4.22. Kod definicji zmiennej fullScreenDialog w pliku employee.tsx. Źródło: Opracowanie własne.

4.11.2 Punkty przerwania

Aplikacja wykorzystuje standardowe punkty przerwania Material-UI:

- **xs** (0px) – bardzo małe ekrany (telefony w orientacji pionowej).
- **sm** (600px) – małe ekrany (telefony w orientacji poziomej).
- **md** (900px) – średnie ekrany (tablety).
- **lg** (1200px) – duże ekrany (laptopy).
- **xl** (1536px) – bardzo duże ekrany (monitory).

Przykład wykorzystania punktów przerwania w stylach przedstawiono na Rys. 4.23.

```
    const  fullScreenDialog=
useMediaQuery(theme.breakpoints.down('sm'));
```

Rys. 4.23. Fragment kodu komponentu Inventory w pliku inventory.tsx. Źródło: Opracowanie własne.

Zmienna fullScreenDialog przyjmuje wartość true na ekranach mniejszych niż punkt sm, powodując wyświetlenie dialogu w trybie pełnoekranowym.

4.12 Testowanie

Dla części frontend aplikacji zdefiniowano testy przy pomocy biblioteki Playwright. Testy symulują rzeczywiste interakcje użytkownika z przeglądarką, weryfikując poprawność integracji warstwy frontend oraz backend, jak i również poprawność integracji samych komponentów aplikacji frontend.

4.12.1 Konfiguracja testów

Konfiguracja Playwright zdefiniowana jest w pliku playwright.config.ts. Jej fragment przedstawiono na Rys. 4.24.

```
dotenv.config();

const AUTH_FILE = 'tests/.auth/user.json';

export default defineConfig({
  use: {
    baseURL: 'http://localhost:5173',
    headless: true,
    screenshot: 'only-on-failure',
    // Reduced timeouts since API responses are mocked
    actionTimeout: 5000,
    navigationTimeout: 10000,
  },
  testDir: './tests',
  // Reduced overall timeout since API calls are mocked
  timeout: 30000,
  // Disable retries since mocked tests should be deterministic
  retries: 0,
  projects: [
    // Setup project - runs first to authenticate
    {
      name: 'setup',
      testMatch: /auth\.setup\.ts/,
    },
    // Unauthenticated tests - run without auth setup
    {
      name: 'unauthenticated',
      testMatch: /auth\.spec\.ts|error-handling\.spec\.ts/,
      testIgnore: /auth\.setup\.ts/,
    },
    // Main tests - depend on setup and use stored auth state
    {
      name: 'chromium',
      use: {
        storageState: AUTH_FILE,
      },
      dependencies: ['setup'],
      testIgnore:           /auth\.setup\.ts|auth\.spec\.ts|error-
handling\.spec\.ts/,
    },
```

```
    ],
    webServer: {
      command: 'npm run dev',
      url: 'http://localhost:5173',
      reuseExistingServer: !process.env.CI,
      timeout: 120 * 1000,
    },
  });
```

Rys. 4.24. Konfiguracja Playwright w pliku playwright.config.ts. Źródło: Opracowanie własne.

Jak widać, w kodzie zdefiniowano trzy projekty testowe:

1. **setup** – projekt inicjalizacyjny odpowiedzialny za uwierzytelnienie użytkownika testowego.
2. **unauthenticated** – testy wykonywane bez zalogowanego użytkownika (np. testy strony logowania).
3. **chromium** – główna grupa testów wymagających autoryzacji, wykorzystująca zapisany stan sesji.

4.12.2 Selektory

Selektory Playwright pozwalają na precyzyjne określenie testowanego elementu HTML przedstawionego na danej stronie. Pliki z selektorami enkapsulują selektory i akcje związane z poszczególnymi stronami aplikacji. Fragment kodu z selektorami dla komponentu Inventory klasy InventoryPage przedstawiono na Rys. 4.25.

```
    // Page header
        this.pageTitle  =  page.locator('[data-testid="inventory-page-
title"]');

        // Tabs
        this.ingredientsTab = page.locator('[data-testid="ingredients-
tab"]');
        this.productsTab    =    page.locator('[data-testid="products-
tab"]');

        // Ingredients section
        this.addIngredientButton = page.locator(
          '[data-testid="add-ingredient-button"]'
        );
        this.ingredientsTable          =          page.locator('[data-
testid="ingredients-table"]');
        this.ingredientRows  =  page.locator('[data-testid="ingredient-
row"]');
```

```
        this.lowStockToggle  =  page.locator('[data-testid="low-stock-
toggle"]');

        // Products section
        this.addProductButton    =    page.locator('[data-testid="add-
product-button"]');
        this.productsTable   =   page.locator('[data-testid="products-
table"]');
        this.productRows    =    page.locator('[data-testid="product-
row"]');
```

Rys. 4.25. Kod klasy InventoryPage dla strony magazynu. Źródło: Opracowanie własne.

Klasa ta definiuje selektory dla elementów strony, takie jak zakładki, przyciski, tabele i formularze.

4.12.3 Struktura testów

Testy zorganizowane są w pliki odpowiadające poszczególnym funkcjonalnościom aplikacji. Testy zdefiniowane dla strony magazynu przedstawiono na Rys. 4.26.

```
    test.beforeEach(async ({ inventoryPage }) => {
      await inventoryPage.goto();
      await inventoryPage.expectToBeOnInventoryPage();
    });

    test.describe('Inventory Page Tests', () => {
      test('should display ingredients tab by default', async ({
        inventoryPage,
      }) => {
        await expect(inventoryPage.ingredientsTab).toBeVisible();
        await expect(inventoryPage.addIngredientButton).toBeVisible();
      });

      test('should switch between ingredients and products tabs', async
({
        inventoryPage,
      }) => {
        await inventoryPage.switchToProductsTab();
        await expect(inventoryPage.addProductButton).toBeVisible();

        await inventoryPage.switchToIngredientsTab();
        await expect(inventoryPage.addIngredientButton).toBeVisible();
      });
```

```
      test('should have low stock toggle filter', async ({ inventoryPage
}) => {
        await expect(inventoryPage.lowStockToggle).toBeVisible();
      });

      test('should display ingredients table', async ({ inventoryPage
}) => {
        await expect(inventoryPage.ingredientsTable).toBeVisible();
      });

      test('should display products table when on products tab', async
({
        inventoryPage,
      }) => {
        await inventoryPage.switchToProductsTab();
        await expect(inventoryPage.productsTable).toBeVisible();
      });
    });
```

Rys. 4.26. Testy strony magazynu. Źródło: Opracowanie własne.

Każdy test poprzedzony jest akcją beforeEach, która nawiguje do testowanej strony i weryfikuje jej poprawne załadowanie. W katalogu tests znajdują się testy dla wszystkich głównych funkcjonalności aplikacji:

- **auth.spec.ts** – testy procesu logowania i wylogowania.
- **dashboard.spec.ts** – testy strony głównej z kafelkami.
- **inventory.spec.ts** – testy zarządzania składnikami i produktami.
- **employees.spec.ts** – testy zarządzania pracownikami.
- **navigation.spec.ts** – testy nawigacji między stronami.
- **responsive.spec.ts** – testy responsywności interfejsu.

4.12.4 Uruchomienie testów

Uruchomienie pełnego zestawu testów poleceniem `npm run test:e2e` skutkuje równoległym wykonaniem 60 testów przy użyciu 8 wątków roboczych. Fragment wyjścia konsoli po uruchomieniu tego polecenia przedstawiono na Rys. 4.27.

```
* Executing task: npm run test:e2e
  ✓  58 [chromium] › tests\navigation.spec.ts:76:3 › Side Menu Tests › should display options menu button (1.5s)
  ✓  59 [chromium] › tests\navigation.spec.ts:80:3 › Side Menu Tests › should open options menu when button is clicked (1.5s)
  ✓  60 [chromium] › tests\navigation.spec.ts:89:3 › Side Menu Tests › should have all navigation items visible (1.5s)

  Slow test file: [chromium] › tests\navigation.spec.ts (21.0s)
  Consider running tests from slow files in parallel, see https://playwright.dev/docs/test-parallel.
  60 passed (31.1s)
```

Rys. 4.27. Zrzut ekranu linii poleceń po wykonaniu polecenia npm run test:e2e. Źródło: Opracowanie własne.

Wszystkie 60 przypadków testowych zakończyło się powodzeniem przy czasie uruchomienia 31,1 sekund. W wyjściu konsoli przy każdym teście wyświetlony został jego numer porządkowy, środowisko uruchomieniowe przeglądarki chromium, ścieżka do pliku specyfikacji, nazwa specyfikacji testu oraz nazwa konkretnego przypadku, a także czas jego wykonania w sekundach. Playwright automatycznie identyfikuje pliki testowe o wydłużonym czasie wykonania — w tym przypadku plik navigation.spec.ts został oznaczony jako wolny przez jego czas wykonania 21,0 sekund, co wynika z dużej liczby zawartych w nim scenariuszy nawigacyjnych wymagających wielokrotnego ładowania stron. Równoległe wykonanie testów na 8 wątkach roboczych pozwoliło znacząco skrócić całkowity czas uruchomienia zestawu testowego w porównaniu z wykonaniem sekwencyjnym.

PODSUMOWANIE I WNIOSKI KOŃCOWE

Celem pracy było zaprojektowanie oraz implementacja systemu webowego (aplikacji internetowej) do wspomagania działalności gastronomicznej. W ramach pracy wykonano w pełni funkcjonalny system GastroM ułatwiający zarządzanie zakładem gastronomicznym, w tym pracownikami, magazynem i transakcjami. System został zaimplementowany w technologiach Spring oraz React z TypeScript, w architekturze klient–serwer. Wykonano testy jednostkowe 6 klas i 31 funkcji, 64 testy integracyjne oraz 60 testów end-to-end. System jest obecnie uruchomiony i dostępny na platformie Render pod adresem https://gastrom.onrender.com.

Dla systemu zdefiniowano 15 wymagań funkcjonalnych i 7 wymagań pozafunkcjonalnych, których weryfikacja została przedstawiona poniżej.

Wymagania funkcjonalne:

F1 – zweryfikowane pozytywnie w 100%. Użytkownik może dodawać, edytować i usuwać składniki; realizację opisano w podrozdziałach 3.4, 3.5, 3.6 oraz 4.8.1.

F2 – zweryfikowane pozytywnie w 100%. Operacja zwiększania stanu składnika z rejestracją kosztu; realizację opisano w podrozdziałach 3.4, 3.6 oraz 4.8.1.

F3 – zweryfikowane pozytywnie w 100%. Użytkownik może dodawać, modyfikować i usuwać produkty wraz ze składnikami; realizację opisano w podrozdziałach 3.4, 3.5, 3.6 oraz 4.8.1.

F4 - zweryfikowane pozytywnie w 100%. Dane są pobierane i buforowane z automatycznym odświeżaniem; realizację opisano w podrozdziałach 3.5, 4.5.2, 4.5.3 oraz 4.8.1.

F5 – zweryfikowane pozytywnie w 100%. Użytkownik może dodawać, edytować i usuwać pracowników; realizację opisano w podrozdziałach 3.2 oraz 2.3.

F6 – zweryfikowane w 60%. System rejestruje przepracowane godziny pracowników wpisywane manualnie do systemu; realizację opisano w podrozdziałach 3.3.1, 2.2 oraz 2.3., natomiast nie zaimplementowano rzeczywistej integracji z systemami liczenia czasu pracy.

F7 - zweryfikowane pozytywnie w 100%. Wynagrodzenia obliczane na podstawie stawki i godzin są rejestrowane jako koszty stałe; realizację opisano w podrozdziałach 3.3.1, 3.4 oraz 2.3.

F8 – zweryfikowane pozytywnie w 100%. Panel wyświetla tabelę ze szczegółami pracowników; realizację opisano w podrozdziałach 3.5 oraz 2.3.

F9 – zweryfikowane pozytywnie w 100%. Administrator może dodawać i usuwać konta użytkowników; realizację opisano w podrozdziałach 3.8.1, 3.8.2 oraz 2.3.

F10 - zweryfikowane pozytywnie w 100%. Dostępna jest lista transakcji ze szczegółami; realizację opisano w podrozdziałach 2.2 oraz 3.3.2.

F11 – zweryfikowane pozytywnie w 100%. Dostęp do aktualnych danych o operacjach handlowych – zweryfikowane pozytywnie. Dane są automatycznie odświeżane dzięki mechanizmowi buforowania; realizację opisano w podrozdziałach 4.5.2 oraz 4.5.3.

F12 – zweryfikowane pozytywnie w 100%. Dane prezentowane są w kafelkach panelu głównego z wyborem przedziału czasowego; realizację opisano w podrozdziałach 3.5, 3.6, 4.8.2 oraz 4.10.

F13 – zweryfikowane w 100%. Wyświetlane są dane o sprzedaży i rentowności poszczególnych produktów; realizację opisano w podrozdziałach 3.3.2, 3.6 oraz 4.10.

F14 – zweryfikowane w 100%. Składniki poniżej progu ostrzegawczego są wyróżniane w interfejsie; realizację opisano w podrozdziałach 3.4 oraz 4.8.1.

F15 - zweryfikowane pozytywnie w 100%. Raport zawiera statystyki za wybrany okres i jest dostępny do pobrania; realizację opisano w podrozdziałach 3.6 oraz 2.2.

Wymagania pozafunkcjonalne:

PF1 - niezweryfikowane. Wymaganie zakłada obsługę pojedynczego żądania w czasie nie dłuższym niż 2 sekundy. Czas ładowania stron nie był mierzony, w związku z czym wymaganie pozostaje niezweryfikowane. Implementacja obejmuje podrozdziały 3.2 oraz 4.5.2.

PF2 - zaimplementowane, niezweryfikowane. Wymaganie zakłada ciągłość działania i bezpieczeństwo danych w przypadku awarii. Zostało zaimplementowane, ale nie zostało zweryfikowane, ponieważ system nie jest użytkowany. Implementacja obejmuje podrozdziały 1.4, 3.6, 3.3.1 oraz 2.4.

PF3 - zweryfikowane pozytywnie w 100%. Wymaganie zakłada haszowanie haseł użytkowników. Weryfikacja potwierdzona na podstawie kodu źródłowego – implementacja obejmuje podrozdziały 3.8.1 oraz 3.8.2.

PF4 - zweryfikowane pozytywnie 100%. Wymaganie zakłada wymóg logowania i autoryzacji zgodnej z rolą użytkownika. Weryfikacja potwierdzona na podstawie kodu źródłowego i testów – implementacja obejmuje podrozdziały 3.8.1, 3.8.2, 3.8.3, 4.6.1, 4.6.2 oraz 4.6.4.

PF5 - niezweryfikowane. Wymaganie zakłada intuicyjność interfejsu użytkownika. Nie zostało zweryfikowane, ponieważ interfejs nie był testowany na użytkownikach. Implementacja obejmuje podrozdziały 2.3 oraz 4.9.

PF6 - zaimplementowane, niezweryfikowane. Wymaganie zakłada skalowalność systemu i możliwość łatwego rozszerzania funkcjonalności. Zostało zaimplementowane, ale nie zostało zweryfikowane, ponieważ system nie jest użytkowany. Implementacja obejmuje podrozdziały 3.2, 4.2, 4.3 oraz 2.4.

PF7 - zweryfikowane pozytywnie w 100%. Wymaganie zakłada działanie w architekturze klient–serwer z protokołem HTTP. Weryfikacja potwierdzona – aplikacja jest dostępna pod publicznym adresem internetowym i działa poprawnie w przeglądarkach na systemach Windows, macOS i Linux. Implementacja obejmuje rozdziały 3 i 4 oraz podrozdział 2.4.

Wybór technologii Spring i React okazał się trafny – oba frameworki oferują rozbudowane ekosystemy, które umożliwiły sprawne zbudowanie systemu z podziałem na warstwy. Szczególnie użyteczna okazała się biblioteka TanStack Query do zarządzania stanem danych po stronie klienta oraz Spring Data JPA upraszczająca dostęp do bazy danych. Implementacja mechanizmu uwierzytelniania JWT oraz systemu blokad pesymistycznych stanowiła jedno z większych wyzwań technicznych, wymagając poświęcenia dodatkowego czasu na testowanie i debugowanie. Testy end-to-end z użyciem Playwright przebiegały sprawnie dzięki wzorcowi Page Object, chociaż konfiguracja środowiska testowego wymagała więcej czasu niż początkowo zakładano.

W dalszej kolejności należałoby przeprowadzić testy użyteczności z udziałem rzeczywistych użytkowników, zmierzyć wydajność systemu pod obciążeniem oraz rozważyć wdrożenie systemu w rzeczywistym zakładzie gastronomicznym. Można również rozszerzyć funkcjonalność o obsługę rezerwacji, system powiadomień czy integrację z systemami płatności.

Prace nad systemem zakończyły się na etapie testowania kodu i integracji modułów. GastroM stanowi obecnie kompletny system przygotowany do wdrożenia i testowania na użytkownikach.

BIBLIOGRAFIA

1. Martin R. C.: Clean Architecture: A Craftsman's Guide to Software Structure and Design, Pearson Education, 2018.
2. https://asrjetsjournal.org/American_Scientific_Journal/article/view/11686/2843, Sereda D.: Creating a Multi-tier Architecture for Web Applications: Design and Implementation, 07.06.2025 r., (data dostępu 03.10.2025 r.).
3. https://learn.microsoft.com/pl-pl/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures, Typowe architektury aplikacji internetowych, Microsoft Learn, 18.01.2025 r., (data dostępu 03.10.2025 r.).
4. https://www.ijert.org/research/software-engineeringweb-development-life-cycle-IJERTV2IS3438.pdf, Kamatchia R., Iyer J., Singh S.: Software Engineering: Web Development Life Cycle, 03.2013 r., (data dostępu 03.10.2025 r.).
5. https://2024.stateofjs.com/en-US/libraries/front-end-frameworks/, Front-end Frameworks, 2024 r., (data dostępu 03.10.2025 r.).
6. https://www.researchgate.net/publication/244446574_Understanding_separation_of_concerns, Mili H. i inni: Understanding Separation of Concerns, 01.2004 r., (data dostępu 08.10.2025 r.).
7. Baeza-Yates R.: SOFSEM 2007: Theory and Practice of Computer Science, Harrachov, Czech Republic, 33rd Conference on Current Trends in Theory and Practice of Computer Science, 2007 r.
8. Frost B.: Atomic Design, 2016 r.
9. https://www.researchgate.net/profile/Venkata-Ballamudi/publication/374154236_Front-End_Development_in_React_An_Overview/links/6510d94f61f18040c220eb13/Front-End-Development-in-React-An-Overview.pdf, Chen S., Thaduri U. R., Ballamudi V. K. R.: Front-End Development in React: An Overview, 2019 r., (data dostępu 03.10.2025 r.).
10. Gackenheimer C.: Introduction to React, New York, Apress L.P., 2015.
11. Elrom E.: React and Libraries: Your Complete Guide to the React Ecosystem, Apress Media LLC, 2021 r.
12. Boduch A.: React 16 Tooling, Packt Publishing, 2018 r.
13. Kato D.: Micro State Management with React Hooks, Packt Publishing, 2022 r.
14. https://www.researchgate.net/profile/Sabyasachi-Mondal-8/publication/387190714_Enhancing_React_Application_Performance_Proven_Strategies_and_Best_Practices/links/6763cf6f7784cf161e0b3fd5/Enhancing-React-Application-Performance-Proven-Strategies-and-Best-P, Mondal S.: Enhancing React Application Performance: Proven Strategies and Best Practices (data dostępu 03.10.2025 r.).
15. https://arxiv.org/pdf/2504.03884, Chen K.: Improving Front-end Performance through Modular Rendering and Adaptive Hydration (MRAH) in React Applications (data dostępu 08.10.2025 r.).

16. https://mui.com/material-ui/, Material UI documentation (data dostępu 08.10.2025 r.).
17. https://chakra-ui.com/, Chakra UI documentation (data dostępu 08.10.2025 r.).
18. https://react.dev/learn/passing-props-to-a-component, Passing props to a component (data dostępu 08.10.2025 r.).
19. https://par.nsf.gov/servlets/purl/10157540, Madsen M., Lhoták O., Tip F.: A Semantics for the Essence of React (data dostępu 08.10.2025 r.).
20. https://www.researchgate.net/publication/371724261_An_Analysis_of_XSS_Vulnerabilities_and_Prevention_of_XSS_Attacks_in_Web_Applications, Jayawardana H. D. i inni: An Analysis of XSS Vulnerabilities and Prevention of XSS Attacks in Web Applications (data dostępu 08.10.2025 r.).
21. https://publications.cispa.saarland/3505/1/raid21-csrf-defenses.pdf, Likaj X., Khodayari S., Pellegrino G.: Where We Stand (or Fall): An Analysis of CSRF Defenses in Web Frameworks (data dostępu 08.10.2025 r.).
22. https://arxiv.org/pdf/2410.14924, Kishnani U., Das S.: Securing the Web: Analysis of HTTP Security Headers in Popular Global Websites (data dostępu 08.10.2025 r.).
23. Kumar S.: A review on client-server based applications and research opportunity, International Journal of Recent Scientific Research, Tom 10, 7, 2109.
24. https://www.researchgate.net/publication/376605859_Research_on_the_Application_of_Layered_Architecture_in_Computer_Software_Development, Tu Z.: Research on the Application of Layered Architecture in Computer Software Development (data dostępu 08.10.2025 r.).
25. https://tms-outsource.com/blog/posts/java-statistics, Sandu B.: Java statistics that highlight its dominance (data dostępu 08.10.2025 r.).
26. https://snyk.io/blog/spring-dominates-the-java-ecosystem-with-60-using-it-for-their-main-applications/, Vermeer B.: Spring dominates the Java ecosystem with 60% using it for their main applications (data dostępu 08.10.2025 r.).
27. Dessi M.: Spring 2.5 Aspect Oriented Programming, Birmingham, Packt Publishing Limited, 2009.
28. Walls C.: Spring w akcji, Wydanie V, HELION S.A, 2019.
29. https://docs.spring.io/spring-framework/reference/web/webmvc.html, Spring Web MVC documentation (data dostępu 08.10.2025 r.).
30. Robillard P. M.: Introduction to Software Design with Java, Cham, Springer, 2022 r.
31. Walls C.: Spring w Akcji, Helion, 2015.
32. https://spring.io/projects/spring-data, Spring Data documentation (data dostępu 08.10.2025 r.).
33. https://spring.io/projects/spring-security, Spring Security documentation (data dostępu 08.10.2025 r.).
34. https://docs.spring.io/spring-framework/reference/testing.html, Spring Testing documentation (data dostępu 08.10.2025 r.).

35. https://cm.tm.kit.edu/download/Authentication_and_Authorization_Patterns_in_Spring_Security.pdf, Dikanski A., Steinegger R., Abeck S.: Identification and Implementation of Authentication and Authorization Patterns in the Spring Security Framework (data dostępu 08.10.2025 r.).
36. https://carijournals.org/journals/index.php/IJCE/article/view/2201/2586, Joshi P. K.: Spring Boot Security in Payment Gateway Applications (data dostępu 08.10.2025 r.).
37. https://www.researchgate.net/publication/337419511_Spring_Security_Architecture_and_Design, Scarioni C., Nardone M.: Spring Security Architecture and Design (data dostępu 08.10.2025 r.).
38. https://dsf.berkeley.edu/papers/fntdb07-architecture.pdf, Hellerstein J. M., Stonebraker M., Hamilton J.: Architecture of a Database System, 2007 r., (data dostępu 08.10.2025 r.).
39. https://dsf.berkeley.edu/papers/ERL-M85-95.pdf, Stonebraker M., Rowe L. A.: The Design of Postgres, (data dostępu 08.10.2025 r.).
40. https://www.cs.purdue.edu/homes/bb/cs542-11Spr/cc.pdf, Bhargava B.: Concurrency Control in Database Systems (data dostępu 08.10.2025 r.).
41. https://citeseerx.ist.psu.edu/document?doi=02f8961adc67f88fa481ec04aef5cf90575264f9&repid=rep1&type=pdf, Kim M. J., Nelson D. A., Rossiter B. N.: Evaluation of the Object Relational DBMS Postgres I Administrative Data (data dostępu 08.10.2025 r.).
42. https://www.researchgate.net/publication/342579919_A_Study_of_Software_Testing_Categories_Levels_Techniques_and_Types, Umar M. A.: A Study of Software Testing: Categories, Levels, Techniques, and Types (data dostępu 08.10.2025 r.).
43. https://www.researchgate.net/publication/387460992_Software_Testing_Techniques_and_Levels_in_Software_Development, Tetteh S. G.: Software Testing Techniques and Levels in Software Development (data dostępu 08.10.2025 r.).
44. https://www.researchgate.net/publication/370747253_A_Comparative_Analysis_of_Static_and_Dynamic_Code_Analysis_Techniques, Silva D. D., Samarasekara P. H., Hettiarachchi R.: A Comparative Analysis of Static and Dynamic Code Analysis Techniques (data dostępu 08.10.2025 r.).
45. https://arxiv.org/pdf/2307.10034, Attouche L. i inni: Validation of Modern JSON Schema: Formalization and Complexity (data dostępu 08.10.2025 r.).
46. https://link.springer.com/article/10.1007/s11219-024-09686-0, Karlsson S. i inni: Exploring behaviours of RESTful APIs in an industrial setting (data dostępu 08.10.2025 r.).
47. https://www.mdpi.com/2079-9292/12/5/1252, Ma S.-P. i inni: RESTful API Analysis, Recommendation, and Client Code Retrieval (data dostępu 08.10.2025 r.).
48. https://link.springer.com/article/10.1007/s10664-023-10367-y, Bogner J., Kotstein S., Pfaff T.: Do RESTful API design rules have an impact on the understandability of Web APIs? (data dostępu 08.10.2025 r.).

49. https://blog.logrocket.com/guide-performance-optimization-webpack, Weber S.: LogRocket (data dostępu 08.10.2025 r.).
50. https://jsaer.com/download/vol-8-iss-7-2021/JSAER2021-8-7-261-264.pdf, Kothapalli M.: The Evolution of Component-Based Architecture in Front-End Development (data dostępu 08.10.2025 r.).
51. https://jtipublishing.com/jti/article/view/7/126, Devarapalli C. A.: Journal of Technological Innovations (data dostępu 08.10.2025 r.).
52. https://www.researchgate.net/publication/387938426_A_Review_on_Concurrency_Control_Techniques_in_Database_Management_Systems, Shams M. Y., Abolaban A. S., Shams M. Y.: A Review on Concurrency Control Techniques in Database Management Systems (data dostępu 08.10.2025 r.).

STRESZCZENIE

Autorzy: Igor Kohsin, Oskar Krupa

Promotor: dr hab. Ewa Ratajczak-Ropel, prof. UMG

Tytuł pracy: Implementacja webowego systemu zarządzania działalnością gastronomiczną z użyciem Spring i React

Aplikacje webowe są obecnie podstawą zarówno nowoczesnego biznesu, jak i codziennego życia. Praca dotyczy tworzenia tego typu systemu klasy ERP (Enterprise Resource Planning) przeznaczonego dla managerów zakładów gastronomicznych, umożliwiając centralizację danych, automatyzację procesów oraz usprawnienie zarządzania przedsiębiorstwem.

Celem pracy było zaprojektowanie i implementacja systemu webowego obsługującego magazyn, pracowników, transakcję, statystyki i raporty finansowe. System, dla którego przyjęto nazwę GastroM został zbudowany w architekturze klient–serwer z wykorzystaniem frameworka Spring Boot w warstwie backend oraz biblioteki React z TypeScript w warstwie frontend. Warstwa danych oparta jest na relacyjnej bazie danych PostgreSQL.

Praca składa się z czterech rozdziałów. Część teoretyczna została poświęcona zagadnieniom tworzenia aplikacji webowych. W części praktycznej opisano projekt i implementację systemu GastroM. Porzedstawiono m. in. analizę wymagań funkcjonalnych i pozafunkcjonalnych, projekt interfejsu użytkownika oraz schemat bazy danych. W pracy opisano implementację strony backend i frontend. Pierwsza z nich obejmuje warstwową architekturę z podziałem na kontrolery, serwisy i repozytoria, mechanizm uwierzytelniania oparty na tokenach JWT, system autoryzacji z podziałem na role oraz obsługę transakcji z blokadami pesymistycznymi. W zakresie warstwy frontend opisano m. in. bibliotekę Material UI, mechanizm zarządzania stanem TanStack Query oraz system tras chronionych.

Słowa kluczowe:

aplikacja internetowa, system ERP, Spring Boot, React, TypeScript, PostgreSQL, architektura klient–serwer, JWT, REST API

ABSTRACT

Authors: Igor Kohsin, Oskar Krupa

Supervisor: dr hab. Ewa Ratajczak-Ropel, prof. UMG

Title: Implementation of a Web-Based Gastronomy Business Management System Using Spring and React

Web applications currently form the foundation of both modern business and everyday life. This thesis addresses the development of an ERP-class (Enterprise Resource Planning) system designed for gastronomy establishment managers, enabling data centralization, process automation, and improved enterprise management.

The objective of this work was to design and implement a web-based system supporting inventory management, employee management, transactions, statistics, and financial reporting. The system, named GastroM, was built using a client–server architecture with the Spring Boot framework on the backend and the React library with TypeScript on the frontend. The data layer is based on the PostgreSQL relational database.

The thesis consists of four chapters. The theoretical part covers the fundamentals of web application development. The practical part describes the design and implementation of the GastroM system, presenting, among other things, the analysis of functional and non-functional requirements, the user interface design, and the database schema. The thesis describes the implementation of both the backend and frontend layers. The former encompasses a layered architecture divided into controllers, services, and repositories, a JWT-based authentication mechanism, a role-based authorization system, and transaction handling with pessimistic locking. Regarding the frontend layer, the thesis covers, among other topics, the Material UI library, the TanStack Query state management mechanism, and a protected route system.

Keywords:

web application, ERP system, Spring Boot, React, TypeScript, PostgreSQL, client–server architecture, JWT, REST API